

Appunti del corso di **LiP**

G. Michele Pinna

Anno Accademico 2025-2026 (Versione 1.0)

Indice

1	Macchine Astratte e Interpreti	7
1.1	Paradigmi di programmazione	7
1.2	Metodi per dare la semantica	8
1.3	Macchine Astratte	8
1.3.1	Come implementare un linguaggio	9
1.3.2	Implementare un linguaggio: interpretazione	10
1.3.3	Implementare un linguaggio: compilazione.	10
1.3.4	Implementare un linguaggio: la Macchina Intermedia	11
1.3.5	Valutazione parziale	12
1.4	Semantica operativa	12
1.4.1	Nomi	12
1.4.2	Oggetti denotabili	13
1.5	Semantica operativa delle espressioni aritmetiche	14
1.6	Ambiente	15
1.6.1	Ambiente: definizione	15
1.6.2	Ambiente: dichiarazione	15
1.6.3	Binding e scope	16
1.6.4	Ambiente: come si organizza	16
1.6.5	Tipi d'ambiente e operazioni	17
1.6.6	Scope statico vs scope dinamico	19
1.6.7	Ambiente locale dinamico	19
1.6.8	Ambiente locale statico	19
1.7	Memoria	20
1.7.1	Allocazione statica	20
1.7.2	Allocazione dinamica: la pila	20
1.7.3	Ambiente e pila degli RdA	21
1.7.4	Implementazione dello scope statico	22
2	Un linguaggio funzionale	23
2.1	Sintassi	23
2.2	Scope Dinamico: implementazione	23
2.2.1	I valori e l'ambiente	23
2.2.2	Semantica Operazionale Big-Step: la relazione $\rightarrow$	23
2.2.3	SECD	24

2.2.4	La relazione $\mapsto$	25
2.2.5	Semantica Operazionale Small-Step: la relazione $\rightsquigarrow$	25
2.2.6	Relazioni tra le semantiche	26
2.3	Scope Statico: implementazione	27
2.3.1	Il linguaggio	27
2.3.2	I valori e l'ambiente	28
2.3.3	Semantica Operazionale Big-Step: la relazione $\rightarrow_{st}$	28
2.3.4	La relazione $\mapsto_{st}$	29
2.3.5	Semantica Operazionale Small-Step: la relazione $\rightsquigarrow_{st}$	30
2.3.6	Relazioni tra le semantiche	31
3	Un linguaggio imperativo	33
3.1	Sintassi	33
3.1.1	Espressioni aritmetiche	33
3.1.2	Espressioni booleane	33
3.1.3	Comandi	33
3.2	Semantica del linguaggio	34
3.2.1	La relazione $\longrightarrow$ per le espressioni aritmetiche	34
3.2.2	La relazione $\rightsquigarrow$ per le espressioni booleane	34
3.2.3	La relazione $\Longrightarrow$ per i comandi	34
3.2.4	La relazione $\rightsquigarrow$ per i comandi	36
4	Un linguaggio dichiarativo	37
4.1	Dichiarazioni, procedure e blocchi	37
4.2	Semantica delle dichiarazioni e dei blocchi	38
4.2.1	Passaggio dei parametri	38
4.2.2	La small step semantics	39
4.3	Side Effects	40
4.4	Scope Statico	40
4.5	Procedure come parametri	41
4.5.1	Scope Statico	41
4.5.2	Scope Dinamico	41
4.6	Procedure come risultato	41
4.7	Funzioni come espressioni e side effects	42
5	Dati	43
5.1	Dati e Tipi di Dato	43
5.1.1	Sistema di tipi	43
5.1.2	Type safeness	44
5.1.3	Tipi di dato base	44
5.1.4	Tipi composti	45
5.1.5	Sistema di tipi, equivalenza, compatibilità, polimorfismo e regole	50
5.2	Polimorfismo	52
5.3	Sicurezza (rispetto ai tipi)	53

5.3.1	Inferenza di tipo	53
5.4	Astrarre sui dati	55
6	Lexer and parser	59
6.1	Linguaggi	59
6.2	Linguaggi regolari	60
6.2.1	Espressioni regolari	60
6.2.2	Automi a stati finiti	62
6.2.3	Proprietà di chiusura dei linguaggi regolari	63
6.2.4	Pumping Lemma per linguaggi regolari	64
6.3	Linguaggi context-free	64
6.3.1	Grammatiche	64
6.3.2	Derivazioni left-most, right-most e ambiguità	67
6.3.3	Ambiguità	67
6.3.4	Push Down Automata	68
6.3.5	Rappresentazione grafica di un PDA	71
6.3.6	Relazione tra push down automata e grammatiche context-free	71
6.3.7	Determinismo nei push down automata	72
6.3.8	Forme normali per le grammatiche context-free	72
6.3.9	Operazioni sui linguaggi context free	74
6.3.10	Pumping lemma per linguaggi context-free	75
6.3.11	Gerarchia di linguaggi	76
6.4	Linguaggi context-free deterministici	76
6.5	Lexer	77
6.6	Parser	77
6.6.1	Parser Top Down	77
6.6.2	Parser Bottom-Up	80

Capitolo 1

Macchine Astratte e Interpreti

Formalizzare la semantica di un linguaggio di programmazione serve a definire con esattezza *cosa* fa un programma scritto in quel linguaggio. La semantica serve a poter scrivere come interpretare i costrutti del linguaggio, e quindi anche a sviluppare tecniche di ottimizzazione, di debugging, oppure fare verifiche di correttezza, fare traduzione fra formalismi diversi, e molto altro. Un interprete per un linguaggio riceve un programma e realizza in qualche modo la semantica. La semantica infine ci serve per realizzare la *macchina astratta*, che definiremo un po' più formalmente in seguito, ma che intuitivamente è la macchina che esegue programmi scritti in quel linguaggio.

La definizione semi-formale della semantica fa parte dello standard di definizione di un linguaggio di programmazione a partire dagli anni '80, quando il funzionamento dei costrutti viene descritto sempre più formalmente, e questo porta anche a vedere che la descrizione di come devono funzionare i costrutti ha bisogno di un approccio formale.

Prima di discutere più formalmente cosa è la *semantica* di un linguaggio di programmazione, e come questa si dà, diciamo cosa è un *paradigma* di programmazione.

1.1 Paradigmi di programmazione

Un *paradigma* di programmazione è uno un insieme di strumenti concettuali forniti da un linguaggio di programmazione per la stesura del codice sorgente di un programma, e definisce il modo in cui il programmatore concepisce e percepisce il programma stesso, e quindi anche la sua esecuzione.

I diversi paradigmi si differenziano per i concetti e le astrazioni usate per rappresentare gli elementi di un programma, come ad esempio le funzioni, gli oggetti, le variabili o i vincoli, e anche per i procedimenti usati per l'esecuzione delle procedure di elaborazione dei dati, come l'assegnamento, l'iterazione, la ricorsione, il data flow, l'applicazione di funzioni.

Facciamo un elenco di paradigmi di programmazione, senza la pretesa di esaurirli tutti.

- **IMPERATIVO:** Nel paradigma imperativo è il programmatore a stabilire quale sarà l'istruzione successiva. Quindi è la scrittura del programma stesso a dire quale sarà la sua esecuzione. Il modello di base è quello dei primi linguaggi di programmazione, come il FORTRAN, quindi spesso ci si concentra sull'algoritmo visto come sequenza di passi. Un passo di computazione in generale modifica lo *stato* (concetto che vedremo meglio dopo).
- **FUNZIONALE:** in questo paradigma il programmatore definisce quali sono le funzioni e come queste vengono invocate ed usate. Il focus non è tanto sull'algoritmo come sequenza di passi ma su quali sono le funzioni per risolvere il problema e quali risultati producono. Il modello è il λ -calcolo. Un passo di di computazione in generale restituisce un valore.

- **PROCEDURALE:** In questo paradigma si prova a coniugare la parte imperativa con la definizione di funzioni, che in questo paradigma vengono chiamate procedure. Un passo di computazioni in generale ha un effetto sullo stato ma anche, potenzialmente, restituisce un valore. I linguaggi come PASCAL o ALGOL possono essere presi come linguaggi propri di questo paradigma.
- **OBJECT ORIENTED:** In questo paradigma le varie parti di programma interagiscono scambiandosi messaggi. Sono quindi questi messaggi, scambiato tra parti di programma che sono visti come oggetti medesimi. Il linguaggio di programmazione paradigmatico è lo SMALLTALK, e la sua precisa semantica è quella algebrica.
- **LOGICO:** la logica, in particolare le clausole di Horn, sono alla base di questo paradigma di programmazione. Il linguaggio proprio di questo paradigma è il PROLOG.

1.2 Metodi per dare la semantica

Ci sono vari metodi per definire la semantica di un linguaggio di programmazione, che qui elenchiamo, cercando di evidenziarne le caratteristiche salienti, di nuovo senza alcuna pretesa di completezza.

- **ALGEBRICA:** Nasce agli inizi anni degli 70, si parte dai tipi di dato che formano un'algebra (spesso detta multi-sorted). La sintassi è data dalla cosiddetta *term algebra*, le operazioni nelle algebre semantiche sono definite tramite *equazioni*, e la semantica è l'omeomorfismo più generale dalla term algebra alle algebre semantiche. Concetti come tipo di dato astratto (moduli, classi, etc) sono ben descritti da questa semantica.
- **ASSIOMATICA:** Tra le prime ad essere formalizzata, caratterizza gli *stati* (proprietà), e per ogni costrutto si cerca di capire come è la *trasformazione* indotta nello stato formalizzata come proprietà dello stato e si basa su una descrizione (più o meno formale) di cosa fa un costrutto. Tale semantica è molto indicata per la verifica di *correttezza* di un programma, e quindi per lo sviluppo di programmi corretti.
- **OPERAZIONALE:** Ottenuta definendo un interprete del linguaggio su di una macchina ospite le cui componenti sono descritte in modo matematico. Lo *stato* descrive complessivamente il sistema, e la *transizione* specifica un passo di computazione che modifica lo stato e/o il programma. Una computazione è una sequenza di transizioni. È utile soprattutto perché fornisce direttamente un modello di implementazione
- **DENOTAZIONALE:** Formalizza quale è la trasformazione sugli stati indotta dal programma, sostanzialmente una funzione dagli input agli output di un programma, ed è focalizzata su quale è la trasformazione: *denoto* = esprimo con chiarezza, indico e non su come tale trasformazione avviene, e poggia su basi matematiche sostanzialmente sviluppate negli anni 70.

1.3 Macchine Astratte

La formalizzazione del significato dei vari costrutti di un linguaggio di programmazione ci serve per poter parlare della *macchina astratta*. L'idea è molto semplice: è una macchina che ha il linguaggio di programmazione come *codice macchina*. Diamone una definizione:

Definizione 1.1 Supponiamo sia dato il linguaggio di programmazione $\mathcal{L}$. Una macchina astratta per $\mathcal{L}$, che indichiamo con $\mathbf{M}_{\mathcal{L}}$, è un qualsiasi insieme di strutture dati ed algoritmi che permettono di memorizzare ed eseguire programmi scritti in $\mathcal{L}$.

Se non ci interessa precisare il linguaggio, quindi siamo principalmente interessati alle strutture dati e agli algoritmi di una macchina astratta, scriviamo solo $\mathbf{M}$, omettendo il pedice che indica il linguaggio di programmazione. La definizione suggerisce che di una macchina astratta dobbiamo precisare come memorizziamo e come eseguiamo i programmi, e su come si eseguono i programmi abbiamo già visto la semantica, che però usa la nozione di stato o ambiente.

Proviamo a precisare cosa c'è nell'insieme di strutture dati ed algoritmi. Il prossimo elenco contiene alcune delle componenti che abbiamo nell'insieme.

- strutture dati ed algoritmi per la memoria: come realizziamo la *memorizzazione*;
- strutture dati ed algoritmi per un interprete, come *eseguiamo* le istruzioni;
- strutture dati ed algoritmi per il controllo, come troviamo le istruzioni da *eseguire*;
- strutture dati ed algoritmi per le operazioni primitive, come realizziamo le *operazioni* di base.

La componente principale (ed è anche quella su cui ci concentriamo) è l'*interprete*. Un interprete serve ad eseguire un costrutto, quindi lo deve recuperare, capire cosa fa (decodificarlo), trovare eventuali operandi, eseguirlo ed eventualmente *comunicare* i risultati. Per poter eseguire il costrutto ha bisogno di strutture dati ed algoritmi per realizzare le varie operazioni.

Partiamo da una macchina astratta $\mathbf{M}$, quindi di un insieme di algoritmi e strutture dati per l'esecuzione di programmi memorizzati in un certo linguaggio.

Definizione 1.2 *Data una macchina astratta $\mathbf{M}$, il linguaggio compreso dall'interprete di $\mathbf{M}$ è detto linguaggio macchina di $\mathbf{M}$ e lo indichiamo con $\mathcal{L}_{\mathbf{M}}$*

I programmi sono particolari dati su cui opera l'interprete ed ai componenti di $\mathbf{M}$ corrispondono componenti di $\mathcal{L}_{\mathbf{M}}$, come i tipi di dato primitivi, o i costrutti di controllo (sia per l'ordine di esecuzione, che per l'acquisizione e il trasferimento dati).

Un esempio molto concreto è la macchina *hardware* (astratta). Le componenti di una macchina hardware astratta sono: l'interprete, la memoria, quindi i registri e le celle di memoria, dove possiamo anche memorizzare i programmi, ci sono dei registri particolari con un significato, come il *program counter* o lo *stack pointer*, ed infine abbiamo la ALU.

1.3.1 Come implementare un linguaggio

Partiamo da una macchina astratta $\mathbf{M}$. Le sue componenti sono realizzate con strutture dati e algoritmi *realizzati* nel linguaggio macchina di una macchina già esistente che è la *macchina ospite* $\mathbf{M}_O$. Le componenti possono essere realizzate in *hardware*, quindi vale che $\mathbf{M} = \mathbf{M}_O$ oppure possono essere simulazione in *software*, quindi $\mathbf{M} \neq \mathbf{M}_O$ e l'interprete di $\mathbf{M}$ può coincidere con quello di $\mathbf{M}_O$ ($\mathbf{M}$ è una *estensione* di $\mathbf{M}_O$), oppure l'interprete può essere diverso da quello di $\mathbf{M}_O$ e quindi $\mathbf{M}$ è una macchina astratta *realizzata* su $\mathbf{M}_O$ in modo *interpretativo*.

Per poter dire come realizziamo una macchina astratta, ricordiamo che una funzione $f : A \rightarrow B$ è una corrispondenza che associa elementi da B ad elementi di A , e che funzione significa che ad ogni elemento di A associamo un elemento di B (non il vice versa). Una funzione è detta *parziale* se non è definita per tutti gli input.

Partiamo da un linguaggio $\mathcal{L}$ che vogliamo *implementare*, quindi dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Ricordiamo che il linguaggio $\mathcal{L}$ ha come macchina astratta $\mathbf{M}_{\mathcal{L}}$ e che la macchina astratta $\mathbf{M}$ ha come linguaggio macchina $\mathcal{L}_{\mathbf{M}}$.

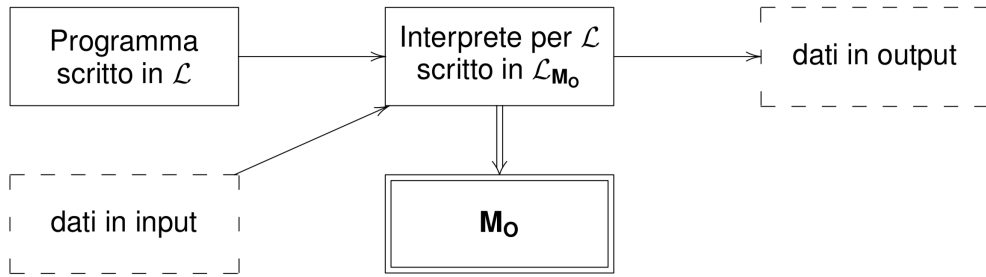
L'implementazione di $\mathcal{L}$ è la realizzazione di $\mathbf{M}_{\mathcal{L}}$ su una macchina ospite $\mathbf{M}_{\mathbf{O}}$, e la macchina ospite $\mathbf{M}_{\mathbf{O}}$ ha il suo linguaggio macchina $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$.

Un programma scritto in $\mathcal{L}$ è una funzione parziale dagli *input* agli *output*, che supponiamo di non distinguere, e che denotiamo con $\mathcal{D}$. Un programma realizza una funzione parziale $\mathcal{P}^{\mathcal{L}} : \mathcal{D} \rightarrow \mathcal{D}$ (parziale perché potrebbe non essere sempre definita per tutti gli input, ad esempio un programma che va in *loop*).

Partiamo da un linguaggio $\mathcal{L}$ che vogliamo *implementare*, e quindi dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Abbiamo la macchina ospite $\mathbf{M}_{\mathbf{O}}$ già realizzata. Scriviamo un interprete per $\mathcal{L}$ usando il linguaggio $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$, e quindi realizziamo $\mathbf{M}_{\mathcal{L}}$ usando $\mathbf{M}_{\mathbf{O}}$ ed il suo linguaggio macchina.

1.3.2 Implementare un linguaggio: interpretazione

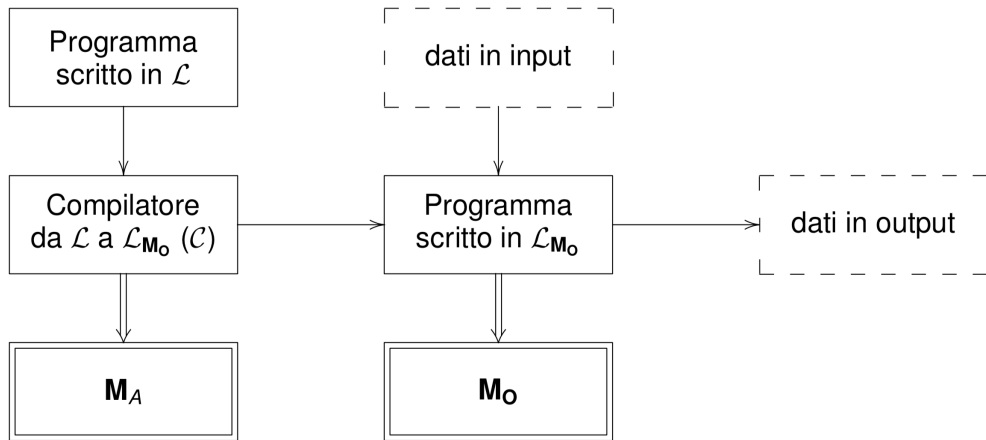
Prima di formalizzare cosa è un'interpretazione vediamo uno schema.



Definizione 1.3 Un interprete per il linguaggio $\mathcal{L}$, scritto nel linguaggio $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$, è un programma $\mathcal{I}$ che realizza una funzione parziale $\mathcal{I} : \text{Prog}^{\mathcal{L}} \times \mathcal{D} \rightarrow \mathcal{D}$ tale che $\mathcal{I}(\mathcal{P}, \text{Input}) = \mathcal{P}(\text{Input})$, dove $\mathcal{P}$ è un programma scritto in $\mathcal{L}$ e $\text{Prog}^{\mathcal{L}}$ è l'insieme dei programmi scritti in $\mathcal{L}$.

1.3.3 Implementare un linguaggio: compilazione.

Partiamo da un linguaggio $\mathcal{L}$ che vogliamo *implementare*. Quindi *in teoria* dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Abbiamo la macchina ospite $\mathbf{M}_{\mathbf{O}}$ già realizzata. Scriviamo un programma che *traduce* ogni programma scritto in $\mathcal{L}$ in uno funzionalmente equivalente scritto in $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$. Non realizziamo $\mathbf{M}_{\mathcal{L}}$.



Definizione 1.4 Un compilatore da $\mathcal{L}$ a $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$ è un programma $\mathcal{C}$ che realizza una funzione totale $\mathcal{C} : \text{Prog}^{\mathcal{L}} \rightarrow \text{Prog}^{\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}}$ tale che, dato un programma $\mathcal{P} \in \text{Prog}^{\mathcal{L}}$, se $\mathcal{C}(\mathcal{P}) = \mathcal{P}_c$, allora per ogni input per $\mathcal{P}$ vale che $\mathcal{P}(\text{Input}) = \mathcal{P}_c(\text{Input})$.

Questa traduzione è chiamata *compilazione*.

Di solito abbiamo che $\mathbf{M}_A = \mathbf{M}_O$ ma potrebbe anche non essere così, non è detto che $\mathcal{C}$ sia scritto in $\mathcal{L}_{\mathbf{M}_O}$.

1.3.4 Implementare un linguaggio: la Macchina Intermedia

Partiamo da un linguaggio $\mathcal{L}$ che vogliamo *implementare*, quindi *in teoria* dobbiamo *realizzare* la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Abbiamo un linguaggio *intermedio* $\mathcal{L}_I$ (che vogliamo *implementare*), e quindi *in teoria* dobbiamo *realizzare* la sua macchina astratta $\mathbf{M}_{\mathcal{L}_I}$. Infine abbiamo la macchina ospite $\mathbf{M}_O$ che è già realizzata.

Combiniamo i due metodi:

1. traduciamo i programmi di $\mathcal{L}$ in programmi di $\mathcal{L}_I$
2. poi interpretiamo i programmi di $\mathcal{L}_I$
3. realizziamo la macchina astratta $\mathbf{M}_{\mathcal{L}_I}$

I due casi limite ricadono in questa visione, infatti:

- interpretazione pura: $\mathbf{M}_{\mathcal{L}} = \mathbf{M}_{\mathcal{L}_I}$ e abbiamo un'*implementazione* puramente *interpretativa*
- compilazione pura: $\mathbf{M}_O = \mathbf{M}_{\mathcal{L}_I}$ e abbiamo un'*implementazione* puramente *compilativa*
- se $\mathbf{M}_O \neq \mathbf{M}_{\mathcal{L}_I}$ e $\mathbf{M}_{\mathcal{L}} \neq \mathbf{M}_{\mathcal{L}_I}$ allora:
 - se l'interprete di $\mathbf{M}_{\mathcal{L}_I}$ è sostanzialmente diverso da quello di $\mathbf{M}_O$: abbiamo un'*implementazione di tipo interpretativo*
non siamo nel caso della interpretazione pura perché ci sono, in $\mathcal{L}_I$, costrutti che hanno un corrispettivo diretto in $\mathcal{L}_{\mathbf{M}_O}$: non tutti i costrutti di $\mathcal{L}$ devono essere simulati
 - se l'interprete di $\mathbf{M}_{\mathcal{L}_I}$ è sostanzialmente uguale a quello di $\mathbf{M}_O$: abbiamo un'*implementazione di tipo compilativo*
la macchina $\mathbf{M}_O$ viene estesa con alcune *funzionalità* che servono per la realizzazione di costrutti di $\mathcal{L}$: il *supporto a tempo d'esecuzione* (**rts**: run time support)

Di solito esiste sempre una macchina intermedia.

Un esempio noto a tutti: la *Java Virtual Machine* (JVM). Un programma Java viene compilato in un altro linguaggio (*bytecode*) che è il linguaggio macchina di JVM. La JVM viene realizzata in modo interpretativo sulla macchina ospite: il vostro PC (sistema operativo + chipset).

Cosa si cerca di fare?

- tradurre in un codice intermedio un linguaggio ad alto livello
- codice intermedio possibilmente ad alto livello (un esempio? il C)
- interpretazione del codice intermedio, in generale cercando di utilizzare lo stesso interprete della macchina ospite

1.3.5 Valutazione parziale

Oltre alla compilazione (traduzione di un programma scritto in un linguaggio in un programma scritto in un altro linguaggio), abbiamo altre tecniche di trasformazione di un programma. $P(X, Y)$ è un programma P che elabora prima i dati X e poi, a seconda del risultato, i dati Y . Se i dati X sono *comuni* a molte esecuzioni, trasformo $P(X, Y)$ in un programma $P'(Y)$ fissando i dati X . Ovviamente $P'(Y)$ è più veloce.

Definizione 1.5 $\mathcal{P}eval_{\mathcal{L}} : (Prog^{\mathcal{L}} \times \mathcal{D}) \rightarrow Prog^{\mathcal{L}}$ è tale che, dato un programma P scritto in $\mathcal{L}$ e che opera su due argomenti, allora $\mathcal{P}eval_{\mathcal{L}}(P, D) = P'$ tale che, qualsiasi sia un input Y , abbiamo che $\mathcal{I}_{\mathcal{L}}(P(D, Y)) = \mathcal{I}_{\mathcal{L}}(P'(Y))$ dove $\mathcal{I}_{\mathcal{L}}$ è l'interprete di $\mathcal{L}$.

Partendo dalla valutazione parziale è interessante osservare che se $\mathcal{P}eval_{\mathcal{L}}$ lo applichiamo ad un interprete scritto in un linguaggio $\mathcal{L}$ e come secondo parametro passiamo il programma da interpretare, scritto in generale in un linguaggio $\mathcal{L}'$, quello che otteniamo è precisamente il programma compilato. Quindi $\mathcal{C} : Prog^{\mathcal{L}'} \rightarrow Prog^{\mathcal{L}}$ applicato al programma P è il programma scritto in $\mathcal{L}'$, altro non è che $\mathcal{P}eval_{\mathcal{L}}$ applicata all'interprete $\mathcal{I}$, ora scritto in $\mathcal{L}$ e a P . Questo giustifica ulteriormente la necessità di sapere realizzare l'interprete, dato che il compilatore lo si ottiene dall'interprete medesimo. Questa è nota come seconda proiezione di Futamura.

1.4 Semantica operativa

La semantica operativa viene in generale formalizzata definendo come interpretare ogni costrutto del linguaggio in uno *stato* generico della macchina astratta che implementa il linguaggio. Daremo quindi le regole dicendo, per ogni costrutto e ogni stato, quale è l'effetto dell'esecuzione di quel costrutto.

Possiamo definirla in modo formale, guidati dalla sintassi di un linguaggio di programmazione, o meglio dalla sintassi dei vari costrutti. Seguiamo il cosiddetto approccio SOS (Structured Operational Semantics). Le regole sono date come regole della seguente forma

$$\frac{\text{premesse}}{\text{conclusione}}$$

Per prima cosa dobbiamo definire cosa è lo *stato* rispetto al quale *valutiamo* (eseguiamo) un costrutto. Per questo abbiamo bisogno prima di tutto di formalizzare cosa è un *nome*.

1.4.1 Nomi

In un linguaggio di programmazione abbiamo, in generale, dei *nomi* che distinguiamo dalle *keywords* (parole chiave) che sono proprie del linguaggio, mentre i nomi ai quali siamo interessati sono quelli definiti dal programmatore. Consideriamo il frammento di programma

```
int pippo (x : int) {
  while x > 10 do x = x - 2;
}
```

In questo frammento ci sono delle *keywords*, ad esempio `int`, `while`, `';`' e altre, ma ci sono anche dei *identificatori* come `pippo` e `x` oppure altre sequenze di caratteri come `10` e `2` che indicano oggetti a cui non bisogna dare particolare interpretazione.

La grammatica con cui vengono generati i nomi non ci interessa precisarla, anche se ci possono essere delle regole particolari per capire cosa è associato a quel nome.

Definizione 1.6 *Dato un linguaggio di programmazione $\mathcal{L}$, con $\text{Name}_{\mathcal{L}}$ indichiamo l'insieme dei nomi di quel linguaggio.*

Questa definizione ci servirà in seguito. Se consideriamo “**var** **pip**” è una sequenza di caratteri dove “**var**” è una keyword e “**pip**” un identificatore, similmente “**const** **pi** = 3.14” è una sequenza di caratteri dove “**const**” è una keyword, “**pi**” un identificatore e poi la costante 3.14.

Quindi nei linguaggi di programmazione i nomi sono spesso gli *identificatori* (token alfa-numeric), ma sono nomi anche le *keywords* o simboli come +, :=, ++, ecc., che non sono identificatori ma fanno lo stesso parte del linguaggio.

Un nome serve ad indicare in generale un oggetto, che è l'oggetto *denotato* da quel nome. I nomi sono gli oggetti simbolici più facili da ricordare (buona regola di programmazione) e, particolarmente importante, permettono l'astrazione (sia sui dati che sul controllo). Ricordiamo che astrarre significa spesso dimenticare dettagli non importanti, che però in qualche forma sono presenti. Il “non importanti” significa che su quelli non ci concentriamo, mentre ci concentriamo sull'uso che si fa dell'astrazione.

1.4.2 Oggetti denotabili

Diamone una definizione molto generale.

Definizione 1.7 *Un oggetto denotabile è un oggetto a cui possiamo dare un nome.*

Osserviamo che non necessariamente diamo nome a tutto, ad esempio, in generale, non diamo nomi alla memoria.

Ci sono i nomi definiti dall'utente come le variabili, parametri formali, le procedure o funzioni, i tipi definiti dall'utente, le etichette, i moduli, le costanti definite dall'utente e simili, mentre ci sono i nomi definiti dal linguaggio come i tipi primitivi, le operazioni primitive, le costanti predefinite e così via.

Dato un programma p in un qualsiasi linguaggio di programmazione $\mathcal{L}$, con $\text{Name}(p)$ indichiamo l'insieme dei *nomi* che compaiono nel programma, quindi $\text{Name}(p) \subseteq \text{Name}_{\mathcal{L}}$ che sono tutti i nomi del linguaggio.

Non ne diamo una definizione formale. I valori che possono essere associati ai vari nomi li indichiamo con *Value* e non dipendono dal programma medesimo, e anche in questo caso non ne diamo una definizione formale.

Diamo una prima definizione di stato, che andrà bene per alcune macchine astratte.

Definizione 1.8 *Dato un programma p , lo stato è un'associazione tra $\text{Name}(p)$ e *Value*.*

La definizione è generica, dato che non si specifica precisamente come è fatta l'associazione tra nomi e valori. Quindi, per noi (e per ora), lo stato può essere una sequenza finita di coppie in cui il primo elemento è il nome e questo compare una sola volta, oppure una funzione che può essere parziale, ed in particolare definita solo per i nomi che sono nel programma. Ogni coppia (X, n) ha come primo elemento il nome (in questo caso X) e come secondo elemento il valore (in questo caso v). Come detto più semplicemente vedere lo stato come una funzione dai nomi ($\text{Name}_{\mathcal{L}}$) ai valori associabili ai nomi (*Value*).

La nozione di stato dipende anche molto dal paradigma che consideriamo. Ad esempio, nel paradigma imperativo e in quello procedurale, lo stato ha una parte che chiameremo ambiente (una funzione da nomi ad oggetti che chiameremo *denotabili*), una memoria, che sarà una funzione da locazioni (che assumeremo denotabili) a oggetti memorizzabili, uno heap, che sarà una struttura dati per la memorizzazione delle strutture dati dinamiche, mentre nel paradigma funzionale avremo solo l'ambiente, cioè l'associazione tra nomi e oggetti denotabili.

1.5 Semantica operativa delle espressioni aritmetiche

Consideriamo il semplice linguaggio delle espressioni aritmetiche dove possono apparire delle variabili, il cui valore sarà specificato nello stato. In una espressione aritmetica i nomi saranno le variabili.

La sintassi di queste espressioni è

$$E ::= n \mid X \mid E_0 + E_1 \mid E_0 \times E_1 \mid E_0 - E_1$$

dove n sono i numeri interi ed X sono le *variabili* (cioè nomi che appartengono ad un insieme che chiamiamo *Var*). Il linguaggio generato da questa grammatica lo indichiamo con **AExpr**.

Per valutare un'espressione abbiamo bisogno dello stato, che in questo caso sarà una funzione da *Var* (i possibili nomi che compaiono in una espressione aritmetica) a quelli che sono gli oggetti denotabili, che in questo caso sono un insieme *Value* che contiene gli interi. Lo stato lo indichiamo con σ , quindi $\sigma : \text{Var} \rightarrow \text{Value}$. L'insieme degli stati lo indichiamo con **States**.

Prima di formalizzare la semantica di una espressione, dobbiamo anche dire cosa restituisce, ed in questo caso è semplice, dato che la valutazione di un'espressione (la sua semantica) restituirà un intero.

Diamo ora le regole per definire la relazione $\rightarrow \subseteq \text{AExpr} \times \text{States} \times \text{Value}$ dove abbiamo che se $(E, \sigma, n) \in \rightarrow$ allora la valutazione di E nello stato σ restituisce n .

$$\begin{array}{c} \overline{\langle n, \sigma \rangle \rightarrow n} \qquad \overline{\langle X, \sigma \rangle \rightarrow \sigma(X)} \qquad \frac{\langle E_0, \sigma \rangle \rightarrow n_0 \quad \langle E_1, \sigma \rangle \rightarrow n_1}{\langle E_0 + E_1, \sigma \rangle \rightarrow n = n_0 + n_1} \\[10pt] \frac{\langle E_0, \sigma \rangle \rightarrow n_0 \quad \langle E_1, \sigma \rangle \rightarrow n_1}{\langle E_0 \times E_1, \sigma \rangle \rightarrow n = n_0 \times n_1} \qquad \frac{\langle E_0, \sigma \rangle \rightarrow n_0 \quad \langle E_1, \sigma \rangle \rightarrow n_1}{\langle E_0 - E_1, \sigma \rangle \rightarrow n = n_0 - n_1} \end{array}$$

Vediamo un esempio di come valutare un'espressione. Prendiamo un'espressione: $4 + 5 - 2$. Lo stato in questo caso non influenza la valutazione dato che nell'espressione non appaiono variabili. Le regole usano metavariables che sono $n, E_0, \dots$ e che hanno valori sui vari domini sintattici, e una istanza di una regola si ottiene istanziando queste metavariables a particolari numeri, variabili, espressioni e simili. Per valutare $4 + 5 - 2$ applichiamo prima la regola

$$\frac{\langle E_0, \delta \rangle \rightarrow n_0 \quad \langle E_1, \delta \rangle \rightarrow n_1}{\langle E_0 + E_1, \delta \rangle \rightarrow n = n_0 + n_1}$$

istanziata in questo modo:

$$\frac{\langle 4, \delta \rangle \rightarrow n_0 \quad \langle 5 - 2, \delta \rangle \rightarrow n_1}{\langle 4 + 5 - 2, \delta \rangle \rightarrow n = n_0 + n_1}$$

Le metavariables sono E_0 ed E_1 , e E_0 è 4, E_1 è $5 - 2$, le altre per ora le lasciamo non specificate.

Sappiamo che dobbiamo valutare ora 4 e $5 - 2$. Valutiamo 4. La regola è

$$\overline{\langle n, \delta \rangle \rightarrow n}$$

che istanziata diventa

$$\overline{\langle 4, \delta \rangle \rightarrow 4}$$

Valutiamo $5 - 2$, la regola da istanziare è ovvia, e la sua istanziazione è

$$\frac{\langle 5, \delta \rangle \rightarrow n_0 \quad \langle 2, \delta \rangle \rightarrow n_1}{\langle 5 - 2, \delta \rangle \rightarrow n = n_0 - n_1}$$

Valutiamo 5 e 2. Abbiamo

$$\overline{\langle 5, \delta \rangle \rightarrow 5} \quad \text{e} \quad \overline{\langle 2, \delta \rangle \rightarrow 2}$$

ottenendo

$$\frac{\langle 5, \delta \rangle \rightarrow 5 \quad \langle 2, \delta \rangle \rightarrow 2}{\langle 5 - 2, \delta \rangle \rightarrow 3 = 5 - 2}$$

ed infine

$$\frac{\langle 4, \delta \rangle \rightarrow 4 \quad \langle 5 - 2, \delta \rangle \rightarrow 3}{\langle 4 + 5 - 2, \delta \rangle \rightarrow 7 = 4 + 3}$$

Facciamo alcune considerazioni. La grammatica che abbiamo visto ha il problema dell'*ambiguità*, prendiamone una che ha le stesse derivazioni (l'idea è dare la grammatica che genera l'albero di derivazione). La sintassi (astratta) è

$$E ::= n \mid val(X) \mid piu(E_0, E_1) \mid per(E_0, E_1) \mid meno(E_0, E_1)$$

che in fondo altro non è che la sintassi astratta e sarà il risultato del parsing dell'espressione da valutare. Le regole ovviamente non cambiano.

$$\begin{array}{ccc} \overline{\langle n, \delta \rangle \rightarrow n} & \overline{\langle val(X), \delta \rangle \rightarrow \delta(X)} & \frac{\langle E_0, \delta \rangle \rightarrow n_0 \quad \langle E_1, \delta \rangle \rightarrow n_1}{\langle piu(E_0, E_1), \delta \rangle \rightarrow n = n_0 + n_1} \\ \frac{\langle E_0, \delta \rangle \rightarrow n_0 \quad \langle E_1, \delta \rangle \rightarrow n_1}{\langle per(E_0, E_1), \delta \rangle \rightarrow n = n_0 \times n_1} & & \frac{\langle E_0, \delta \rangle \rightarrow n_0 \quad \langle E_1, \delta \rangle \rightarrow n_1}{\langle meno(E_0, E_1), \delta \rangle \rightarrow n = n_0 - n_1} \end{array}$$

1.6 Ambiente

Abbiamo visto che una componente dello stato è l'associazione tra nomi ed oggetti denotabili, che nel caso delle espressioni aritmetiche sono anche i valori. Precisiamo meglio cosa è l'ambiente.

1.6.1 Ambiente: definizione

Possiamo ora definire cosa è l'*ambiente*.

Definizione 1.9 *L'ambiente è un insieme di associazioni tra nomi ed oggetti denotabili esistenti a tempo d'esecuzione (run-time) in uno specifico punto del programma ed in un specifico momento dell'esecuzione*

L'ambiente è anche chiamato *referencing environment*.

1.6.2 Ambiente: dichiarazione

Ci domandiamo ora come formare (o inserire) un'associazione tra nome ed oggetto denotabile in un ambiente. In generale lo facciamo esplicitamente con una *dichiarazione*.

Definizione 1.10 *La dichiarazione è un meccanismo, implicito o esplicito, con cui si crea un'associazione nell'ambiente.*

Il modo esplicito è quello che ci immaginiamo: all'interno del programma troviamo parti come `int x = 3`, `int f ()` e simili, e quando si incontrano queste dichiarazioni si crea un'associazione nell'ambiente tra il nome e l'oggetto denotato, che in generale viene creato al momento della dichiarazione. Il modo implicito lo abbiamo principalmente quando abbiamo programmi divisi in moduli, in questo caso l'associazione si crea al momento dell'uso di un nome, che è dichiarato in un'interfaccia.

1.6.3 Binding e scope

Collegate alla nozione di ambiente abbiamo varie nozioni che è opportuno sottolineare. La prima è quella ovvia di *binding*, ossia di legame, o anche associazione, che è quella che abbiamo tra nomi ed oggetti denotati, la seconda, altrettanto rilevante, è quella di *binding time*, che è il momento in cui avviene il *binding*, ed infine abbiamo la nozione di *scope*, che è la parte di programma in cui una particolare associazione tra nome e oggetto denotato, cioè un *binding* è valida. Lo scope ha come sinomini visibilità, campo d'azione, e portata.

Definizione 1.11 *Dato un programma ed un dato nome, lo scope di una certa associazione tra il nome e l'oggetto denotato da quel nome, è dato dalla parte di programma in cui quell'associazione è nota a tempo d'esecuzione.*

In altre parole lo scope serve per capire come è fatto l'ambiente in un certo punto del programma.

Il momento in cui avviene il legame, cioè il *binding time* ha particolare rilevanza. Può essere *statico*:

- al momento della progettazione del linguaggio: tipi primitivi, nomi per operazioni e costanti predefinite,
- al momento della scrittura del programma: definizione di alcuni nomi, per esempio variabili, funzioni, il cui legame potrà essere completato dopo (definizioni di interfacce),
- al momento, se c'è, della compilazione: il legame di alcuni nomi, ad esempio le variabili globali, può essere determinato in questa fase.

Oppure può essere *dinamico*:

- al momento dell'esecuzione: legame definitivo di tutti i nomi non ancora legati, come le variabili locali ai blocchi

Non va confuso con lo scope: sono due fenomeni diversi.

Ci sono dei fenomeni particolari che è utile sottolineare. Lo stesso nome può denotare oggetti distinti o in *punti* diversi del programma (vedremo che sono in fondo ambienti diversi) o in *momenti* diversi (ad esempio, procedure ricorsive) Oppure nomi diversi possono denotare lo stesso oggetto (*aliasing*).

1.6.4 Ambiente: come si organizza

La nozione centrale per l'organizzazione dell'ambiente è quella di blocco.

Definizione 1.12 *Un blocco è regione testuale del programma, identificata da un segnale di inizio ed uno di fine, che può contenere dichiarazioni locali a quella regione.*

I blocchi sono o *anonimi* (o in-line) che significa che alla regione non è associato alcun nome, oppure possono non essere anonimi e quindi sono in generale associati ad una procedura o funzione. In questo modo il blocco ha un nome, quello della procedura o della funzione. Per identificare i blocchi anonimi

abbiamo bisogno di alcuni *segnali*. In Algol abbiamo `begin ... end`, in C o Java abbiamo le parentesi graffe `{ ... }`, in Lisp abbiamo le parentesi tonde `( ... )` ed in ML invece il `let ... in ....` Per convenzione le dichiarazioni sono subito dopo il segnale di inizio del blocco, in ML sono tra il `let` e l'`in`.

Un blocco può essere anche visto come un modo per combinare una serie di istruzioni facendole vedere all'esterno come un'unica istruzione (con effetti che sono quelli cumulati). Nel caso della sintassi astratta sarà facile capire cosa sta in un blocco, dato avremo un costruttore di blocco. Inoltre il blocco consente la gestione locale dei nomi (quelli dichiarati nel blocco sono locali al blocco). Questo aumenta la chiarezza e si possono scegliere i nomi più adatti e significativi. Inoltre i blocchi consentono di migliorare l'occupazione di memoria e permettono la ricorsione. Infine i blocchi possono essere solo annidati (uno dentro l'altro) e mai parzialmente sovrapposti.

1.6.5 Tipi d'ambiente e operazioni

In effetti c'è solo un ambiente, come la definizione di ambiente suggerisce, ma per convenienza introduciamo delle distinzioni:

- ambiente *locale*: sono le associazioni create all'ingresso nel blocco (variabili locali, parametri formali, ecc.),
- ambiente *non locale*: sono le associazioni create fuori dal blocco ma che sono ancora valide all'interno di quel blocco,
- ambiente *globale*: sono le associazioni che sono comuni a tutti i blocchi,

quindi l'ambiente si suddivide in questi tre tipi. Un ambiente ha sempre la parte locale, mentre può avere la parte globale o non locale. Osserviamo che l'ambiente globale è un caso particolare di ambiente non locale.

Elenchiamo ora quali sono le operazioni che si compiono sull'ambiente.

- *creazione*: si crea un'associazione nome - oggetto denotato (tipicamente in ingresso ad un blocco, quando si incontrano le dichiarazioni),
- *riferimento*: si usa in qualche modo l'oggetto denotato attraverso il suo nome,
- *disattivazione*: un'associazione viene disattivata (ad esempio, in un blocco interno uso un nome usato in un blocco più esterno),
- *riattivazione*: un'associazione disattivata ritorna in *vita* (ad esempio, esco dal blocco interno),
- *distruzione*: l'associazione nome - oggetto denotato viene cancellata definitivamente (esco dal blocco dove l'associazione è stata creata).

Osserviamo che le operazioni sull'ambiente riguardano la creazione, uso e distruzione di un legame, legame tra un nome trovato nel testo del programma ed un oggetto. Fino ad ora abbiamo considerato dell'ambiente solo il legame ed il nome, e tramite il legame ed il nome siamo in grado di accedere o usare l'oggetto denotato. Ci domandiamo quali operazioni si possono compiere su un oggetto.

- la *creazione* dell'oggetto, possibilmente prima o al momento della creazione del legame (per creare un legame devo avere l'oggetto),
- l'*accesso* all'oggetto, facendovi riferimento,

- la *modifica* dell'oggetto, attraverso il nome con cui vi accediamo e con le opportune operazioni che lo *modificano* l'oggetto ma non il nome ed il legame,
- la *distruzione* dell'oggetto.

Una volta creato un nome ed un legame, dove può essere usato? Cioè dove è noto sia il nome che il legame? Abbiamo una regola generale.

Definizione 1.13 *Una dichiarazione locale ad un blocco è visibile in quel blocco e in tutti i blocchi in esso annidati, a meno che non intervenga in tali blocchi una nuova dichiarazione dello stesso nome (che nasconde, o maschera, la precedente).*

Come interpretare la regola in presenza di procedure/funzioni e come interpretarla in presenza di ambiente non locale (e non globale)? Consideriamo il seguente programma (di cui molte parti sono omesse) scritto, come molti, in stile imperativo.

```

{a. int x = 10;
  ...

  b. void uno () {
    c.      x++;
        }
    ...

    ...

    ...
  }

  d. void due () {
    e.      int x = 2;
    f.      uno();
        }
    ...

    g. due();
    ...

    ...
  }

```

Il nome `x` viene definito in due blocchi. Il primo blocco in cui viene dichiarata è un blocco anonimo mentre il secondo blocco è quello con un nome (la procedura `due`). Nell'esecuzione del programma si effettua una chiamata alla procedura `due`. In questo caso quale `x` viene incrementata? Si possono adottare due strategie: la prima è far riferimento al nome dichiarato nel blocco più vicino che racchiude testualmente il blocco dove il nome viene usato, la seconda è fare riferimento al legame attivo al momento della chiamata della procedura/funzione.

- Scope *statico*: si fa riferimento al legame creato nel blocco testualmente più vicino, quindi dato che il blocco più vicino (che è quello in cui si trova la dichiarazione della `uno`) si fa riferimento alla `x` della riga `a`.
- Scope *dinamico*: si fa riferimento al legame attivo, quindi dato che la `uno` viene invocata all'interno della `due` e il blocco attivo è quello della `due`, la `x` è quella dichiarata alla riga `e`.

Sostanzialmente si deve decidere in quale ambiente risolvere il legame, nel caso dello scope *statico* nell'ambiente determinato dalla scrittura del programma, nel caso dello scope *dinamico* nell'ambiente determinato dall'esecuzione del programma.

Quasi tutti i linguaggi di programmazione hanno lo scope statico, dato che lo scope statico soddisfa i principi di *indipendenza* dalla posizione dell'uso del nome: una procedura può essere chiamata in contesti diversi ma importa solo dove è stata dichiarata, e di *indipendenza* dai nomi locali: se si modifica un nome locale non si ripercuote sulla procedura. Lo scope statico consente di *specializzare* le funzioni/procedure, cioè di far sì che una certa variabile sia usata in un contesto diverso da quello in cui era stata dichiarata.

1.6.6 Scope statico vs scope dinamico

Confrontiamo un po' i due approcci allo scope, cercando di evidenziarne i vantaggi e gli svantaggi. Nel caso dello scope statico (detto anche scoping statico, statically scoped, lexical scope) abbiamo

- l'informazione è completa dal testo del programma
- le associazioni sono note a tempo di compilazione
- valgono i principi di indipendenza
- è complesso da implementare (lo vedremo), ma in generale efficiente

Nel caso dello scope dinamico (o scoping dinamico, dynamically scoped), abbiamo invece

- l'informazione è derivata dall'esecuzione
- i programmi con scope dinamico sono meno "leggibili"
- è semplice da implementare, ma meno efficiente

I due tipi di scope differiscono solo in presenza congiunta di ambiente non locale e non globale, quindi certamente con le procedure o le funzioni, ma anche in assenza di ambiente globale che coinvolge tutti i nomi non locali.

1.6.7 Ambiente locale dinamico

Ricordiamo che l'ambiente è sempre locale (può anche essere globale o non locale), e in effetti ci riferiamo a quelle locali, non locali o globali (appartenenti ad un solo blocco, comuni a vari blocchi, comuni a tutti i blocchi). La creazione di una associazione tra nome e oggetto denotato avviene in generale all'ingresso di un blocco, e viene inserita nell'ambiente, mentre la distruzione di una associazione tra nome e oggetto denotato avviene all'uscita dal blocco in cui l'associazione è stata creata e viene eliminata dall'ambiente.

L'associazione è utilizzabile solo all'interno del blocco (a parte eventuali buchi perché siamo entrati in un altro blocco) e se rientriamo nello stesso blocco, eseguiamo nuovamente le dichiarazioni e creiamo *nuove* associazioni. Lo chiamiamo *dinamico* perché ogni volta creiamo nuove associazioni.

1.6.8 Ambiente locale statico

In questo caso non creiamo associazioni quando si entra in un blocco, ma all'ingresso di questo avviene la riattivazione di un'associazione tra nome e oggetto denotato nell'ambiente, e all'uscita del blocco in cui l'associazione è stata attivata, avviene la disattivazione dell'associazione nell'ambiente. Inoltre l'associazione è utilizzabile solo all'interno del blocco, cioè quando è attiva, e se rientriamo nello stesso blocco, non rieseguiamo le dichiarazioni, ma ci limitiamo a riattivare le associazioni precedenti. La domanda è quindi chi esegue le dichiarazioni, e queste sono eseguite in generale o dal compilatore (prima dell'inizio dell'esecuzione) oppure alla prima esecuzione del blocco (o simile).

Nei moderni linguaggi di programmazione l'ambiente locale è dinamico, ma alcuni linguaggi hanno (anche) ambiente locale statico. In particolare nel Fortran dove non ci sono blocchi, questa è la regola per l'ambiente locale dei sottoprogrammi, le diverse attivazioni (chiamate) dello stesso sottoprogramma condividono ambiente e memoria locali, e quindi la chiamata *n*-esima trova lo stato lasciato dalla chiamata (*n-1*)-esima. Nell'Algol, nel PL/I o nel C alcune dichiarazioni locali (a blocchi o sottoprogrammi) possono essere dichiarate statiche (`static`, `own`, ...) e per queste particolari associazioni

l'ambiente è statico, le altre sono trattate con l'ambiente dinamico. In Java le dichiarazioni `static` all'interno di una classe provocano la creazione di ambiente (ed eventuale memoria) che appartengono alla classe e non agli oggetti sue istanze, sono eseguite una sola volta all'atto della esecuzione della dichiarazione di classe e possono costituire uno stato interno comune a tutti gli oggetti della classe.

Perché usare l'ambiente locale statico? Le dichiarazioni statiche sono più efficienti, ma creano comportamenti molto più complessi da comprendere dal punto di vista semantico dato che la presenza di uno stato interno fa sì che la semantica debba tener conto di tutta la sequenza di attivazioni. Queste sono utili se si vogliono definire blocchi (o meglio sottoprogrammi) dotati di stato interno (ad esempio, un generatore di numeri casuali, o un generatore di nomi sempre nuovi, ecc.), inoltre l'implementazione dell'ambiente locale statico è complessa.

1.7 Memoria

Per definire lo stato di una macchina astratta abbiamo bisogno non solo di una funzione che associa ai nomi del programma gli oggetti denotabili, ma anche, a seconda di quale paradigma di programmazione adottiamo, di altre componenti, ed in particolare della *memoria*. La memoria sarà uno spazio dove possiamo memorizzare oggetti (gli oggetti memorizzabili) e a cui in genere accediamo tramite una *locazione* (che spesso immagineremo come un valore denotabile).

La memoria può essere *statica*, cioè la memoria viene allocata a tempo di compilazione (ad esempio, le variabili globali), oppure *dinamica*, ad esempio la memoria viene allocata o su una *pila* (stack), dove tutti gli oggetti sono allocati con politica LIFO, oppure su uno *heap*, dove gli oggetti sono allocati e deallocati in qualsiasi momento (in generale per mezzo di puntatori). Inoltre l'implementazione delle regole di scope è legata all'avere sottoprogrammi e/o meccanismi come blocchi per strutturare l'ambiente (se non li avessimo, la gestione della memoria sarebbe ovviamente sempre statica), non avremmo necessità di realizzare le regole di scope.

1.7.1 Allocazione statica

In questo caso un oggetto ha un *indirizzo assoluto* che è mantenuto per tutta l'esecuzione del programma.

Solitamente sono allocate staticamente le variabili *globali*, le variabili *locali* di sottoprogrammi senza ricorsione, le costanti determinabili a tempo di compilazione e le tabelle usate dal supporto a run-time (per type checking, garbage collection, ecc.). Spesso sono usate zone protette di memoria.

Va osservato che l'allocazione statica ha come conseguenza che non è possibile avere allo stesso tempo incarnazioni diverse della stessa funzione o porzione di programma, quindi la ricorsione non è sostanzialmente implementabile, a parte la ricorsione in codo per motivi che saranno chiari successivamente.

1.7.2 Allocazione dinamica: la pila

Ogni istanza di un sottoprogramma a *run-time* ha una porzione di memoria detta *record di attivazione* (RdA o frame) contenente le informazioni relative alla specifica istanza (compreso l'indirizzo di ritorno). Anche ogni blocco anonimo ha un suo record di attivazione (che è più semplice). La pila (LIFO) è la struttura dati *naturale* per gestire i RdA perché le chiamate di procedura (anche ricorsive) ed i blocchi sono annidati uno dentro l'altro.

Vediamo nel dettaglio come viene gestita la pila dei RdA, precisando prima cosa in generale contiene un generico RdA. Un RdA contiene tutte le informazioni relative alla specifica istanza del blocco in esecuzione, quindi il record d'attivazione di un blocco anonimo ha spazio per i nomi *locali*, quelli cioè

che sono dichiarati nel blocco (e che serviranno a capire cosa è associato a quel nome, quindi l'ambiente locale), eventuale spazio per memorizzare risultati intermedi, cioè quelli che sono i risultati parziali della valutazione di un'espressione complessa (ad esempio $(x + y)/(2 * x) + x * y$ ha bisogno di memorizzare i valori delle sotto espressioni $(x + y)/(2 * x)$ e $x * y$), il riferimento al RdA del blocco (anonimo o no) che *racchiude* il blocco che stiamo eseguendo, cioè quello in cima alla pila degli RdA, e che si chiama in generale *puntatore di catena dinamica*, ed il *puntatore di catena statica*. La catena statica consente di identificare il record d'attivazione che contiene l'ambiente non locale nel caso di scope statico. Il record d'attivazione di un blocco non anonimo, quindi di una procedura o funzione, contiene, oltre a quello che contiene il RdA di un blocco anonimo, anche spazio per i parametri, che fanno parte dell'ambiente locale del blocco, informazioni sull'*indirizzo* di ritorno, e l'*indirizzo* dove mettere il risultato se è una funzione che restituisce un risultato.

Nella gestione della pila dei RdA identifichiamo i passi:

1. *sequenza di chiamata*: è il codice eseguito dal chiamante immediatamente prima della chiamata, in questa fase si prepara il RdA che deve essere messo sulla pila
2. *prologo*: è il codice eseguito all'inizio del blocco (con nome o senza), e qui si *eseguono* le dichiarazioni e se serve si effettuano le associazione per il passaggio dei parametri
3. *epilogo*: è il codice eseguito alla fine del blocco (con nome o senza), che prepara all'effettivo *pop* dalla pila del record d'attivazione,
4. *sequenza di ritorno*: è il codice eseguito dal chiamante immediatamente dopo la chiamata, e coinvolge non solo il levare il RdA dalla cima della pila, ma anche la comunicazione dei risultati e delle modifiche effettuate dal blocco che era in cima alla pila, cioè in esecuzione.

L'indirizzo in memoria di un RdA non può essere noto prima di eseguire un programma, quindi c'è un *Puntatore RdA* (o SP, *stack pointer*) che punta al RdA del blocco attivo e le informazioni contenute in un RdA (nomi locali, risultati intermedi, ecc.) sono accessibili per offset rispetto allo SP. Vale che **indirizzo-info = contenuto(SP) + offset** e l'**offset** è determinabile staticamente.

Come abbiamo sempre un ambiente *locale* abbiamo anche sempre una memoria *locale*, e questa è *allocata* nel RdA. In particolare troviamo tutta la memoria relativa alle strutture dati statiche (la cui dimensione è determinata al momento della compilazione o scrittura del programma) è allocata nel RdA, mentre nel caso di memoria relativa a strutture dati dinamiche, la cui dimensione è determinata dall'esecuzione, nel RdA si alloca solo il modo per arrivarci (un altro puntatore).

1.7.3 Ambiente e pila degli RdA

I record d'attivazione servono anche ad implementare l'*ambiente* (alla memoria ci accederò normalmente con un nome, e quindi devo capire soprattutto come si *implementano* le regole di scope).

Ricordiamo che abbiamo una regole di visibilità che dice

Una dichiarazione locale ad un blocco è visibile in quel blocco e in tutti i blocchi in esso annidati, a meno che non intervenga in tali blocchi una nuova dichiarazione dello stesso nome (che nasconde, o maschera, la precedente).

se una dichiarazione è *locale* ad un blocco, come la troviamo se siamo in un altro blocco? Per usarla non possiamo usare l'offset e l'SP ma dobbiamo *risalire*, all'interno della pila, per trovare il giusto RdA dove la variabile è dichiarata e per questo si possono usare solo i link a disposizione: il puntatore di catena dinamica o il puntatore di catena statica. Abbiamo visto che nel RdA di un blocco non anonimo abbiamo due puntatori. Quale dei due usiamo? Vale prima la pena sottolineare che che

lo scope statico o dinamico hanno senso solo in presenza di ambiente non locale e non globale, dato che l'ambiente globale lo possiamo realizzare all'esterno della pila dei RdA. Il *puntatore di catena statica* è quello usato per realizzare lo scope statico (che abbiamo detto è un po' più complesso da implementare dato che dobbiamo *calcolare* questo puntatore), mentre per realizzare lo scope dinamico basta il puntatore di catena dinamica, che sappiamo calcolare facilmente.

1.7.4 Implementazione dello scope statico

Ricordiamo che nello scope *statico* si fa riferimento al legame creato nel blocco testualmente più vicino, ed al momento dell'esecuzione, come si trova il RdA del blocco *testualmente* più vicino? Devo fare attenzione, dato che tra il RdA *attivo* (quello in cima alla pila) e quello che contiene il non locale al quale sono interessato ci possono essere RdA che contengono come nome locale proprio quello. Usiamo l'*annidamento* dei blocchi.

Definizione 1.14 *Dato un frammento di programma scritto in un linguaggio $\mathcal{L}$ ad alto livello, l'annidamento misura il numero di blocchi/sottoprogrammi aperti e non ancora richiusi all'interno del quale si trova l'occorrenza di un certo nome.*

Ricordiamo che i nomi che usiamo devono essere in *scope*, che significa che devono essere noti.

Per calcolare il puntatore di catena statico abbiamo a disposizione solo il puntatore di catena statico del chiamante, cioè del blocco dove avviene la chiamata alla funzione o da dove si accede ad un nuovo blocco annidato all'interno. Abbiamo due casi:

- il chiamato si trova all'*interno* del chiamante, e qui è tutto facile: il puntatore di catena statica è il puntatore di catena dinamica coincidono, oppure
- il chiamato si trova all'*esterno* del chiamante, quindi dato che il chiamato è un nome noto, lo avremo trovato utilizzando il puntatore di catena statico del chiamante. In che modo? Siamo risaliti lungo la *catena statica* fino a che non abbiamo trovato il nome corretto, e questa *distanza* è nota al momento della scrittura del programma, quindi basta aggiornare il puntatore di catena statica del nuovo RdA in base a questo calcolo.

Capitolo 2

Un linguaggio funzionale

In questo capitolo vedremo la semantica e l'implementazione di un linguaggio funzionale.

2.1 Sintassi

Abbiamo un insieme di costanti $\mathcal{C}$ che sono indicate con $\mathbf{c}$ (le costanti possono essere interi, caratteri, booleani o simili), un insieme Var di variabili che sono indicate con $x, y, \dots, f, g, h, \dots$, e un certo numero di operazioni $\mathbf{op}_i$ che hanno arietà a_i .

I termini del linguaggio sono generati dalla seguente grammatica.

$$t ::= \mathbf{c} \mid x \mid \mathbf{op}_i(t_1, \dots, t_{a_i}) \mid \mathbf{if}(t_1, t_2, t_3) \mid \mathbf{fun}(x, t) \mid \mathbf{apply}(t_1, t_2) \mid \mathbf{let} \ x = t_1 \ \mathbf{in} \ t_2$$

Le parole generate da questa grammatica formano un linguaggio che chiamiamo **Term**. Vale la pena osservare che, dato un programma in questo linguaggio, cioè un termine t , abbiamo che $\mathbf{Name}(t) \subseteq Var$.

2.2 Scope Dinamico: implementazione

L'ambiente è lo *stato* di un linguaggio funzionale. Per capire come si realizza vediamo come definire la semantica di un linguaggio funzionale e quindi sostanzialmente come realizzare la sua macchina astratta.

2.2.1 I valori e l'ambiente

Dobbiamo stabilire quali sono i valori che la valutazione di un termine può restituire. Dato che la valutazione di una costante è la costante medesima, ovviamente l'insieme **Value** contiene $\mathcal{C}$ dove $\mathbf{c}$ è la valutazione (semantica) di $\mathbf{c}$, e contiene anche i termini $\mathbf{fun}(x, t)$. Questo insieme lo indichiamo come **Fun** ed è $\{\mathbf{fun}(x, t) \mid f, x \in Var \text{ e } t \in \mathbf{Term}\}$. Quindi $\mathbf{Value} = \mathcal{C} \cup \mathbf{Fun}$.

Possiamo ora dire cosa è un ambiente per il nostro linguaggio funzionale, che sarà un'associazione tra nomi e valori, quindi una funzione parziale $\delta : Var \rightarrow \mathbf{Value}$. $\mathbf{Env}$ denota l'insieme degli ambienti.

2.2.2 Semantica Operazionale Big-Step: la relazione $\rightarrow$

Diamo prima le regole per la semantica del nostro linguaggio, definiamo quindi una relazione che prende un termine, un ambiente e restituisce un valore, che vedremo essere deterministica, e quindi

dato un termine ed un ambiente restituisce sempre lo stesso valore. Abbiamo che $\rightarrow \subseteq \text{Term} \times \text{Env} \times \text{Value}$.

$$\begin{array}{c}
\frac{}{\langle x, \delta \rangle \rightarrow \delta(x)} \quad \frac{}{\langle c, \delta \rangle \rightarrow c} \quad \frac{}{\langle \text{fun}(x, t), \delta \rangle \rightarrow \text{fun}(x, t)} \\
\\
\frac{\langle t_1, \delta \rangle \rightarrow v_1 \quad \dots \quad \langle t_{a_i}, \delta \rangle \rightarrow v_{a_i}}{\langle \text{op}_i(t_1, \dots, t_{a_i}), \delta \rangle \rightarrow \widetilde{\text{op}}_i(v_1, \dots, v_{a_i})} \\
\\
\frac{\langle t_0, \delta \rangle \rightarrow \text{true} \quad \langle t_1, \delta \rangle \rightarrow v}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow v} \quad \frac{\langle t_0, \delta \rangle \rightarrow \text{false} \quad \langle t_2, \delta \rangle \rightarrow v}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow v} \\
\\
\frac{\langle t_1, \delta \rangle \rightarrow \text{fun}(x, t) \quad \langle t_2, \delta \rangle \rightarrow v' \quad \langle t, \delta[x/v_2] \rangle \rightarrow v}{\langle \text{apply}(t_1, t_2), \delta \rangle \rightarrow v} \\
\\
\frac{\langle t_1, \delta \rangle \rightarrow v_1 \quad \langle t_2, \delta[x/v_1] \rangle \rightarrow v}{\langle \text{let } x = t_1 \text{ in } t_2, \delta \rangle \rightarrow v}
\end{array}$$

Le regole che gestiscono l'ambiente sono quella per l'**apply** e quella per il **let**, ed è opportuno notare che il nuovo ambiente che viene definito per la valutazione del corpo della funzione o del termine del **let**, una volta terminata la valutazione, non sarà più utilizzato. L'ambiente per la valutazione del corpo della funzione o del termine del **let** è stato utilizzato solo localmente a quei termini, e terminata questa valutazione sarà di nuovo *disponibile* l'ambiente precedente.

E importante sottolineare che la relazione è deterministica.

Teorema 2.1 *La relazione $\rightarrow$ è deterministica.*

Questo significa che per ogni termine t ed ogni ambiente δ , se esiste un valore v tale che $(t, \delta, v) \in \rightarrow$, allora se $(t, \delta, v') \in \rightarrow$ vale che $v = v'$.

2.2.3 SECD

Descriviamo ora un'altra semantica per il nostro linguaggio funzionale e per questo ricorriamo ad una macchina astratta, la SECD (Stack, Environment, Control, Dump), che ci consentirà di capire meglio come implementiamo gli ambienti per valutare i termini.

La macchina è una pila di tripla (S, E, C) dove S è una pila di valori, E è un'ambiente, e C è una pila di elementi di controllo. Vediamo per prima cosa quali sono gli elementi di controllo, che sono parole generate dalla seguente grammatica.

$$ce ::= t \mid \text{Prim}(\text{op}_i) \mid \text{App} \mid \text{If} \mid \text{Let}$$

Le $\text{Prim}(\text{op}_i)$ sono la controparte implementativa delle varie op_i sintattiche e $\widetilde{\text{op}}_i$ sono la controparte *semantica* di queste.

Lo *stato iniziale* della macchina è la pila $([], \delta_\perp, t :: []) :: []$ dove $\delta_\perp t$ è la funzione ovunque indefinita. Lo *stato finale* della macchina è una pila tale che non può essere più fatto un passo di computazione (secondo la relazione che vedremo di seguito). La computazione ha successo se la pila finale è della forma $(v :: [], \perp, []) :: []$, con $v \in \text{Value}$.

Tutte le pile che sono macchine SECD le indichiamo con SECD.

2.2.4 La relazione $\mapsto$

Forniamo ora una implementazione delle regole della semantica dando una relazione ($\mapsto$) che collega due SECD, quindi $\mapsto \subseteq \text{SECD} \times \text{SECD}$.

- $(S, \delta, \mathbf{c} :: C) :: D \mapsto (\mathbf{c} :: S, \delta, C) :: D$
- $(S, \delta, x :: C) :: D \mapsto (\delta(x) :: S, \delta, C) :: D$
- $(S, \delta, \mathbf{op}_i(t_1, \dots, t_{a_i}) :: C) :: D \mapsto (S, \delta, t_1 :: \dots :: t_{a_i} :: \mathbf{Prim}(op_i) :: C) :: D$
- $(S, \delta, \mathbf{if}(t_1, t_2, t_3) :: C) :: D \mapsto (S, \delta, t_1 :: \mathbf{If} :: t_2 :: t_3 :: C) :: D$
- $(S, \delta, \mathbf{apply}(t_1, t_2) :: C) :: D \mapsto (S, \delta, t_1 :: t_2 :: \mathbf{App} :: C) :: D$
- $(S, \delta, \mathbf{fun}(x, t) :: C) :: D \mapsto (\mathbf{fun}(x, t) :: S, \delta, C) :: D$
- $(S, \delta, \mathbf{let } x = t_1 \mathbf{ in } t_2 :: C) :: D \mapsto (S, \delta, t_1 :: \mathbf{Let} :: x :: t_2 :: C) :: D$
- $(v_2 :: v_1 :: S, \delta, \mathbf{App} :: C) :: D \mapsto ([], \delta[f/v_1, x/v_2], t :: []) :: (S, \delta, C) :: D$ dove v_1 deve essere il valore $\mathbf{fun}(x, t)$
- $(v :: S, \delta, \mathbf{Let} :: x :: t :: C) :: D \mapsto ([], \delta[x/v], t :: []) :: (S, \delta, C) :: D$
- $(v :: S, \delta, []) :: (S', E', C') :: D \mapsto (v :: S', E', C') :: D$
- $(v_{a_i} :: \dots :: v_1 :: S, \delta, \mathbf{Prim}(op_i) :: C) :: D \mapsto (\tilde{op}_i(v_1, \dots, v_{a_i}) :: S, \delta, C) :: D$
- $(\mathbf{true} :: S, \delta, \mathbf{If} :: t_1 :: t_2 :: C) :: D \mapsto (S, \delta, t_1 :: C) :: D$
- $(\mathbf{false} :: S, \delta, \mathbf{If} :: t_1 :: t_2 :: C) :: D \mapsto (S, \delta, t_2 :: C) :: D$

Osserviamo che la macchina medesima realizza sostanzialmente una pila di ambienti, dato che ogni *chiamata* di funzione (la **apply**) o di dichiarazione locale (il **let**) porta alla creazione di una nuova tripla che viene messa in cima alla pila.

Per valutare un certo costrutto facciamo riferimento all'ambiente che è nella tripla che sta in cima alla pila. Quindi in quest'approccio abbiamo che gli ambienti che non sono attivi sono quelli "sotto" l'ambiente in cima alla pila.

Teorema 2.2 *La relazione $\mapsto$ è deterministica.*

2.2.5 Semantica Operazionale Small-Step: la relazione $\leadsto$

Le regole per la relazione $\rightarrow$ che abbiamo visto definiscono una relazione *big-step* sostanzialmente perché la valutazione di un termine si fa in un unico passo. Vediamo una diversa semantica, quindi una relazione, che chiameremo *small-step*, dove si vede esplicitamente come trattiamo la pila degli ambienti e come valutiamo i termini. Qui l'idea è che per valutare un termine composto da vari termini dobbiamo prima valutare questi ultimi e poi il termine che li compone.

La semantica la definiamo per questo linguaggio, che chiamiamo ETerm (extended terms)

$$tt ::= t \mid \mathbf{funblock}(tt)$$

I valori restituiti sono dei termini, e in particolare sono quelli generati dalla grammatica

$$v ::= c \mid \mathbf{fun}(x, t)$$

I valori sono indicati con v e sono un insieme **VALUE**. Quando valuteremo $\mathbf{op}_i(v_1, \dots, v_{a_i})$ restituiremo una costante v del linguaggio.

Ora avremo una pila di ambienti che indicheremo con Δ , ed ogni ambiente associa ai nomi un valore in **VALUE**.

Vediamo le regole che definiscono la relazione $\rightsquigarrow$ che prende un termine in **ETerm**, una pila di ambienti e restituisce un termine in **ETerm** ed una pila d'ambienti. La pila d'ambienti con solo l'ambiente $\delta_\perp$ ovunque indefinito la indichiamo con $\Delta_\perp$.

$$\begin{array}{c}
\frac{}{\langle\langle x, \Delta \rangle\rangle \rightsquigarrow \langle\text{top}(\Delta)(x), \Delta\rangle} \qquad \frac{}{\langle\langle \mathbf{op}_i(v_1, \dots, v_{a_i}), \Delta \rangle\rangle \rightsquigarrow \langle\langle v, \Delta \rangle\rangle} \\
\\
\frac{\langle\langle t_j, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_j, \Delta' \rangle\rangle}{\langle\langle \mathbf{op}_i(v_1, \dots, v_{j-1}, t_j, \dots, t_{a_i}), \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{op}_i(v_1, \dots, v_{j-1}, t'_j, \dots, t_{a_i}), \Delta' \rangle\rangle} \\
\\
\frac{}{\langle\langle \mathbf{if true then } t_1 \mathbf{ else } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle t_1, \Delta \rangle\rangle} \qquad \frac{}{\langle\langle \mathbf{if false then } t_1 \mathbf{ else } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle t_2, \Delta \rangle\rangle} \\
\\
\frac{\langle\langle t_0, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_0, \Delta' \rangle\rangle}{\langle\langle \mathbf{if } t_0 \mathbf{ then } t_1 \mathbf{ else } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{if } t'_0 \mathbf{ then } t_1 \mathbf{ else } t_2, \Delta' \rangle\rangle} \\
\\
\frac{}{\langle\langle \mathbf{apply}(\mathbf{fun}(x, t), v), \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{funblock}(t), \text{push}(\text{top}(\Delta)[x/v], \Delta) \rangle\rangle} \\
\\
\frac{\langle\langle t_1, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_1, \Delta' \rangle\rangle}{\langle\langle \mathbf{apply}(t_1, t_2), \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{apply}(t'_1, t_2), \Delta' \rangle\rangle} \\
\\
\frac{\langle\langle t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_2, \Delta' \rangle\rangle}{\langle\langle \mathbf{apply}(\mathbf{fun}(x, t), t_2), \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{apply}(\mathbf{fun}(x, t), t'_2), \Delta' \rangle\rangle} \\
\\
\frac{\langle\langle t, \Delta \rangle\rangle \rightsquigarrow \langle\langle t', \Delta' \rangle\rangle}{\langle\langle \mathbf{funblock}(t), \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{funblock}(t'), \Delta' \rangle\rangle} \qquad \frac{}{\langle\langle \mathbf{funblock}(v), \Delta \rangle\rangle \rightsquigarrow \langle\langle v, \text{pop}(\Delta) \rangle\rangle} \\
\\
\frac{\langle\langle t_1, \Delta \rangle\rangle \rightsquigarrow \langle\langle t'_1, \Delta' \rangle\rangle}{\langle\langle \mathbf{let } x = t_1 \mathbf{ in } t_2, \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{let } x = t'_1 \mathbf{ in } t_2, \Delta' \rangle\rangle} \\
\\
\frac{}{\langle\langle \mathbf{let } x = v \mathbf{ in } t, \Delta \rangle\rangle \rightsquigarrow \langle\langle \mathbf{funblock}(t), \text{push}(\text{top}(\Delta)[x/v], \Delta) \rangle\rangle}
\end{array}$$

2.2.6 Relazioni tra le semantiche

Abbiamo visto sostanzialmente tre relazioni che danno la semantica del medesimo linguaggio di programmazione. La relazione tra la semantica big-step e quella definita dalla SECD è semplice, osservando che i valori che la semantica restituisce sono gli stessi.

Teorema 2.3 *Sia dato un termine $t \in \mathbf{Term}$ e sia $\delta_\perp$ l'ambiente ovunque indefinito. Allora $\langle t, \delta_\perp \rangle \rightarrow^* v$ se e solo se $([\], \delta_\perp, t :: [\]) :: [\] \mapsto^* (v :: [\], \delta_\perp, [\]) :: [\]$.*

La relazione con la semantica small-step è un po' più complicata, e passa dalla seguente definizione.

Definizione 2.1 Dato un termine $t \in \text{Term}$, t è in forma normale se t è una costante od un termine della forma $\text{fun}(f, x, t)$, quindi se t è generato da ν .

Dato un termine $t \in \text{ETerm}$ ed una pila di ambienti Δ , diciamo che t è irriducibile se $\langle\langle t, \Delta \rangle\rangle \not\rightsquigarrow$. Osserviamo che le forme normali sono irriducibili. Da ultimo definiamo una relazione $\Downarrow \subseteq \text{Term} \times \text{Value}$ che associa ad ogni forma normale il suo corrispondente valore ed è indefinita per tutte le forme che non sono forme normali.

Teorema 2.4 Sia dato un termine $t \in \text{Term}$, sia $\delta_\perp$ l'ambiente ovunque indefinito e sia *emptystack* la pila vuota. Allora $\langle t, \delta_\perp \rangle \rightarrow^* \nu$ se e solo se $\langle\langle t, \text{push}(\delta_\perp, \text{emptystack}) \rangle\rangle \rightsquigarrow^* \langle\langle t', \text{push}(\delta_\perp, \text{emptystack}) \rangle\rangle$ dove t' è irriducibile e $\Downarrow t' = \nu$.

Facciamo alcune considerazioni. Nella semantica big-step la pila degli ambienti è implementata implicitamente, mentre nella semantica small-step questa è implementata esplicitamente. Nella semantica realizzata come SECD abbiamo invece che la macchina stessa è una pila. Emerge comunque la necessità di avere una pila per trovare correttamente l'ambiente in cui valutare il termine.

2.3 Scope Statico: implementazione

Abbiamo dato la semantica di un linguaggio funzionale in tre modi: big-step, small-step e tramite la SECD, definendo tre relazioni. Non ci siamo preoccupati dello scope ma è ovvio che abbiamo implementato lo scope dinamico perché la valutazione delle funzioni avviene nell'ambiente ottenuto a partire dall'ambiente in cui appare la **apply**. Per implementare lo scope statico abbiamo invece bisogno di *ricordare* dove una variabile è stata dichiarata. Cambia la sintassi del linguaggio, cambiano le regole e cambiano soprattutto gli oggetti denotabili, cioè i valori.

2.3.1 Il linguaggio

Come sarà chiaro provando a dare la semantica di un termine in cui una funzione *richiama* se stessa, abbiamo bisogno di esplicitamente prevedere questo nella sintassi. I termini quindi diventano

$$t ::= \mathbf{c} \mid x \mid \mathbf{op}_i(t_1, \dots, t_{a_i}) \mid \mathbf{if}(t_1, t_2, t_3) \mid \mathbf{fun}(x, t) \mid \mathbf{apply}(t_1, t_2) \mid \mathbf{let} \ x = t_1 \ \mathbf{in} \ t_2 \\ \mid \mathbf{letrec} \ f = t_1 \ \mathbf{in} \ t_2$$

e l'insieme di questi termini lo indichiamo come StTerm . Per poter formalizzare quali sono i valori abbiamo bisogno di dire quali sono le variabili *libere* che compaiono in un termine. Ovviamente le variabili libere sono quei nomi per cui non c'è un legame nell'ambiente. Dato un termine t , l'insieme delle sue variabili libere è definito come segue:

$$FV(t) = \begin{cases} \{x\} & \text{se } t \text{ è } x \\ \emptyset & \text{se } t \text{ è } \mathbf{c} \\ \bigcup_{1 \leq i \leq a_i} FV(t_i) & \text{se } t \text{ è } \mathbf{op}_i(t_1, \dots, t_{a_i}) \\ FV(t') \setminus \{x\} & \text{se } t \text{ è } \mathbf{fun}(x, t') \\ FV(t_0) \cup FV(t_1) \cup FV(t_2) & \text{se } t \text{ è } \mathbf{if} \ t_0 \ \mathbf{then} \ t_1 \ \mathbf{else} \ t_2 \\ FV(t_1) \cup (FV(t_2)) \setminus \{x\} & \text{se } t \text{ è } \mathbf{let} \ x = t_1 \ \mathbf{in} \ t_2 \\ (FV(t_1) \cup FV(t_2)) \setminus \{f\} & \text{se } t \text{ è } \mathbf{letrec} \ f = t_1 \ \mathbf{in} \ t_2 \end{cases}$$

2.3.2 I valori e l'ambiente

Prima di definire formalmente i valori abbiamo bisogno di stabilire un ordine tra le funzioni parziali da un insieme A ad un insieme B . Una funzione f è più piccola di una funzione g se, per ogni $a \in A$, se $f(a)$ è definita lo è anche $g(a)$ e sono uguali. La funzione ovunque indefinita è la più piccola tra tutte le funzioni. Dato un insieme F di funzioni che soddisfano una certa proprietà, con μF indichiamo la più piccola funzione di tale insieme. Infine con $\text{dom}(f)$ indichiamo il sottoinsieme di A per cui $f(a)$ è definita.

Dato che la valutazione di una costante è la costante medesima, ovviamente l'insieme StValue contiene $\mathbf{C}$ dove c è la valutazione (semantica) di $\mathbf{c}$, come prima.

Per le funzioni dobbiamo ora pensare che non può essere più un termine come $\mathbf{fun}(x, t)$, dato che questo non ha alcuna informazione sui nomi che compaiono nel termine. Dovrà essere una *chiusura*, cioè una coppia formata dalla funzione $\mathbf{fun}(x, t)$ e da un ambiente che è quello dove la funzione andrà valutata. Quindi ora StFun ed è

$$\{(\mathbf{fun}(x, t), \delta) \mid x \in \text{Var}, t \in \text{StTerm}, \delta \in \text{StEnv} \text{ e } \delta = \mu\{\delta \mid FV(t) \subseteq \text{dom}(\delta)\}\}$$

Come effettivamente viene calcolato il più piccolo ambiente che soddisfa tale proprietà (quella che per tutti i nomi c è un'associazione) non lo formalizziamo ma l'idea è quella di aggiungere all'ambiente le associazioni che mancano e che servono fino a che non si sono aggiunte tutte quelle per i nomi che compaiono nel corpo della funzione.

Di nuovo $\text{StValue} = \mathbf{C} \cup \text{StFun}$ e StEnv denota l'insieme degli ambienti.

2.3.3 Semantica Operazionale Big-Step: la relazione $\rightarrow_{st}$

Definiamo le regole per la relazione $\rightarrow_{st} \subseteq \text{StTerm} \times \text{Env} \times \text{Value}$.

$$\begin{array}{c} \overline{\langle x, \delta \rangle \rightarrow_{st} \delta(x)} \quad \overline{\langle \mathbf{c}, \delta \rangle \rightarrow_{st} c} \quad \overline{\langle \mathbf{fun}(x, t), \delta \rangle \rightarrow_{st} (\mathbf{fun}(x, t), \delta')} \\[10pt] \frac{\langle t_1, \delta \rangle \rightarrow_{st} v_1 \quad \dots \quad \langle t_{a_i}, \delta \rangle \rightarrow_{st} v_{a_i}}{\langle \mathbf{op}_i(t_1, \dots, t_{a_i}), \delta \rangle \rightarrow_{st} \widetilde{\text{op}_i}(v_1, \dots, v_{a_i})} \\[10pt] \frac{\langle t_0, \delta \rangle \rightarrow_{st} \mathbf{true} \quad \langle t_1, \delta \rangle \rightarrow_{st} v}{\langle \mathbf{if } t_0 \mathbf{ then } t_1 \mathbf{ else } t_2, \delta \rangle \rightarrow_{st} v} \quad \frac{\langle t_0, \delta \rangle \rightarrow_{st} \mathbf{false} \quad \langle t_2, \delta \rangle \rightarrow_{st} v}{\langle \mathbf{if } t_0 \mathbf{ then } t_1 \mathbf{ else } t_2, \delta \rangle \rightarrow_{st} v} \\[10pt] \frac{\langle t_1, \delta \rangle \rightarrow_{st} (\mathbf{fun}(x, t), \delta') \quad \langle t_2, \delta \rangle \rightarrow_{st} v' \quad \langle t, \delta'[x/v_2] \rangle \rightarrow_{st} v}{\langle \mathbf{apply}(t_1, t_2), \delta \rangle \rightarrow_{st} v} \\[10pt] \frac{\langle t_1, \delta \rangle \rightarrow_{st} v_1 \quad \langle t_2, \delta[x/v_1] \rangle \rightarrow_{st} v}{\langle \mathbf{let } x = t_1 \mathbf{ in } t_2, \delta \rangle \rightarrow_{st} v} \\[10pt] \frac{\langle t_1, \delta \rangle \rightarrow_{st} v_1 \quad v_1 = (\mathbf{fun}(x, t), \delta') \quad \langle t_2, \delta[f/v_1] \rangle \rightarrow_{st} v}{\langle \mathbf{letrec } f = t_1 \mathbf{ in } t_2, \delta \rangle \rightarrow_{st} v} \end{array}$$

Ora la regola per l'**apply** recupera l'ambiente in cui valutare il corpo della funzione dalla chiusura. La regola per la valutazione di una funzione ricorsiva (**funrec**) calcola in qualche modo il più piccolo ambiente dove a questa funzione è associata una chiusura.

E importante sottolineare che la relazione è deterministica.

Teorema 2.5 *La relazione $\rightarrow_{st}$ è deterministica.*

Questo significa che per ogni termine t ed ogni ambiente δ , se esiste un valore v tale che $(t, \delta, v) \in \rightarrow$, allora se $(t, \delta, v') \in \rightarrow_{st}$ vale che $v = v'$.

2.3.4 La relatione $\mapsto_{st}$

Vediamo ora come modificare le regole che governano la relazione $\mapsto_{st}$ definita tra le SECD.

Vediamo per prima cosa quali sono gli elementi di controllo, che sono parole generate dalla seguente grammatica. Gli elementi di controllo sono

$$ce_{st} ::= ce \mid \text{LetRec}$$

I valori sono quelli visti prima.

- $(S, \delta, c :: C) :: D \mapsto_{st} (c :: S, \delta, C) :: D$
- $(S, \delta, x :: C) :: D \mapsto_{st} (\delta(x) :: S, \delta, C) :: D$
- $(S, \delta, \text{op}_i(t_1, \dots, t_{a_i}) :: C) :: D \mapsto_{st} (S, \delta, t_1 :: \dots :: t_{a_i} :: \text{Prim}(op_i) :: C) :: D$
- $(S, \delta, \text{if}(t_1, t_2, t_3) :: C) :: D \mapsto_{st} (S, \delta, t_1 :: \text{If} :: t_2 :: t_3 :: C) :: D$
- $(S, \delta, \text{apply}(t_1, t_2) :: C) :: D \mapsto_{st} (S, \delta, t_1 :: t_2 :: \text{App} :: C) :: D$
- $(S, \delta, \text{fun}(x, t) :: C) :: D \mapsto_{st} ((\text{fun}(x, t), \delta) :: S, \delta, C) :: D$
- $(S, \delta, \text{let } x = t_1 \text{ in } t_2 :: C) :: D \mapsto_{st} (S, \delta, t_1 :: \text{Let} :: x :: t_2 :: C) :: D$
- $(S, \delta, \text{letrec } f = t_1 \text{ in } t_2 :: C) :: D \mapsto_{st} (S, \delta, t_1 :: \text{LetRec} :: f :: t_2 :: C) :: D$
- $(v_2 :: v_1 :: S, \delta, \text{App} :: C) :: D \mapsto_{st} ([], \delta'[x/v_2], t :: []) :: (S, \delta, C) :: D$ dove v_1 deve essere il valore $(\text{fun}(x, t), \delta')$
- $(v :: S, \delta, \text{Let} :: x :: t_2 :: C) :: D \mapsto_{st} ([], \delta[x/v], t_2 :: []) :: (S, \delta, C) :: D$
- $((\text{fun}(x, t), \delta') :: S, \delta, \text{LetRec} :: f :: t_2 :: C) :: D \mapsto_{st} ([], \delta[f/(\text{fun}(x, t), \delta')], t_2 :: []) :: (S, \delta, C) :: D$
- $(v :: S, \delta, []) :: (S', E', C') :: D \mapsto_{st} (v :: S', E', C') :: D$
- $(v_{a_i} :: \dots :: v_1 :: S, \delta, \text{Prim}(op_i) :: C) :: D \mapsto_{st} (\tilde{op}_i(v_1, \dots, v_{a_i}) :: S, \delta, C) :: D$
- $(\text{true} :: S, \delta, \text{If} :: t_2 :: t_3 :: C) :: D \mapsto_{st} (S, \delta, t_2 :: C) :: D$
- $(\text{false} :: S, \delta, \text{If} :: t_2 :: t_3 :: C) :: D \mapsto_{st} (S, \delta, t_3 :: C) :: D$

Osserviamo che la macchina medesima realizza sostanzialmente una pila di ambienti, dato che ogni *chiamata* di funzione (la **apply**) o di dichiarazione locale (il **let**) porta alla creazione di una nuova tripla che viene messa in cima alla pila.

Per valutare un certo costrutto facciamo riferimento all'ambiente che è nella tripla che sta in cima alla pila. Quindi in quest'approccio abbiamo che gli ambienti che non sono attivi sono quelli "sotto" l'ambiente in cima alla pila.

Teorema 2.6 *La relazione $\mapsto_{st}$ è deterministica.*

2.3.5 Semantica Operazionale Small-Step: la relazione $\leadsto_{st}$

Come fatto per lo scope dinamico, arricchiremo il linguaggio, che ora deve incorporare i valori. I valori restituiti sono dei termini, e in particolare sono quelli generati dalla grammatica

$$v ::= c \mid (\mathbf{fun}(x, t), \delta)$$

e come prima tra le coppie $(\mathbf{fun}(x, t), \delta)$ devono appartenere a **StFun**. I valori sono indicati con v e sono un insieme **VALUE**. Quando valuteremo $\mathbf{op}_i(v_1, \dots, v_{a_i})$ restituirò una costante v del linguaggio.

Cambia anche il linguaggio, che ora deve tenere conto della nuova forma che hanno i valori.

$$tt_{st} ::= v \mid t \mid \mathbf{funblock}(tt_{st})$$

Vediamo le regole che definiscono la relazione $\leadsto_{st}$ che prende un termine in **EStTerm**, una pila di ambienti e restituisce un termine in **ETerm** ed una pila d'ambienti.

$$\begin{array}{c}
\frac{}{\langle\langle x, \Delta \rangle\rangle \leadsto_{st} \langle\mathit{top}(\Delta)(x), \Delta\rangle} \qquad \frac{}{\langle\langle \mathbf{op}_i(v_1, \dots, v_{a_i}), \Delta \rangle\rangle \leadsto_{st} \langle\langle v, \Delta \rangle\rangle} \\
\\
\frac{\langle\langle t_j, \Delta \rangle\rangle \leadsto_{st} \langle\langle t'_j, \Delta' \rangle\rangle}{\langle\langle \mathbf{op}_i(v_1, \dots, v_{j-1}, t_j, \dots, t_{a_i}), \Delta \rangle\rangle \leadsto_{st} \langle\langle \mathbf{op}_i(v_1, \dots, v_{j-1}, t'_j, \dots, t_{a_i}), \Delta' \rangle\rangle} \\
\\
\frac{}{\langle\langle \mathbf{if \ true \ then \ } t_1 \ \mathbf{else \ } t_2, \Delta \rangle\rangle \leadsto_{st} \langle\langle t_1, \Delta \rangle\rangle} \qquad \frac{}{\langle\langle \mathbf{if \ false \ then \ } t_1 \ \mathbf{else \ } t_2, \Delta \rangle\rangle \leadsto_{st} \langle\langle t_2, \Delta \rangle\rangle} \\
\\
\frac{\langle\langle t_0, \Delta \rangle\rangle \leadsto_{st} \langle\langle t'_0, \Delta' \rangle\rangle}{\langle\langle \mathbf{if \ } t_0 \ \mathbf{then \ } t_1 \ \mathbf{else \ } t_2, \Delta \rangle\rangle \leadsto_{st} \langle\langle \mathbf{if \ } t'_0 \ \mathbf{then \ } t_1 \ \mathbf{else \ } t_2, \Delta' \rangle\rangle} \\
\\
\frac{}{\langle\langle \mathbf{apply}((\mathbf{fun}(x, t), \delta), v), \Delta \rangle\rangle \leadsto_{st} \langle\langle \mathbf{funblock}(t), \mathit{push}(\delta[x/v], \Delta) \rangle\rangle} \\
\\
\frac{\langle\langle t_1, \Delta \rangle\rangle \leadsto_{st} \langle\langle t'_1, \Delta' \rangle\rangle}{\langle\langle \mathbf{apply}(t_1, t_2), \Delta \rangle\rangle \leadsto_{st} \langle\langle \mathbf{apply}(t'_1, t_2), \Delta' \rangle\rangle} \\
\\
\frac{\langle\langle t_2, \Delta \rangle\rangle \leadsto_{st} \langle\langle t'_2, \Delta' \rangle\rangle}{\langle\langle \mathbf{apply}((\mathbf{fun}(x, t), \delta), t_2), \Delta \rangle\rangle \leadsto_{st} \langle\langle \mathbf{apply}((\mathbf{fun}(x, t), \delta), t'_2), \Delta' \rangle\rangle} \\
\\
\frac{\langle\langle t, \Delta \rangle\rangle \leadsto_{st} \langle\langle t', \Delta' \rangle\rangle}{\langle\langle \mathbf{funblock}(t), \Delta \rangle\rangle \leadsto_{st} \langle\langle \mathbf{funblock}(t'), \Delta' \rangle\rangle} \qquad \frac{}{\langle\langle \mathbf{funblock}(v), \Delta \rangle\rangle \leadsto_{st} \langle\langle v, \mathit{pop}(\Delta) \rangle\rangle} \\
\\
\frac{\langle\langle t_1, \Delta \rangle\rangle \leadsto_{st} \langle\langle t'_1, \Delta' \rangle\rangle}{\langle\langle \mathbf{let \ } x = t_1 \ \mathbf{in \ } t_2, \Delta \rangle\rangle \leadsto_{st} \langle\langle \mathbf{let \ } x = t'_1 \ \mathbf{in \ } t_2, \Delta' \rangle\rangle}
\end{array}$$

$$\overline{\langle\langle \text{let } x = v \text{ in } t, \Delta \rangle\rangle \rightsquigarrow_{st} \langle\langle \text{funblock}(t), \text{push}(\text{top}(\Delta)[x/v], \Delta) \rangle\rangle}$$

$$\frac{\langle\langle t_1, \Delta \rangle\rangle \rightsquigarrow_{st} \langle\langle t'_1, \Delta' \rangle\rangle}{\langle\langle \text{letrec } f = t_1 \text{ in } t_2, \Delta \rangle\rangle \rightsquigarrow_{st} \langle\langle \text{letrec } f = t'_1 \text{ in } t_2, \Delta' \rangle\rangle}$$

$$\frac{v = (\text{fun}(x, t'), \delta') \quad FV(t') \subseteq \text{dom}(\delta')}{\langle\langle \text{letrec } f = v \text{ in } t, \Delta \rangle\rangle \rightsquigarrow_{st} \langle\langle \text{funblock}(t), \text{push}(\delta'[f/v], \Delta) \rangle\rangle}$$

2.3.6 Relazioni tra le semantiche

Abbiamo visto sostanzialmente tre relazioni che danno la semantica del medesimo linguaggio di programmazione. La relazione tra la semantica big-step e quella definita dalla SECD è semplice, osservando che i valori che la semantica restituisce sono gli stessi.

Teorema 2.7 *Sia dato un termine $t \in \text{StTerm}$ e sia $\delta_\perp$ l'ambiente ovunque indefinito. Allora $\langle t, \delta_\perp \rangle \rightarrow_{st}^* v$ se e solo se $([\], \delta_\perp, t :: [\]) :: [\] \mapsto_{st}^* (v :: [\], \delta_\perp, [\]) :: [\]$.*

La relazione con la semantica small-step è un po' più complicata, e passa dalla seguente definizione.

Definizione 2.2 *Dato un termine $t \in \text{EStTerm}$, t è in forma normale se t è una costante od un termine del forma $(\text{fun}(x, t), \delta)$, quindi se t è generato da v .*

Dato un termine $t \in \text{EStTerm}$ ed una pila di ambienti Δ , diciamo che t è irriducibile se $\langle\langle t, \Delta \rangle\rangle \not\rightsquigarrow$. Osserviamo che le forme normali sono irriducibili. Da ultimo definiamo una relazione $\Downarrow \subseteq \text{EStTerm} \times \text{Value}$ che associa ad ogni forma normale il suo corrispondente valore ed è indefinita per tutte le forme che non sono forme normali.

Teorema 2.8 *Sia dato un termine $t \in \text{EStTerm}$, sia $\delta_\perp$ l'ambiente ovunque indefinito e sia emptystack la pila vuota. Allora $\langle t, \delta_\perp \rangle \rightarrow_{st}^* v$ se e solo se $\langle\langle t, \text{push}(\delta_\perp, \text{emptystack}) \rangle\rangle \rightsquigarrow_{st}^* \langle\langle t', \text{push}(\delta_\perp, \text{emptystack}) \rangle\rangle$ dove t' è irriducibile e $\Downarrow t' = v$.*

Capitolo 3

Un linguaggio imperativo

Vediamo il nucleo di un linguaggio *imperativo*, che chiamiamo IMP.

3.1 Sintassi

Il linguaggio IMP ha un insieme di identificatori che appartengono ad un insieme di nomi e che continuiamo a chiamare *Var*, mentre i valori in *Value* saranno i numeri interi. Lo stato lo vediamo come una funzione parziale da *Var* a $\mathbb{Z}$. Quindi l'insieme degli stati è

$$States = \{st \mid st : Var \rightarrow \mathbb{Z}\}$$

Per dire che, dato un nome X ed uno stato st , al nome X è associato un valore, scriviamo $st(X) \downarrow$.

Il linguaggio ha espressioni aritmetiche, booleane e comandi.

3.1.1 Espressioni aritmetiche

Il linguaggio ha espressioni aritmetiche, che sono generate dalla grammatica

$$a ::= n \mid X \mid a_0 + a_1 \mid a_0 \times a_1 \mid a_0 - a_1$$

dove $X \in Var$ sono le *variabili*.

3.1.2 Espressioni booleane

Nel linguaggio IMP abbiamo anche bisogno di espressioni booleane la cui sintassi è

$$b ::= \text{true} \mid \text{false} \mid b_0 \vee b_1 \mid b_0 \wedge b_1 \mid \neg b_0 \mid \neg b \mid a_1 = a_2 \mid a_1 < a_2$$

Dobbiamo dire quali sono i valori che la valutazione di una espressione booleana restituisce, e questi sono $\mathbb{B} = \{true, false\}$.

3.1.3 Comandi

Vediamo ora la sintassi dei comandi di un semplice linguaggio imperativo:

$$c ::= \text{skip} \mid X := a \mid c_0; c_1 \mid \text{if } b \text{ then } c_1 \text{ else } c_2 \mid \text{while } b \text{ do } c$$

i comandi usano anche le variabili, le espressioni aritmetiche e quelle booleane.

3.2 Semantica del linguaggio

Vediamo come dare semantica al linguaggio definendo le opportune relazioni per le espressioni aritmetiche, quelle booleane e per i comandi. Per le espressioni aritmetiche e booleane diamo solo le relazioni big-step mentre per i comandi diamo sia quella big-step che quella small-step.

3.2.1 La relazione $\longrightarrow$ per le espressioni aritmetiche

Per valutare un'espressione abbiamo bisogno dello stato, che abbiamo detto indichiamo con st . Le regole sono le seguenti che sono sostanzialmente una riscrittura di alcune regole della big-step semantics del linguaggio funzionale.

$$\begin{array}{c} \frac{}{\langle n, st \rangle \longrightarrow n} \quad \frac{st(X) \downarrow}{\langle X, st \rangle \longrightarrow st(X)} \quad \frac{\langle a_0, st \rangle \longrightarrow n_0 \quad \langle a_1, st \rangle \longrightarrow n_1}{\langle a_0 + a_1, st \rangle \longrightarrow n = n_0 + n_1} \\[10pt] \frac{\langle a_0, st \rangle \longrightarrow n_0 \quad \langle a_1, st \rangle \longrightarrow n_1}{\langle a_0 \times a_1, st \rangle \longrightarrow n = n_0 \times n_1} \quad \frac{\langle a_0, st \rangle \longrightarrow n_0 \quad \langle a_1, st \rangle \longrightarrow n_1}{\langle a_0 - a_1, st \rangle \longrightarrow n = n_0 - n_1} \end{array}$$

3.2.2 La relazione $\mapsto$ per le espressioni booleane

Le regole per la valutazione big-step delle espressioni booleane sono le seguenti.

$$\begin{array}{c} \frac{}{\langle true, st \rangle \mapsto true} \quad \frac{}{\langle false, st \rangle \mapsto false} \quad \frac{\langle b, st \rangle \mapsto t}{\langle \neg b, st \rangle \mapsto \neg t} \\[10pt] \frac{\langle b_0, st \rangle \mapsto t_0 \quad \langle b_1, st \rangle \mapsto t_1}{\langle b_0 \vee b_1, st \rangle \mapsto t_0 \vee t_1} \quad \frac{\langle b_0, st \rangle \mapsto t_0 \quad \langle b_1, st \rangle \mapsto t_1}{\langle b_0 \wedge b_1, st \rangle \mapsto t_0 \wedge t_1} \\[10pt] \frac{\langle a_0, st \rangle \longrightarrow n_0 \quad \langle a_1, st \rangle \longrightarrow n_1}{\langle a_0 = a_1, st \rangle \mapsto n_0 = n_1} \quad \frac{\langle a_0, st \rangle \longrightarrow n_0 \quad \langle a_1, st \rangle \longrightarrow n_1}{\langle a_0 < a_1, st \rangle \mapsto n_0 < n_1} \end{array}$$

3.2.3 La relazione $\Longrightarrow$ per i comandi

Per poterne dare la semantica dobbiamo prima capire cosa fa un comando in generale. La risposta, come in questi casi capita sempre, è ovvia: un comando modifica lo stato, e nel nostro caso lo stato non sarà altro che un'associazione tra i nomi, che qui sono le variabili, ed i valori associabili alle variabili, che qui sono gli interi.

Lo stato lo aggiorniamo quando assegniamo un valore ad una variabile. La notazione è la solita: $st[n/X]$ che significa che abbiamo assegnato n alla variabile X . $st[n/X]$ è la funzione dalle variabili Var ai valori $\mathbb{Z}$ che

$$st[n/X](Y) = \begin{cases} n & \text{se } X = Y \\ st(Y) & \text{altrimenti} \end{cases}$$

Descriviamo prima cosa fanno i comandi.

- **skip.** Questo comando non fa nulla, lascia invariato lo stato.
- **L'assegnamento:** il comando $X := a$ modifica lo stato assegnando alla variabile X il risultato della valutazione dell'espressione aritmetica a .

- il ‘;’, cioè quello che stabilisce la sequenza di due comandi. La regola è ovvia: bisogna eseguire il secondo nello stato ottenuto eseguendo il primo
- il condizionale (**if** b **then** c_1 **else** c_2). Come al solito bisogna valutare l’espressione booleana e a seconda del risultato eseguire uno dei due rami.
- l’iterazione (il **while** b **do** c). Se la condizione booleana è falsa, allora non bisogna fare nulla, se invece è vera bisogna prima eseguire il comando c e, nello stato ottenuto, eseguire il costrutto di iterazione, cioè il **while**.

Le regole sono:

$$\begin{array}{c}
\frac{}{\langle \text{skip}, st \rangle \Longrightarrow st} \quad \frac{\langle a, st \rangle \longrightarrow n}{\langle X := a, st \rangle \Longrightarrow st[n/X]} \quad \frac{\langle c_0, st \rangle \Longrightarrow st'' \quad \langle c_1, st'' \rangle \Longrightarrow st'}{\langle c_0; c_1, st \rangle \Longrightarrow st'} \\
\\
\frac{\langle b, st \rangle \mapsto true \quad \langle c_1, st \rangle \Longrightarrow st'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, st \rangle \Longrightarrow st'} \quad \frac{\langle b, st \rangle \mapsto false \quad \langle c_2, st \rangle \Longrightarrow st'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, st \rangle \Longrightarrow st'} \\
\\
\frac{\langle b, st \rangle \mapsto false}{\langle \text{while } b \text{ do } c, st \rangle \Longrightarrow st} \\
\\
\frac{\langle b, st \rangle \mapsto true \quad \langle c, st \rangle \Longrightarrow st'' \quad \langle \text{while } b \text{ do } c, st'' \rangle \Longrightarrow st'}{\langle \text{while } b \text{ do } c, st \rangle \Longrightarrow st'}
\end{array}$$

Le regole della semantica operativa ci sono servite per definire tre relazioni:

- $\longrightarrow \subseteq Aexp \times States \times \mathbb{Z}$
- $\mapsto \subseteq Bexp \times States \times \mathbb{B}$
- $\Longrightarrow \subseteq Com \times States \times mathit{States}$

dove $Aexp$ sono le espressioni aritmetiche, $Bexp$ le espressioni booleane e Com sono le istruzioni del nostro linguaggio IMP.

Queste relazioni sono *deterministiche* nel senso che, fissato un elemento in $Aexp$, $Bexp$ o Com ed uno stato, ogni relazione contiene una sola tripla che ha esattamente quei due primi elementi, quindi sostanzialmente definiscono delle funzioni. In particolare la relazione $\longrightarrow$ definisce la funzione $\mathcal{A} : Aexp \times st \rightarrow \mathbb{Z}$, la relazione $\mapsto$ definisce la funzione $\mathcal{B} : Bexp \times st \rightarrow \mathbb{B}$, ed infine la relazione $\Longrightarrow$ la funzione $\mathcal{C} : Com \times States \rightarrow States$.

La semantica ci aiuta anche a dire quando due costrutti sono *equivalenti*. Dato che a noi interessano nel linguaggio imperativo i comandi, formalizziamo cosa significa che due comandi sono equivalenti.

Definizione 3.1 Due comandi $c, c' \in Com$ sono equivalenti se e solo se per ogni stato $st \in States$ vale che $\mathcal{C}(c, st) = \mathcal{C}(c', st)$.

Va osservato che la definizione di equivalenza può essere usata in due modi. Se una parte della semantica la definizione serve a caratterizzare quali sono i costrutti tra di loro equivalenti, se si parte invece dalla *richiesta* che certi costrutti siano equivalenti allora possiamo costruire la semantica in modo che tale equivalenza venga rispettata.

3.2.4 La relazione $\rightsquigarrow$ per i comandi

La relazione $\implies$ pone l'accento sulla trasformazione di stato, immaginando che tutti i comandi vengano eseguiti *integralmente*, dove integralmente significa che l'esecuzione del comando restituisce uno stato e non una coppia (costrutto, stato), prima di passare allo stato successivo. Si può però porre l'attenzione anche su cosa bisogna ancora eseguire. Di seguito le regole modificate tenendo conto di voler sottolineare, nella semantica, proprio questo. Quando dobbiamo eseguire **skip** non dobbiamo fare nulla e quindi non abbiamo una regola.

$$\begin{array}{c}
\frac{\langle a, st \rangle \longrightarrow n}{\langle X := n, st \rangle \rightsquigarrow \langle \mathbf{skip}, st[X/n] \rangle} \\[10pt]
\frac{\langle c_0, st \rangle \rightsquigarrow \langle \mathbf{skip}, st' \rangle}{\langle c_0; c_1, st \rangle \rightsquigarrow \langle c_1, st' \rangle} \qquad \frac{\langle c_0, st \rangle \rightsquigarrow \langle c'_0, st' \rangle}{\langle c_0; c_1, st \rangle \rightsquigarrow \langle c'_0; c_1, st' \rangle} \\[10pt]
\frac{\langle b, st \rangle \rightsquigarrow \mathbf{true}}{\langle \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2, st \rangle \rightsquigarrow \langle c_0, st \rangle} \qquad \frac{\langle b, st \rangle \rightsquigarrow \mathbf{false}}{\langle \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2, st \rangle \rightsquigarrow \langle c_1, st \rangle} \\[10pt]
\frac{}{\langle \mathbf{while } b \mathbf{ do } c, st \rangle \rightsquigarrow \langle \mathbf{if } b \mathbf{ then } c; \mathbf{while } b \mathbf{ do } c \mathbf{ else skip}, st \rangle}
\end{array}$$

Perché per noi è importante la semantica (operazionale)? Ha il vantaggio che rende molto semplice e naturale definire il nostro interprete: prendiamo le regole, che vediamo come delle funzioni, che scriviamo nel linguaggio di programmazione della macchina ospite. In più, è facile da usare, e ci serve per capire cosa dobbiamo mettere nel supporto a tempo d'esecuzione, cioè gli algoritmi e le strutture dati per l'esecuzione dei programmi del nostro linguaggio.

Capitolo 4

Un linguaggio dichiarativo

Il linguaggio IMP è Turing equivalente, quindi tutti i programmi che rappresentano le funzioni intuitivamente calcolabili sono scrivibili in tale linguaggio. Però rispetto ad un linguaggio funzionale, dove c'è la chiara definizione di funzione e con la possibilità di dare nomi abbiamo un chiaro modo di astrarre, quindi un termine può avere un nome ed essere usato in contesti diversi. Estendiamo quindi il linguaggio con dichiarazioni e blocchi.

Dato che estendiamo il linguaggio dobbiamo anche domandarci cosa serve per modellare lo stato che, ricordiamo, per ora è solo una funzione da nomi a valori (nel caso dei linguaggi funzionali questi valori sono le costanti e le funzioni, con ambiente o meno, mentre nei linguaggi imperativi sono gli interi).

Per capire cosa è lo stato in un linguaggio procedurale ricordiamo che stiamo estendendo un linguaggio imperativo che *modifica* lo stato e stiamo aggiungendo dichiarazioni che assoceranno oggetti a nomi, quindi è naturale che lo stato sia ora una coppia con un ambiente ed una memoria. Questi due oggetti sono collegati dato che ora ai nomi possiamo associare una locazione che identifica un oggetto in memoria. Definiamo $\text{Env} = \{\delta \mid \delta : \text{Name} \rightarrow \text{Obj}_d\}$ dove **Name** sono i nomi del linguaggio (identificatori) e Obj_d sono gli oggetti denotabili che contengono le *locazioni* che chiamiamo **Loc** e definiamo anche $\text{Mem} = \{\sigma \mid \sigma : \text{Loc} \rightarrow \text{Obj}_m\}$ dove Obj_m sono gli oggetti memorizzabili.

Definizione 4.1 *Lo stato è una coppia $(\delta, \sigma) \in \text{Env} \times \text{Mem}$.*

Dato un nome x , se questo è una variabile ed ha un valore associato, allora questo valore è $\sigma(\delta(x))$.

4.1 Dichiarazioni, procedure e blocchi

Le dichiarazioni sono il meccanismo con cui creiamo le associazioni nell'ambiente. Dichiariamo le variabili e anche le procedure. Le procedure sono il modo che abbiamo per *astrarre*: diamo un nome ad una porzione di codice, e questo nome lo possiamo usare ovunque purchè sia in scope.

$$d ::= \text{null} \mid \text{var } X \mid \text{const } X = a \mid \text{proc } f(p)\{c\} \mid d_1; d_2$$

La dichiarazione $\text{proc } f(p)\{c\}$ associerà al nome f il corpo della funzione e p sono i parametri che sono sequenze di identificatori (per semplicità ne assumiamo uno). La dichiarazione $\text{const } X = a$ associa ad X la valutazione di a .

I blocchi in un linguaggio imperativo servono a strutturare il programma e a delimitare lo scope dei nomi, proprio come il **let** in un linguaggio funzionale. Quindi i comandi vengono estesi con le seguenti produzioni:

$$cc ::= c \mid \{d; cc\} \mid \text{call } f(pa)$$

I blocchi sono delimitati dalle parentesi graffe e hanno la seguente forma: una dichiarazione seguita da un comando. Tra i comandi abbiamo aggiunto quello che consente di usare il nome di un'astrazione, che il `call`, e `pa` sono i parametri attuali che per noi, ora possono essere solo espressioni aritmetiche (che ricordiamo contengono anche i nomi delle variabili).

4.2 Semantica delle dichiarazioni e dei blocchi

Diamo ora le regole per le dichiarazioni (assumiamo che lo scope sia dinamico). Definiamo quindi una relazione $\Rightarrow_d$ che prende una dichiarazione, uno stato e restituisce un ambiente. Gli oggetti denotabili sono ora gli interi, le locazioni e le procedure. Quindi $\text{Obj}_d = \mathbb{Z} \uplus \text{Loc} \uplus \{\text{proc}(X, c) \mid X \text{ è un nome e } c \text{ è un comando}\}$.

$$\begin{array}{c} \frac{}{\langle \text{null}, (\delta, \sigma) \rangle \Rightarrow_d \delta} \quad \frac{l = \text{nl}()}{\langle \text{var } X, (\delta, \sigma) \rangle \Rightarrow_d \delta[X/l]} \quad \frac{\langle a, (\delta, \sigma) \rangle \longrightarrow v}{\langle \text{const } X = a, (\delta, \sigma) \rangle \Rightarrow_d \delta[X/v]} \\[10pt] \frac{}{\langle \text{proc } f(p)\{c\}, (\delta, \sigma) \Rightarrow_d \delta[f/\text{proc}(p, c)]} \quad \frac{\langle d_1, (\delta, \sigma) \rangle \Rightarrow_d \delta'' \quad \langle d_2, (\delta'', \sigma) \rangle \Rightarrow_d \delta'}{\langle d_1; d_2, (\delta, \sigma) \rangle \Rightarrow_d \delta'} \end{array}$$

Una volta definita la relazione per la costruzione dell'ambiente dobbiamo aggiungere le regole alla relazione $\Longrightarrow$ per la valutazione dei comandi:

$$\begin{array}{c} \frac{\langle d, (\delta, \sigma) \rangle \Rightarrow_d \delta' \quad \langle c, (\delta', \sigma) \rangle \Longrightarrow \sigma'}{\langle \{d; c\}, (\delta, \sigma) \rangle \Longrightarrow \sigma'} \\[10pt] \frac{\delta(f) = \text{proc}(X, c) \quad l = \text{nl}() \quad \langle a, (\delta, \sigma) \rangle \longrightarrow v \quad \langle c, (\delta[X/l], \sigma[l/v]) \rangle \Longrightarrow \sigma'}{\langle \text{call } f(a), (\delta, \sigma) \rangle \Longrightarrow \sigma'} \end{array}$$

Vale la pena di osservare che la relazione che abbiamo definito per le dichiarazioni ($\Rightarrow_d$) è deterministica ed anche lo è l'estensione della relazione $\Longrightarrow$.

4.2.1 Passaggio dei parametri

Nell'esecuzione del comando `call f(a)` abbiamo valutato l'espressione che è il parametro attuale. Il passaggio che abbiamo realizzato è il passaggio per *valore* dato che valuto il parametro. Possiamo però considerare almeno un altro modo di passare i parametri. Fino ad ora la grammatica per i parametri era $p ::= X$ con X nome, ora diventa $p ::= X \mid \text{rif } X$ per dire che il parametro è passato per riferimento. Bisogna quindi aggiungere una regola:

$$\frac{\delta(f) = \text{proc}(\text{rif } X, c) \quad Y \in \text{Name} \quad \langle c, (\delta[X/\delta(Y)], \sigma) \rangle \Longrightarrow \sigma'}{\langle \text{call } f(Y), (\delta, \sigma) \rangle \Longrightarrow \sigma'}$$

Va osservato che in effetti quando si passa un parametro per riferimento quello che succede è che viene creata un'*alias* per quel nome che è il parametro attuale. Nell'ambiente entrambi i nomi fanno riferimento alla medesima locazione. Se avessi il seguente pezzo di codice:

{ ...

```

var X;
proc f (rif Y) {
  Y := Y + 2;
  X := X + Y; };
X := 1;
call f(X);
print(Y);
...
}

```

L'esecuzione della chiamata della `f` fa sì che ci siano due nomi (`X` e `Y`) che fanno riferimento alla stessa locazione. Quindi quando si esegue `print(Y)` viene stampato il valore 6, dato che quando si aggiorna il valore associato alla locazione della `Y` si modifica anche la `X`.

4.2.2 La small step semantics

La relazione $\rightsquigarrow$ per i comandi vista prima implementa la small-step semantics per i comandi senza blocchi e dichiarazioni. Come per i linguaggi funzionali avremo bisogno di una pila per gli stati e dovremo tenere conto del fatto che entriamo ed usciamo da un blocco o eseguiamo una chiamata di procedura. Dobbiamo quindi introdurre nella sintassi dei comandi il comando `block(cc)`. Avendo introdotto blocchi e astrazioni dobbiamo anche considerare che avremo una pila per gli ambienti e ricordando che la semantica deve in definitiva restituire una memoria abbiamo che lo stato è ora una coppia fatta da una pila di ambienti e una memoria. Quindi ora $\text{States} = \text{EnvStack} \times \text{Mem}$. Le regole sono:

$$\begin{array}{c}
\frac{\delta = \text{top}(\Delta) \quad \langle a, (\delta, \sigma) \rangle \longrightarrow n}{\langle X := n, (\Delta, \sigma) \rangle \rightsquigarrow \langle \text{skip}, (\Delta, \sigma[\delta(X)/n]) \rangle} \\[10pt]
\frac{\langle c_0, (\Delta, \sigma) \rangle \rightsquigarrow \langle \text{skip}, (\Delta', \sigma') \rangle}{\langle c_0; c_1, (\Delta, \sigma) \rangle \rightsquigarrow \langle c_1, (\Delta', \sigma') \rangle} \quad \frac{\langle c_0, (\Delta', \sigma') \rangle \rightsquigarrow \langle c'_0, (\Delta', \sigma') \rangle}{\langle c_0; c_1, (\Delta, \sigma) \rangle \rightsquigarrow \langle c'_0; c_1, (\Delta', \sigma') \rangle} \\[10pt]
\frac{\langle b, (\Delta, \sigma) \rangle \rightsquigarrow \text{true}}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, (\Delta, \sigma) \rangle \rightsquigarrow \langle c_0, (\Delta, \sigma) \rangle} \\[10pt]
\frac{\langle b, (\Delta, \sigma) \rangle \rightsquigarrow \text{false}}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, (\Delta, \sigma) \rangle \rightsquigarrow \langle c_1, (\Delta, \sigma) \rangle} \\[10pt]
\frac{}{\langle \text{while } b \text{ do } c, (\Delta, \sigma) \rangle \rightsquigarrow \langle \text{if } b \text{ then } c; \text{while } b \text{ do } c \text{ else skip}, (\Delta, \sigma) \rangle} \\[10pt]
\frac{\langle d, (\text{top}(\Delta), \sigma) \rangle \Rightarrow_d \delta'}{\langle \{d; c\}, (\Delta, \sigma) \rangle \Longrightarrow \langle \text{block}(c), (\text{push}(\delta', \Delta), \sigma) \rangle} \\[10pt]
\frac{}{\langle \text{block}(\text{skip}), (\Delta, \sigma) \rangle \Longrightarrow \langle \text{skip}, (\text{pop}(\Delta), \sigma) \rangle} \\[10pt]
\frac{\langle c, (\Delta, \sigma) \rangle \rightsquigarrow \langle c', (\Delta', \sigma') \rangle}{\langle \text{block}(c), (\Delta, \sigma) \rangle \Longrightarrow \langle \text{block}(c'), (\Delta', \sigma') \rangle}
\end{array}$$

$$\frac{\delta = \text{top}(\Delta) \quad \delta(f) = \text{proc}(X, c) \quad l = \text{newloc}() \quad \langle a, (\delta, \sigma) \rangle \longrightarrow v}{\langle \text{call } f(a), (\Delta, \sigma) \rangle \Longrightarrow \langle \text{block}(c), (\text{push}(\delta[X/l], \Delta), \sigma[l/v]) \rangle}$$

$$\frac{\delta = \text{top}(\Delta) \quad \delta(f) = \text{proc}(\text{rif } X, c) \quad Y \in \text{Name}}{\langle \text{call } f(Y), (\Delta, \sigma) \rangle \Longrightarrow \langle \text{block}(c), (\text{push}(\delta[X/\delta(Y)], \Delta), \sigma) \rangle}$$

4.3 Side Effects

Abbiamo visto che il passaggio dei parametri per riferimento ha degli effetti che sono, per così dire, effetti non espliciti e per questo sono chiamati effetti collaterali o *side effects*. Anche l'aggiornamento di una variabile che non è dichiarata localmente ha in qualche modo un side effect. Diamone una definizione.

Definizione 4.2 *Una funzione o un pezzo di codice produce un side effect quando modifica un valore o uno stato al di fuori del proprio ambito locale.*

Nel paradigma funzionale non abbiamo in generale side effects dato che la valutazione di un termine restituisce sempre un valore, mentre nel paradigma imperativo (dichiarativo) i side effects sono presenti. Nel caso della chiamata di una funzione con la modifica di una variabile ereditata dall'ambiente abbiamo un effetto collaterale. Questo è anche un modo per comunicare tra il programma chiamante ed il programma chiamato.

4.4 Scope Statico

Fino ad ora abbiamo considerato solo lo scope dinamico: quando si chiama una procedura questa viene eseguita nell'ambiente e memoria (lo stato) che sono attivi al momento della chiamata. Con lo scope statico l'ambiente è quello al momento della scrittura del programma, esattamente come nei linguaggi funzionali. E come nei linguaggi funzionali abbiamo bisogno di arricchire il linguaggio per avere le procedure ricorsive. Alle dichiarazioni aggiungiamo dunque la produzione $d \rightarrow \text{procrif } f(p)\{c\}$ che indica che la procedura f è ricorsiva.

Ricordiamo che gli oggetti denotabili sono ora $\text{Obj}_d = \mathbb{Z} \uplus \text{Loc} \uplus \text{Proc}$ dove $\text{Proc} = \{(\text{proc}(X, c), \delta) \mid X \text{ è un nome, } c \text{ è un comando e } \delta \text{ è un ambiente}\}$. Le regole della dichiarazione sono quindi

$$\overline{\langle \text{proc } f(p)\{c\}, (\delta, \sigma) \Rightarrow_d \delta[f/(\text{proc}(p, c), \delta)] \rangle}$$

$$\frac{\hat{\delta}(f) = (\text{proc}(p, c), \hat{\delta})}{\langle \text{procrif } f(p)\{c\}, (\delta, \sigma) \Rightarrow_d \delta[f/(\text{proc}(p, c), \hat{\delta})] \rangle}$$

nella regola per le procedure ricorsive come premessa mettiamo che l'ambiente deve contenere un'associazione in cui all'identificatore della procedura è associata la coppia in cui l'ambiente è il più piccolo che contiene l'associazione per quel nome.

La regola per l'invocazione di una procedura sarà:

$$\frac{\delta(f) = (\text{proc}(X, c), \delta') \quad l = \text{newloc}() \quad \langle a, (\delta, \sigma) \rangle \longrightarrow v \quad \langle c, (\delta'[X/l], \sigma[l/v]) \rangle \Longrightarrow \sigma'}{\langle \text{call } f(a), (\delta, \sigma) \rangle \Longrightarrow \sigma'}$$

Le regole per i diversi passaggi dei parametri seguono lo schema visto prima, adattato al fatto che ora la procedura va eseguita nell'ambiente stabilito al momento della scrittura del programma.

4.5 Procedure come parametri

Nel linguaggio funzionale abbiamo visto che le funzioni possono essere dei parametri passati al momento dell'invocazione di una funzione: nel termine **apply**(t_1, t_2) il termine t_2 può essere una funzione. Anche nei linguaggi imperativi abbiamo questa possibilità. Non abbiamo sostanzialmente bisogno di cambiare nulla.

4.5.1 Scope Statico

Quando passiamo una procedura come parametro dobbiamo solo far sì che venga passata per riferimento. Nell'ambiente al nome della procedura sarà associato il corpo della funzione/procedura che deve essere eseguito e l'ambiente in cui eseguire la chiamata di procedura o funzione. La regola non cambia:

$$\frac{\delta(f) = (\text{proc}(\text{rif } X, c), \delta') \quad Y \in \text{Name} \quad \langle c, (\delta'[X/\delta(Y)], \sigma) \rangle \Longrightarrow \sigma'}{\langle \text{call } f(Y), (\delta, \sigma) \rangle \Longrightarrow \sigma'}$$

4.5.2 Scope Dinamico

Anche nello scope dinamico bisogna passare le procedure per riferimento. Però in questo caso bisogna distinguere due casi, il primo, chiamato *shallow binding* è sostanzialmente lo scope dinamico, quindi la regola è

$$\frac{\delta(f) = \text{proc}(\text{rif } X, c) \quad Y \in \text{Name} \quad \langle c, (\delta[X/\delta(Y)], \sigma) \rangle \Longrightarrow \sigma'}{\langle \text{call } f(Y), (\delta, \sigma) \rangle \Longrightarrow \sigma'}$$

Il secondo caso invece implica che l'ambiente dove eseguire la procedura passata come parametro non è quello al momento della chiamata della procedura passata come parametro ma è quello al momento della chiamata della procedura che riceve il parametro. La regola diventa quindi

$$\frac{\delta(f) = \text{proc}(\text{rif } X, c) \quad Y \in \text{Name} \quad \langle c, (\delta[X/(\delta(Y), \delta)], \sigma) \rangle \Longrightarrow \sigma'}{\langle \text{call } f(Y), (\delta, \sigma) \rangle \Longrightarrow \sigma'}$$

Ovviamente se una procedura ha un ambiente questo è quello da usare.

4.6 Procedure come risultato

Una procedura può anche essere il risultato della chiamata di una procedura. Introduciamo un nuovo comando: **return** *id* dove *id* è un identificatore. Per poter dare semantica a questo costrutto dobbiamo modificare quale è il risultato della semantica, che fino ad ora era sostanzialmente una memoria. Ora dobbiamo aggiungere anche le procedure. Quindi **Mem** conterrà non solo le associazioni locazioni con oggetti denotabili ma anche **Proc** che prima erano solo denotabili.

La semantica di questo comando sarà quindi la seguente:

$$\frac{\delta(X) \in \text{Proc}}{\langle \text{return } X, (\delta, \sigma) \rangle \Longrightarrow \delta(X)}$$

Dobbiamo considerare che il risultato della chiamata di una procedura può essere una procedura, quindi dobbiamo tenerne conto nella semantica della chiamata:

$$\frac{\mathbb{C} \in \text{Com} \quad \langle \mathbb{C}, (\delta, \sigma) \rangle \Longrightarrow \text{proc}(X, c)}{\langle \text{call } \mathbb{C}, (\delta, \sigma) \rangle \Longrightarrow \text{proc}(X, c)}$$

Va osservato che ora, se la procedura p restituisce una procedura, per poterla usare bisognerà scrivere $\text{call } (\text{call } p)$ dove le parentesi sono messe per eliminare ambiguità.

4.7 Funzioni come espressioni e side effects

Come abbiamo arricchito i comandi con chiamate di procedura, possiamo anche arricchire le espressioni con le chiamate di funzioni, che termineranno con il comando **return** e dove e è un'espressione. Tra le espressioni dovremo mettere una chiamata alla funzione e tra le dichiarazioni anche la dichiarazione di funzione. Quindi aggiungiamo la dichiarazione $d \rightarrow \text{fun } f(pf)\{c; e\}$ e tra le espressioni $e \rightarrow \text{callf } f(pa)$. Ovviamente dobbiamo aggiungere agli oggetti denotabili anche le dichiarazioni di funzione, quindi $\text{Fun} = \{\text{fun}(X, c, e) \mid X \text{ è un nome, } c \text{ è un comando ed } e \text{ è un'espressione}\}$. Per le dichiarazioni dobbiamo aggiungere la regola

$$\overline{\langle \text{fun } f(p)\{c; e\}, (\delta, \sigma) \Rightarrow_d \delta[f/\text{fun}(p, c, e)]}$$

Se prima la valutazione di un'espressione restituiva un numero intero, generalizziamo e diciamo che restituisce un valore in **Value** che contiene anche **Fun** e non solo gli interi. Ora la valutazione di un'espressione non restituisce solo un valore ma può dare anche effetti collaterali (per semplicità:

$$\begin{array}{c} \frac{}{\langle v, (\delta, \sigma) \rangle \longrightarrow (v, \sigma)} \quad \frac{\delta(X) \downarrow \quad \delta(X) \in \text{Loc}}{\langle X, (\delta, \sigma) \rangle \longrightarrow (\sigma((\delta(X))), \sigma)} \quad \frac{\delta(X) \downarrow \quad \delta(X) \in \text{Value}}{\langle X, (\delta, \sigma) \rangle \longrightarrow (\delta(X), \sigma)} \\[10pt] \frac{\langle a_0, (\delta, \sigma) \rangle \longrightarrow (n_0, \sigma'') \quad \langle a_1, (\delta, \sigma'') \rangle \longrightarrow (n_1, \sigma') \quad n = n_0 + n_1}{\langle a_0 + a_1, (\delta, \sigma) \rangle \longrightarrow (n, \sigma')} \\[10pt] \frac{\langle a_0, (\delta, \sigma) \rangle \longrightarrow (n_0, \sigma'') \quad \langle a_1, (\delta, \sigma'') \rangle \longrightarrow (n_1, \sigma') \quad n = n_0 \times n_1}{\langle a_0 \times a_1, (\delta, \sigma) \rangle \longrightarrow (n, \sigma')} \\[10pt] \frac{\langle a_0, (\delta, \sigma) \rangle \longrightarrow (n_0, \sigma'') \quad \langle a_1, (\delta, \sigma'') \rangle \longrightarrow (n_1, \sigma') \quad n = n_0 - n_1}{\langle a_0 - a_1, (\delta, \sigma) \rangle \longrightarrow (n, \sigma')} \end{array}$$

e per le chiamate abbiamo le due regole (passaggio per riferimento e per valore):

$$\frac{\delta(f) = \text{fun}(\text{rif } X, c, e) \quad Y \in \text{Name} \quad \langle c, (\delta[X/\delta(Y)], \sigma) \rangle \Longrightarrow \sigma'' \quad \langle e, (\delta[X/\delta(Y)], \sigma'') \rangle \longrightarrow (v, \sigma')}{\langle \text{callf } f(Y), (\delta, \sigma) \rangle \longrightarrow (v, \sigma')}$$

$$\frac{\delta(f) = \text{fun}(X, c, e) \quad l = \text{nl}() \quad \langle a, (\delta, \sigma) \rangle \longrightarrow (v, \sigma_1) \quad \langle c, (\delta_1, \sigma'_1) \rangle \Longrightarrow \sigma_2 \quad \langle e, (\delta_1, \sigma_2) \rangle \longrightarrow (v, \sigma')}{\langle \text{callf } f(a), (\delta, \sigma) \rangle \longrightarrow (v, \sigma')}$$

dove $\delta_1 = \delta[X/l]$ e $\sigma'_1 = \sigma_1[l/v]$.

Capitolo 5

Dati

5.1 Dati e Tipi di Dato

Cosa è un *dato*? In generale è un oggetto appartenente ad un qualche *tipo* che viene manipolato dal programma. Possiamo quindi dare una definizione di *tipo di dato*.

Definizione 5.1 *Un tipo di dato è una collezione di valori omogenei ed effettivamente presentati, dotata di un insieme di operazioni che manipolano tali valori.*

Abbiamo che i valori di un tipo di dato devono essere *omogenei*, cioè devono avere le stesse caratteristiche), e devono essere *effettivamente* presentati, quindi abbiamo per ogni valore una rappresentazione effettiva.

Non deve sfuggire il fatto che la definizione implica che i valori sono un *insieme*, che questo insieme è dotato di operazioni (un'algebra) e la richiesta che siano effettivamente presentati implica che li sappiamo manipolare in una macchina astratta. Bisogna fare *attenzione*: cosa è un tipo e cosa invece non lo è dipende *fortemente* dal linguaggio di programmazione

A che servono i tipi di dato? Nel progetto della soluzione di un problema consentono di organizzare l'informazione. Tipi diversi per concetti diversi e un identificatore deve venir usato con il tipo corretto: il tipo è una sorta di *commento* sull'uso dell'identificatore, mentre al momento della scrittura del programma consentono di identificare e prevenire errori. Quindi i tipi sono controllabili automaticamente, e sono un controllo *dimensionale* (3+"pippo" deve essere sbagliato). Infine, al momento dell'implementazione del linguaggio permettono alcune ottimizzazioni, ad esempio `bool` richiede meno bit di `real` e si può fare un precalcolo di offset per l'accesso a

5.1.1 Sistema di tipi

Definizione 5.2 *Un sistema di tipi è il complesso delle informazioni e delle regole che governano i tipi di un linguaggio di programmazione.*

Nel sistema di tipi ci saranno i tipi predefiniti (spesso detti di base), i meccanismi per definire nuovi tipi e dei meccanismi relativi al controllo dei tipi. In particolare meccanismi per dire se due tipi sono equivalenti (quali sono formalmente diversi ma sono poi lo stesso tipo), quali sono compatibili (quando un valore di un tipo può essere usato come se fosse di un altro tipo) e come fare per attribuire un tipo ad un'espressione complessa (inferenza di tipo). Inoltre il sistema di tipi specifica se i controlli (e anche quali controlli) sono da effettuare staticamente (prima dell'esecuzione) o dinamicamente (al momento dell'esecuzione).

5.1.2 Type safeness

Definizione 5.3 *Un sistema di tipi è type safe se in esecuzione non possono avvenire errori non segnalati derivanti da un errore di tipo.*

Un linguaggio type safe viene detto anche fortemente tipizzato.

5.1.3 Tipi di dato base

I valori di un tipo possono essere

- *denotabili*: possono essere associati ad un nome
- *esprimibili*: possono essere il risultato della valutazione di un'espressione complessa
- *memorizzabili*: possono essere associati ad una locazione

Ovviamente le scelte di cosa è denotabile, memorizzabile, esprimibile dipendono dal linguaggio (e quindi dalla sua progettazione e implementazione)

I controlli di tipo possono essere statici, fatti cioè al momento della compilazione, quindi la type safeness può essere garantita, ma la progettazione del linguaggio è più complessa e si possono *rifiutare* programmi che sono sostanzialmente eseguibili, come `if true then "pippo" else 1` che potrebbe venir eseguito senza problemi, oppure dinamici cioè fatti al momento dell'esecuzione. In questo caso un oggetto ha un descrittore a run time che la macchina astratta usa e la macchina astratta controlla i descrittori e prova se sono corretti. Questo può essere fonte di inefficienza. La soluzione è che spesso si combinano tutti e due i metodi di controllo.

Passiamo in rassegna vari tipi di dato.

- *Booleani*. I *valori* sono vero e falso, le *operazioni* sono le operazioni logiche e la *rappresentazione* dei valori è in generale un byte (per motivi di indirizzabilità).

Il C non ha un tipo booleano.

I booleani sono in generale esprimibili, denotabili e memorizzabili.

- *Caratteri*. I *valori* sono un insieme di codici di caratteri, le *operazioni* sono l'uguaglianza, code/decode, o dipendenti dal linguaggio. La *rappresentazione* è un byte (ASCII), o due byte (UNICODE).

In generale esprimibili, denotabili e memorizzabili.

- *Interi*. I *valori* sono $0, 1, -1, 2, -2, \dots$, maxint, le *operazioni* sono $+, \times, -, \div, mod, \dots$ e la *rappresentazione* è con alcuni byte, e complemento a due.

Interi e interi lunghi (anche 8 byte) possono avere limitati problemi nella portabilità (se la lunghezza non è specificata nella definizione del linguaggio).

In generale esprimibili, denotabili e memorizzabili.

- *Reali*. I *valori* sono i razionali in un certo intervallo, le *operazioni* sono $+, \times, -, \backslash, \dots$ e la *rappresentazione* è data alcuni byte, virgola mobile (standard IEEE 754).

In generale esprimibili, denotabili e memorizzabili.

- *Virgola fissa*. I *valori* sono i razionali in un certo intervallo, le *operazioni* sono $+, \times, -, \backslash, \dots$, e la *rappresentazione* è in generale di alcuni byte, virgola fissa (complemento a 2).

In Ada sono usati per operazioni efficienti con poca precisione.

Sono in generale esprimibili, denotabili e memorizzabili.

- *Complessi*. I *valori* sono un opportuno sottoinsieme di coppie di numeri reali e le *operazioni* sono le solite definite sulle componenti: $+$, $\times$, $-$, $\backslash$, $\dots$ e ovviamente la loro *rappresentazione* è una coppia di numeri in virgola mobile.

Sono in generale esprimibili, denotabili e memorizzabili.

- *Void*. Il *valore* è un solo elemento, non ci sono operazioni e non ha rappresentazione.

Il valore non è esprimibile, denotabile e memorizzabile

Indica il tipo delle operazioni che modificano lo stato e non restituiscono un valore

Quelli che abbiamo elencato sono tra i *tipi scalari*, detti anche tipi semplici, o di base. Nei tipi scalari i valori non sono aggregazioni di altri valori e hanno una rappresentazione diretta nell'implementazione, inoltre possono essere ordinati (quindi è possibile definire un successore e predecessore), e per questa caratteristica è possibile usarli per iterare, si possono usare come *indici*.

Con il Pascal si introducono i tipi enumerati, seguendo la filosofia che gli elementi costitutivi di un programma siano “algoritmi” e “strutture dati”. Un esempio è il tipo *giorni* dichiarato in questo modo: `type Giorni = (Lun, Mar, Mer, Giov, Ven, Sab, Dom)`, In questo modo si possono scrivere programmi di più semplice comprensione. I valori di un tipo enumerato sono ordinati (ad esempio `Mar < Ven`) ed è quindi possibile compiere iterazione sui valori (possiamo scrivere `for i:= Lun to Sab do ...`), ed hanno un `succ` e `pred`

Sono spesso rappresentati come short integer. In C il tipo `enum giorni = {Lun, Mar, Mer, Giov, Ven, Sab, Dom}` equivale a `typedef int giorni; e const giorni Lun=0, Mar=1, Mer=2, ..., Dom=6;`, quindi in C `Mar` e il numero 1 sono uguali, mentre nel Pascal sono diversi. In generale i valori dei tipi enumerati sono esprimibili, denotabili e memorizzabili.

Un altro tipo di dato introdotto in Pascal è il tipo “intervallo”, I valori sono un intervallo dei valori di un tipo scalare (il tipo base dell'intervallo), ad esempio `type MenoDiDieci = 0..9` o `type GiorniLav = Lun..Ven`. I valori di questo tipo sono rappresentati come il tipo base. Si usa un tipo intervallo invece del suo tipo base per avere documentazione “controllabile” ed anche generazione di codice efficiente. Sono in generale esprimibili, denotabili e memorizzabili.

I tipi visti fino ad ora sono detti anche *tipi ordinali* e sono i tipi naturali su cui definire gli indici di tipi composti (come gli array).

5.1.4 Tipi composti

I tipi composti sono spesso chiamati *tipi strutturati*, o non scalari.

- *Record*: collezione di campi (fields), ciascuno di un (diverso) tipo, ed un campo viene selezionato col suo nome
- *Record varianti*: record dove solo alcuni campi (mutuamente esclusivi) sono attivi ad un dato istante
- *Array*: sono una *funzione* da un tipo indice (scalare) ad un altro tipo. Gli array di caratteri sono chiamati *stringhe*, ed hanno operazioni speciali
- *Insiemi*: sottoinsieme di un tipo base (da non confondere con il tipo intervallo)
- *Puntatore*: riferimento (reference) ad un oggetto di un altro tipo

Vediamoli. Il tipo di dato *Record* serve a manipolare in modo unitario dati di tipo eterogeneo (basi di dati). In C, C++, CommonLisp, Algol68 sono definiti con la keyword **struct** (strutture), mentre

Java non ha tipi record, sono sussunti dalle classi. Ad esempio, dichiariamo il seguente tipo record (simil C)

```
struct studente {
    char nome[20];
    int matricola;
};
```

e la selezione di campo avviene in questo modo

```
studente s;
s.matricola=343536;
```

I record possono essere annidati, e sono in genere memorizzabili, esprimibili e denotabili. Il Pascal non ha modo di esprimere “un valore record costante” mentre C lo può fare, ma solo nell’inizializzazione (*initializer*), e l’uguaglianza non è generalmente definita (eccezione: Ada). Quando sono memorizzabili, si ha la memorizzazione sequenziale dei campi, con l’allineamento alla parola (16/32 bit), che comporta un potenziale spreco di memoria, ma consente l’accesso con offset. Con i *Packed records* si ha il disallineamento ma accesso più costoso (devo ricordare le posizioni, offset da ricordare).

I *Record varianti* sono record in cui alcuni campi sono alternativi tra loro: solo uno di essi è attivo in un dato istante:

```
type stud = record
    nome : packed array [1..6] of char;
    matricola: integer;
    case fuoricorso : Boolean of
        true: (ultimoanno: 2000..maxint);
        false: (anno:(primo,secondo,terzo);
                inpari: Boolean;);
end;
```

Se avessimo `var s: stud;` e `s.fuoricorso := true;` allora solo il campo `s.ultimoanno:= 2001;` può essere usato. I due campi (le due varianti) `ultimoanno` e `anno` possono condividere la stessa locazione di memoria. Il tipo del tag `fuoricorso` può essere un qualsiasi tipo scalare. I record varianti sono possibili in molti linguaggi. In C abbiamo il tipo `union` e il solito record (`struct`), e l’unione in C rappresenta due alternative mutuamente esclusive

```
struct student {
    char nome[6];
    int matricola;
    bool fuoricorso;
    union {
        int ultimoanno;
        struct {
            int anno;
            bool inpari;} stud_in_corso
    } campivarianti;
};
```

per accedervi

```
student s;
s.campivarianti.stud_in_corso.inpari
```

Oltre ad una scrittura più complessa che altri linguaggi per l'accesso, il problema è di sicurezza, dato che non c'è un legame tra il valore formale del tag del record e la significatività di una delle varianti (se, ad esempio, contiene un valore e di quale tipo), e anche se introduco controlli non risolvo realmente il problema (il problema sta nell'unione).

```
type Tre = 1..3;
var tmp = record
    case quale : Tre of
        1 : (a : int);
        2 : (b : bool);
        3 : (c : char);
    end
```

e con

```
tmp.quale := 1;
tmp.a := 12;
write(tmp.c) /* errore dinamico
```

ma se prima del write faccio `tmp.quale := 3` non c'è errore, anche se il contenuto non è significativo.

Le Unioni in Algol 68 sono invece sicure

```
union (int, bool, char) tmp;
...
tmp := true;
...
tmp := 123;
```

La macchina astratta mantiene per ogni variabile di tipo unione un tag nascosto che viene settato al momento dell'assegnamento, e quando uso la variabile devo prevedere ogni *caso*

```
case tmp of
    (int a) : a := a+1;
    (bool b) : b := false;
    (char c) : print(c);
esac
```

Questo ovviamente rende la programmazione e l'implementazione più pesante.

Gli *array* sono invece collezioni di dati *omogenei* e sono una *funzione* da un tipo indice al tipo degli elementi, quindi il tipo indice è in genere un tipo discreto mentre l'*elemento* può essere un qualsiasi tipo (raramente un tipo funzionale)

Un array lo si dichiara nei seguenti modi:

- C: `int vet[30]`; il tipo indice sono i valori tra 0 e 29
- Pascal: `var vett : array [0..29] of integer;`

Gli array multidimensionali (matrici) sono una funzione da tipo indice a tipo array. In Pascal le seguenti sono equivalenti

```
var mat : array [0..29,'a'..'z'] of real;
var mat : array [0..29] of array ['a'..'z'] of real;
```

ma non sono equivalenti in Ada (la seconda permette slicing, vedremo tra poco cosa è). Da ultimo osserviamo che il C fonde array e puntatori. La principale operazione permessa sugli array è la selezione di un elemento (`vet[3]` o `mat[10,'c']`), mentre la modifica non è un'operazione sull'array, ma sulla locazione modificabile che memorizza l'elemento di un array. Alcuni linguaggi permettono lo slicing, cioè la selezione di parti contigue di un array (in Ada, data la dichiarazione `mat : array(1..10) of array(1..10) of real;`, con `mat(3)` si indica la terza riga della matrice quadrata `mat`). Se si ha lo slicing si possono avere anche operazioni sugli array:

- Fortran 90: `A+B` somma gli elementi di `A` e `B` (dello stesso tipo)
- l'APL è linguaggio (dei primissimi anni '60) per manipolazioni di array, e ha potenti operazioni su array, è un linguaggio sintetico e austero, il lessico è *matematico* (lettere greche) ed è impossibile da leggere (e da scrivere sulle telescriventi dell'epoca)

La notazione è importante: *Notation as a tool for thought* di Kenneth E. Iverson, ACM Turing Lecture del 1979

La memorizzazione degli array è stata molto importante (in generale gli array vanno nel RdA) Gli elementi sono in genere memorizzati in locazioni contigue, e l'ordine può essere o di riga o di colonna. Ordine di riga: `V[1,1];V[1,2];...;V[1,10];V[2,1];...`, è quello maggiormente usato, il "subarray" di un array di array (ad esempio la terza riga) sta in locazioni contigue. Nell'ordine di colonna `V[1,1];V[2,1];V[3,1];...;V[10,1];V[1,2];...` le colonne stanno in locazioni contigue. L'ordine è rilevante per l'efficienza in sistemi con cache. Come calcolare degli indirizzi: per calcolare la locazione corrispondente a `A[i,j,k]` (memorizzato per riga)

```
A : array[l1..u1, l2..u2, l3..u3] of elem_type;
```

- `S3` dimensione di (un elemento di) `elem_type`
- `S2 = (u3-l3+1)*S3` è la dimensione di una riga
- `S1 = (u2-l2+1)*S2` è la dimensione di un piano

la locazione di `A[i,j,k]` è

```
 $\alpha$           indirizzo di inizio di A
+ (i-l1)*S1  indirizzo di inizio del piano di A[i,j,k]
+ (j-l2)*S2  indirizzo di inizio della riga di A[i,j,k]
+ (k-l3)*S3  indirizzo di A[i,j,k]
```

Se il numero di dimensioni è noto a tempo di compilazione, si può riorganizzare come segue `i*S1+j*S2+k*S3+  $\alpha$  - (l1*S1+l2*S2+l3*S3)`, e se l'indice inizia sempre da zero (`l1=l2=l3=0`) non c'è bisogno di precomputazione.

La *forma*, o *shape*, di un array è il numero delle dimensioni e l'intervallo dell'indice. Se la forma è statica al momento della compilazione (nell'RdA si prevede lo spazio che serve) oppure lo si fa al momento dell'elaborazione della dichiarazione, e anche in questo caso nell'RdA si prevede lo spazio che serve, lo si fa nel codice del preambolo, ma l'RdA si divide nella parte dati di dimensioni *fisse* e quella di dati di dimensioni *non determinabili* al momento della compilazione. Le informazioni su come trovare effettivamente gli elementi devono essere mantenute in qualche modo (il *dope vector*). Nel caso la forma sia dinamica, allora l'array può cambiare forma durante l'esecuzione del programma.

Il *dope vector* mantiene le informazioni sulla forma dell'array, che nel caso statico non può essere manipolata se non al momento della dichiarazione. Il *dope vector* di un array contiene:

- il puntatore all'inizio dell'array (nella parte variabile dell'RdA)
- il numero delle dimensioni
- il limite inferiore (se la forma cambia a run-time vogliamo controllare anche l'out-of-bound, quindi serve anche il limite superiore)
- occupazione di memoria per ogni dimensione (valore `Si`)

Il `dope vector` è memorizzato nella parte fissa del RdA. L'accesso viene effettuato calcolando tutto l'indirizzo a run-time, usando il `dope vector`.

L'array in Java non c'è. `int[] V`; equivale a fare `V = new int[]` che alloca spazio sullo heap e dà il suo riferimento a `V`.

Vediamo ora i *Puntatori*. Dato un *indirizzo* (locazione), *dereferenziare* significa accedere al dato *puntato* (indirizzato) dall'indirizzo. Nel seguito locazione e *l*-valore saranno sinonimi, e i puntatori sono le locazioni. Ci sono linguaggi con variabili modificabili che permettono di riferirsi (e modificare) un *l*-valore senza dereferenziarlo, promuovendo gli *l*-valori a dati che si manipolano. Quindi abbiamo che per i puntatori abbiamo che i *valori* sono proprio le locazioni (*l*-valori o riferimenti) e `null` (o `nil`) è il puntatore che non punta a nulla. Le *operazioni* sono la creazione, la dereferenziazione e l'uguaglianza. La loro *rappresentazione* è data dagli indirizzi. Se gli oggetti puntati sono di un certo tipo (`T`), i puntatori hanno in qualche modo anche il tipo dell'oggetto puntato (si indica spesso con `T*`), e in alcuni linguaggi i puntatori possono essere trattati aritmeticamente (ma si perde certamente la type safeness).

In C i puntatori possono essere creati a partire dal dato

```
float r = 3.1415;
float* q;
q = &r
```

in `q` c'è l'indirizzo di `r`.

I puntatori sono in generale usati per le strutture dati ricorsive

```
type int_list = elemento*;
type elemento = struct {
    int val;
    int_list next;
};
```

In C array e puntatori sono la stessa cosa

```
int n;
int *a;      // puntatore a interi
int b[10];   // array di 10 interi
...
a = b;       // a punta all'elemento iniziale di b
n = a[3];    // n ha il valore del terzo elemento di b
n = *(a+3);  // idem
n = b[3];    // idem
n = *(b+3);  // idem
```

array multidimensionali: `b[i][j]` è la stessa cosa di `*(b+i))[j]` o `*(b[i]+j)` o ancora `*(*(b+i)+j)`.

Un puntatore implica in generale che si alloca della memoria sullo heap, e questa si esaurisce, ma le locazioni libere vengono recuperate con la *garbage collection*. Si può prevedere la deallocazione esplicita, ma il problema è il solito: la vita breve e i dangling reference.

```
int* p;
int* q;
p = (int*) malloc(sizeof(int));
*p = 5;
q = p;    // alias
free(p);
p = null;
...      // non modifico p o q
print(*p) // la macchina astratta rileva l'errore (grazie al null)
print(*q) // errore non rilevabile: dangling reference...
```

I *tipi ricorsivi* sono tipi composti da valori di un tipo e riferimenti a valori di quel tipo, gli oggetti di questo tipo sono allocati sullo heap e dipendono fortemente dal linguaggio. Nel ML i tipi ricorsivi sono presenti esplicitamente e non devo preoccuparmi di allocazioni (una buona norma nella definizione di un tipo ricorsivo è la stessa per la definizione induttiva: bisogna avere i casi base), mentre in Java l'allocazione avviene con la **new** (che serve per allocare lo spazio sullo heap) e c'è anche il **null**.

Da ultimo vediamo il tipo *funzione*. In generale le funzioni sono denotabili e, in linguaggi funzionali, esprimibili, ma in generale non sono memorizzabili.

5.1.5 Sistema di tipi, equivalenza, compatibilità, polimorfismo e regole

Prima di vedere cosa è un sistema di tipi effettivamente ricordiamo che abbiamo visto come formare tipi a partire da altri tipi. Possiamo ora domandarci quali sono le regole che governano la correttezza di un programma rispetto ai tipi. I nuovi tipi li dichiariamo in questo modo

```
type nome = espressione
```

dove **espressione** è ottenuta combinando opportunamente i costruttori e tipi base visti prima. Abbiamo due tipi di definizione di tipo, una è la definizione detta *opaca* (che ha per equivalenza quella per *nome*) e l'altra è la definizione *trasparente* (che ha per equivalenza quella *strutturale*).

L'equivalenza è una relazione transitiva, riflessiva e simmetrica, mentre la compatibilità è una relazione transitiva e riflessiva.

Definizione 5.4 *Due tipi sono equivalenti per nome solo se hanno lo stesso nome.*

Un tipo è equivalente solo a se stesso (usata in Pascal, Ada, Java). In Pascal questi sono due tipi diversi (non hanno lo stesso nome):

```
var V : array[1..10] of integer;
    W : array[1..10] of integer;
```

L'equivalenza può essere *lasca* (loose, in Pascal o Modula-2): una dichiarazione di un alias di tipo non genera un nuovo tipo, ma solo un nuovo nome

```
type A : espressione;
type B = A;
```

A e B sono due nomi dello stesso tipo.

Definizione 5.5 Si consideri la minima relazione d'equivalenza sui tipi (indicata con $\sim$) tale che

- un nome di tipo è equivalente a se stesso
- se un tipo T è introdotto con una definizione di tipo **type** $T = \text{espressione}$, allora T è equivalente a **espressione**
- se due tipi T e S sono costruiti usando lo stesso costruttore di tipo a tipi equivalenti, allora sono equivalenti

Due tipi T e S sono equivalenti strutturalmente se solo se $T \sim S$

Se abbiamo

```
type T1 = int;
type T2 = char;
type T3 = struct {
    T1 a;
    T2 b;}
type T4 = struct {
    int a;
    char b;}
```

allora $T3 \sim T4$.

L'equivalenza strutturale è governata dalla riscrittura: un tipo complesso viene *riscritto* nei suoi componenti elementari. A basso livello non rispetta l'astrazione desiderata dal programmatore. Quasi tutti i linguaggi hanno un qualche tipo d'equivalenza (o strutturale, o per nome, o per nome loose).

Definizione 5.6 T è compatibile con S quando oggetti di T possono essere usati in un contesto dove ci si attende valori S .

La definizione dipende in modo cruciale dal linguaggio. T è compatibile con S se

- T e S sono equivalenti
- i valori di T sono un sotto insieme dei valori di S (un intervallo)
- tutte le operazioni sui valori di S sono possibili anche sui valori di T (principio di *estensione* di record)
- i valori di T corrispondono in modo canonico ad alcuni valori di S (ad esempio `int` e `float`)
- i valori di T possono essere fatti corrispondere ad alcuni valori di S (`float` e `int` con troncamento)

Tra tipi compatibili si può anche definire (implicitamente o esplicitamente) un'operazione di conversione. implicita (coercion) o esplicita (indicata nel testo)

La *coercion* è una conversione implicita (detta anche coercizione): la macchina astratta inserisce la conversione, senza che ve ne sia traccia a livello linguistico. La coercizione serve per indicare una situazione di compatibilità e per indicare cosa deve fare l'implementazione.

Se i tipi sono diversi ma hanno gli stessi valori e stessa rappresentazione (ad esempio, tipi strutturalmente uguali ma nomi diversi), la conversione c'è solo a tempo di compilazione, non c'è codice; se invece i tipi hanno valori diversi, ma la stessa rappresentazione nell'intersezione (ad esempio intervalli

e interi), allora c'è del codice per il controllo dinamico sull'appartenenza all'intersezione, infine se hanno valori e rappresentazione diversi (ad esempio, interi e reali), allora c'è bisogno di codice per la conversione.

Il *cast* è invece un operatore di conversione esplicita: il programmatore deve inserire esplicite conversioni di tipo. Possiamo considerarle come annotazioni nel linguaggio che specificano che un valore di un tipo deve essere convertito in un altro tipo. Il **cast** è un costrutto del linguaggio. Non è consentita ogni conversione esplicita, ma solo quelle per le quali il linguaggio conosce come implementare la conversione, e si può sempre inserire un cast laddove esiste una compatibilità. I linguaggi moderni tendono a favorire il cast rispetto alla coercion.

5.2 Polimorfismo

Definizione 5.7 *Un sistema di tipi nel quale uno stesso oggetto può avere più di un tipo è detto polimorfo. Un oggetto è polimorfo quando il sistema di tipi gli assegna più di un tipo.*

Si può avere polimorfismo *ad hoc* (overloading), o polimorfismo *universale* (esplicito, implicito, di sottotipo). L'*overloading* non è vero polimorfismo. Uno stesso simbolo denota significati diversi, ad esempio se abbiamo '3 + 5' e '4.5 + 5.3' il compilatore traduce il + in modi diversi, a seconda del contesto. L'*overloading* è sempre risolto a tempo di compilazione, dopo l'inferenza dei tipi, e non va confuso con la coercion. Anche se spesso hanno lo stesso effetto, rispondono a problemi diversi.

Definizione 5.8 *Un valore esibisce polimorfismo universale parametrico quando ha un'infinità di tipi diversi, che si ottengono per istanziazione da un unico schema di tipo generale.*

Una funzione polimorfa è costituita da un unico codice che si applica uniformemente a tutte le istanze del suo tipo generale

```
void swap (reference <T> x, reference <T> y) {
  <T> tmp = x;
  x := y;
  y := tmp;
}
```

`swap` ha tipo $\langle T \rangle \times \langle T \rangle \rightarrow \text{void}$ che viene indicato anche con l'espressione di tipo $\forall T. \langle T \rangle \times \langle T \rangle \rightarrow \text{void}$

Il polimorfismo qui è esplicito, dato che si indica lo schema di tipo. Può essere implicito: il programma non riporta indicazioni sul tipo, ma queste vengono ottenute tramite inferenza (ad esempio *ML*).

Definizione 5.9 *Un valore esibisce polimorfismo di sottotipo (o limitato) quando ha un'infinità di tipi diversi, che si ottengono per istanziazione da uno schema di tipo generale, sostituendo ad un opportuno parametro i sottotipi di un tipo assegnato.*

Il polimorfismo di *sottotipo* è analogo a quello esplicito, ma non tutti i tipi possono essere usati per istanziare il tipo generale, c'è una relazione (data) di sottotipo. $T < S$ significa che T è sottotipo di S .

Una funzione polimorfa è costituita da un unico codice che si applica uniformemente a tutte le istanze *legali* del suo tipo generale, ossia $\forall T < D. \langle T \rangle [] \rightarrow \text{void}$. Il polimorfismo di *sottotipo* è tipico del paradigma object oriented.

5.3 Sicurezza (rispetto ai tipi)

I linguaggi *non sicuri* sono quelli in cui il sistema di tipi è un suggerimento metodologico per il programmatore, ma il linguaggio permette di aggirare o rilassare i controlli (esempio C, C++ e linguaggi della famiglia), i linguaggi *localmente non sicuri* sono quelli in cui il sistema di tipi è ben regolato e i tipi controllati ma ci sono alcuni costrutti che consentono di aggirare i controlli (Algol, Pascal, Ada), e la locale non sicurezza dipende principalmente dalle unioni (varianti non controllate) e deallocazione esplicita. I linguaggi *sicuri* sono quelli in cui il sistema di tipi è ben regolato e i tipi controllati e non ci sono costrutti per aggirare i controlli (Java, ML, Lisp, Scheme).

Vediamo ora le regole per i tipi. In questo caso i tipi sono definiti da una grammatica, ad esempio

$$\tau ::= \mathbf{int} \mid \tau_1 * \tau_2 \mid \tau_1 \Rightarrow \tau_2$$

abbiamo bisogno di un linguaggio

$$t ::= x \mid n \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid (t_1, t_2) \mid \text{fst}(t) \mid \text{snd}(t) \mid \\ \mid \lambda x.t \mid \text{apply}(t_1, t_2) \mid \text{let } x \Leftarrow t_1 \text{ in } t_2 \mid \text{rec } y.(\lambda x.t)$$

ed abbiamo un sistema di regole per assegnare ad ogni espressione del linguaggio un tipo:

$$\begin{array}{c} \frac{}{x : \text{type}(x)} \qquad \frac{}{n : \mathbf{int}} \qquad \frac{t_1 : \mathbf{int} \quad t_2 : \mathbf{int}}{t_1 \text{ op } t_2 : \mathbf{int}} \text{ op} \in \{+, -, \times\} \\ \\ \frac{t_0 : \mathbf{int} \quad t_1 : \tau \quad t_2 : \tau}{\text{if } t_0 \text{ then } t_1 \text{ else } t_2 : \tau} \quad \frac{t_1 : \tau_1 \quad t_2 : \tau_2}{(t_1, t_2) : \tau_1 * \tau_2} \quad \frac{t : \tau_1 * \tau_2}{\text{fst}(t) : \tau_1} \\ \\ \frac{t : \tau_1 * \tau_2}{\text{snd}(t) : \tau_2} \quad \frac{x : \tau_1 \quad t : \tau_2}{\lambda x.t : \tau_1 \Rightarrow \tau_2} \quad \frac{t_1 : \tau_1 \Rightarrow \tau_2 \quad t_2 : \tau_1}{\text{apply}(t_1, t_2) : \tau_2} \\ \\ \frac{x : \tau_1 \quad t_1 : \tau_1 \quad t_2 : \tau_2}{\text{let } x \Leftarrow t_1 \text{ in } t_2 : \tau_2} \quad \frac{y : \tau \quad \lambda x.t : \tau}{\text{rec } y.(\lambda x.t) : \tau} \end{array}$$

5.3.1 Inferenza di tipo

Considerate il linguaggio di programmazione, il cui scope è statico (il linguaggio riflette lo scope statico quando presenta esplicitamente un costrutto per gestire la ricorsione), e la cui sintassi è la seguente:

$$t ::= x \mid n \mid c \mid \text{true} \mid \text{false} \mid \text{empty} \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t_1 \mid t_1 = t_2 \\ \mid t_1 < t_2 \mid \text{cons}(t_1, t_2) \mid \text{head}(t) \mid \text{tail}(t) \mid \text{fst}(t) \mid \text{snd}(t) \mid \text{pair}(t_1, t_2) \mid \\ \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid \text{fun}(x, t) \mid \text{apply}(t_1, t_2) \mid \text{let } x \Leftarrow t_1 \text{ in } t_2 \mid \text{rec } y.(\text{fun}(x, t))$$

dove n sono interi e c caratteri, *empty* indica la lista vuota, *cons*, *head* e *tail* sono le ovvie operazioni sulle liste (costruzione, selezione del primo elemento e del resto di una lista), *pair* è il costruttore delle coppie e avete anche gli ovvi selettori (*fst* e *snd*), e le altre parti sono standard. Il predicato di uguaglianza è polimorfo, quindi lo potete usare in qualsiasi contesto abbia senso usarlo.

Considerate inoltre la seguente grammatica per i tipi

$$\tau ::= a \mid \mathbf{int} \mid \mathbf{bool} \mid \mathbf{char} \mid \tau \text{ list} \mid \tau_1 * \tau_2 \mid \tau_1 \rightarrow \tau_2$$

dove $a \in \mathbf{A}$ sono le variabili di tipo e le seguenti regole descrivono come si fa ad attribuire un tipo ad un termine.

$$\begin{array}{c}
\frac{}{x : \text{type}(x)} \quad \frac{}{n : \text{int}} \quad \frac{}{c : \text{char}} \quad \frac{t_1 : \text{int} \quad t_2 : \text{int}}{t_1 \text{ op } t_2 : \text{int}} \text{ op} \in \{+, -, \times\} \\
\\
\frac{}{\text{true} : \text{bool}} \quad \frac{}{\text{false} : \text{bool}} \quad \frac{t_1 : \text{bool} \quad t_2 : \text{bool}}{t_1 \text{ op } t_2 : \text{bool}} \text{ op} \in \{\wedge, \vee\} \quad \frac{t_1 : \tau \quad t_2 : \tau}{t_1 = t_2 : \text{bool}} \\
\\
\frac{t : \text{bool}}{\neg t : \text{bool}} \quad \frac{t_1 : \text{int} \quad t_2 : \text{int}}{t_1 < t_2 : \text{bool}} \quad \frac{t_1 : \tau_1 \quad t_2 : \tau_2}{\text{pair}(t_1, t_2) : \tau_1 * \tau_2} \\
\\
\frac{t : \tau_1 * \tau_2}{\text{fst}(t) : \tau_1} \quad \frac{t : \tau_1 * \tau_2}{\text{snd}(t) : \tau_2} \quad \frac{}{\text{empty} : \tau \text{ list}} \quad \frac{t_1 : \tau \quad t_2 : \tau \text{ list}}{\text{cons}(t_1, t_2) : \tau \text{ list}} \quad \frac{t : \tau \text{ list}}{\text{head}(t) : \tau} \\
\\
\frac{t : \tau \text{ list}}{\text{tail}(t) : \tau \text{ list}} \quad \frac{t_0 : \text{bool} \quad t_1 : \tau \quad t_2 : \tau}{\text{if } t_0 \text{ then } t_1 \text{ else } t_2 : \tau} \quad \frac{x : \tau_1 \quad t : \tau_2}{\text{fun}(x, t) : \tau_1 \rightarrow \tau_2} \\
\\
\frac{t_1 : \tau_1 \rightarrow \tau_2 \quad t_2 : \tau_1}{\text{apply}(t_1, t_2) : \tau_2} \quad \frac{x : \tau_1 \quad t_1 : \tau_1 \quad t_2 : \tau_2}{\text{let } x \leftarrow t_1 \text{ in } t_2 : \tau_2} \quad \frac{y : \tau \quad \text{fun}(x, t) : \tau}{\text{rec } y.(\text{fun}(x, t)) : \tau}
\end{array}$$

Assumiamo sempre che i termini che dobbiamo valutare siano *ben tipati*, nel senso che riusciamo sempre ad attribuire loro un tipo (che può contenere variabili di tipo).

L'algoritmo di inferenza di tipo usa le regole per l'attribuzione del tipo elencate prima per costruire un insieme di vincoli che poi dovrà essere risolto. I vincoli vengono costruiti nel seguente modo. Dato un termine t e la funzione type che associa ad ogni variabile del linguaggio un tipo τ della grammatica dei tipi, generiamo una coppia formata dal tipo di t e da un insieme di vincoli, quindi riscriviamo le regole in modo più operativo. La funzione $\text{newvar}()$ genera una nuova variabile di tipo.

$$\begin{array}{c}
\frac{}{\langle\langle x, \text{type} \rangle\rangle \Rightarrow (\text{type}(x), \emptyset)} \quad \frac{}{\langle\langle n, \text{type} \rangle\rangle \Rightarrow (\text{int}, \emptyset)} \quad \frac{}{\langle\langle c, \text{type} \rangle\rangle \Rightarrow (\text{char}, \emptyset)} \\
\\
\frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 \text{ op } t_2, \text{type} \rangle\rangle \Rightarrow (\text{int}, \{(\tau_1, \text{int}), (\tau_2, \text{int})\} \cup T_1 \cup T_2)} \text{ op} \in \{+, -, \times\} \quad \frac{}{\langle\langle \text{true}, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \emptyset)} \\
\\
\frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 \text{ op } t_2, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \{(\tau_1, \text{bool}), (\tau_2, \text{bool})\} \cup T_1 \cup T_2)} \text{ op} \in \{\wedge, \vee\} \quad \frac{}{\langle\langle \text{false}, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \emptyset)} \\
\\
\frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 < t_2, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \{(\tau_1, \text{int}), (\tau_2, \text{int})\} \cup T_1 \cup T_2)} \quad \frac{\langle\langle t, \text{type} \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle \neg t, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \{(\tau, \text{bool})\} \cup T)} \\
\\
\frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 = t_2, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \{(\tau_1, \tau_2)\} \cup T_1 \cup T_2)} \quad \frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle \text{pair}(t_1, t_2), \text{type} \rangle\rangle \Rightarrow (\tau, \{(\tau, \tau_1 * \tau_2)\} \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t, \text{type} \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle \text{fst}(t), \text{type} \rangle\rangle \Rightarrow (\tau_1, \{(\tau, \tau_1 * a)\} \cup T)} \quad \frac{a = \text{newvar}() \quad \langle\langle t, \text{type} \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle \text{snd}(t), \text{type} \rangle\rangle \Rightarrow (\tau_2, \{(\tau, a * \tau_2)\} \cup T)} \\
\\
\frac{a = \text{newvar}()}{\langle\langle \text{empty}, \text{type} \rangle\rangle \Rightarrow (a \text{ list}, \emptyset)} \quad \frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle \text{cons}(t_1, t_2), \text{type} \rangle\rangle \Rightarrow (\tau_1 \text{ list}, \{(\tau_2, \tau_1 \text{ list})\} \cup T_1 \cup T_2)} \\
\\
\frac{\langle\langle t, \text{type} \rangle\rangle \Rightarrow (\tau_2, T)}{\langle\langle \text{head}(t), \text{type} \rangle\rangle \Rightarrow (\tau, \{(\tau_2, \tau \text{ list})\} \cup T)} \quad \frac{\langle\langle t, \text{type} \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle \text{tail}(t), \text{type} \rangle\rangle \Rightarrow (\tau, \{(\tau, \tau_1 \text{ list})\} \cup T)}
\end{array}$$

$$\begin{array}{c}
\frac{\langle\langle t_0, type \rangle\rangle \Rightarrow (\tau_0, T_0) \quad \langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, type \rangle\rangle \Rightarrow (\tau, \{(\tau_0, \text{bool}), (\tau, \tau_1), (\tau_1, \tau_2)\} \cup T_0 \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t, type[x/a] \rangle\rangle \Rightarrow (\tau_2, T)}{\langle\langle \text{fun}(x, t), type \rangle\rangle \Rightarrow (a \rightarrow \tau_2, T)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t_1, type \rangle\rangle \Rightarrow (\tau, T_2) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_1, T_1)}{\langle\langle \text{apply}(t_1, t_2), type \rangle\rangle \Rightarrow (a, \{(\tau, \tau_1 \rightarrow a)\} \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type[x/a] \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle \text{let } x \leftarrow t_1 \text{ in } t_2, type \rangle\rangle \Rightarrow (\tau_2, \{(\tau_1, a)\} \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle \text{fun}(x, t), type[y/a] \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle \text{rec } y.(\text{fun}(x, t)), type \rangle\rangle \Rightarrow (\tau, \{(\tau, a)\} \cup T)}
\end{array}$$

Una volta ottenuto l'insieme di vincoli, questi vanno *risolti* tramite un algoritmo che si basa sulla *sostituzione* e sull'*unificazione*. Una sostituzione è una funzione che associa ad ogni variabile di tipo un tipo (con eventuali variabili). Ovviamente l'identità è una sostituzione (ogni variabile di tipo viene *mandata* in se stessa) e le sostituzioni possono venire *composte*. Dato un vincolo, che è una coppia, si cerca la sostituzione che rende i due elementi della coppia uguali. Quindi, dato un insieme C di coppie, per trovare la sostituzione corretta si procede come segue, partendo dalla sostituzione *identica*. La sostituzione la chiamiamo s .

- se la coppia è fatta da due termini uguali, la si elimina
- se la coppia è fatta da una variabile a ed un termine q in cui tale variabile non occorre allora si aggiorna la sostituzione s ponendo che $s(a) = q$ e si applica tale sostituzione a tutte le coppie in C
- se la coppia è fatta da due termini composti da costruttori (nel nostro caso coppie, liste e frecce) allora si verifica che i costruttori più esterni siano uguali, e se lo sono si aggiungono le coppie fatte dai componenti accoppiati secondo le regole dell'equivalenza e si elimina la coppia di partenza
- si ripetono i passi fino a che non si ottiene che C è vuoto (ed allora abbiamo la sostituzione cercata) oppure se si incontra una coppia che non è unificabile

Per chiarire, alcuni esempi: $\{(int, int)\}$ unifica con la sostituzione identica, $\{(a, int)\}$ unifica con la sostituzione $s(a) = int$, $\{(int, a \rightarrow b)\}$ non unifica mentre $\{(a \rightarrow int, bool \rightarrow b)\}$ unifica con la sostituzione $s(a) = bool$ e $s(b) = int$. Osservate che da $\{(a \rightarrow int, bool \rightarrow b)\}$ si sono generati $\{(a, bool), (int, b)\}$.

5.4 Astrarre sui dati

Come nel caso dell'astrazione sul controllo o funzionale, si può realizzare anche l'astrazione sui dati. Le ragioni sono simili. La prima è per nascondere dettagli non necessari, e quindi focalizzarsi su quelli che interessano davvero, e la seconda è l'aumento della portabilità.

Come si realizza l'astrazione sui dati? Abbiamo i seguenti concetti:

- *componente*: unità di programma (ad esempio funzione, struttura dati, modulo)
- *interfaccia*: tipi e operazioni definiti in un componente che sono visibili fuori del componente stesso

- *specifica*: è il funzionamento *inteso* del componente, espresso mediante proprietà osservabili attraverso l'interfaccia
- *implementazione*: strutture dati e funzioni definiti dentro al componente, non necessariamente visibili da fuori

L'idea è di formalizzare il comportamento inteso senza parlare della sua implementazione, e poi realizzare un'implementazione che però non è visibile all'utente (*information hiding*).

Vediamo un esempio. Per la pila devono valere le seguenti proprietà (ci sono tre operazioni ed un predicato, le solite, la pila vuota la indichiamo con `Empty`):

1. `pop(push(e,p)) = p`
2. `top(push(e,p)) = e`
3. `push(top(p),pop(p)) = p`
4. `isempty(Empty) = true` e `isempty(push(e,p)) = false`

Ogni implementazione deve essere tale che la specifica sia soddisfatta.

Un tipo di dato è in generale *fatto* da un insieme di valori base e di operazioni, nell'astrazione nascondiamo come rappresentiamo gli elementi del tipo (ad esempio come realizziamo davvero la pila), implementiamo le operazioni in modo invisibile all'utente e la specifica dice come vengono manipolati i dati. L'implementazione è *inaccessibile* all'utente, perché protetta da una capsula che la isola.

Se l'astrazione sul controllo significa *nascondere* la realizzazione dei corpi di procedure/funzioni, astrarre sui dati significa nascondere decisioni sulla rappresentazione delle strutture dati e nascondere l'implementazione delle operazioni.

Il supporto linguistico fornito da un linguaggio per l'*information hiding* è la nozione di *Abstract Data Type* (ADT) che è tra i maggiori contributi ai linguaggi di programmazione degli anni '70. L'idea è semplice: separare l'interfaccia (quali operazioni e simili) dall'implementazione. **Sets** è un tipo di dato in cui ho operazioni come `empty`, `insert`, `union`, `ismember`. La specifica di queste operazioni è chiara, e gli insiemi li posso implementare come voglio (liste, vettori, ecc.). Si usa il controllo di tipo per garantire la separazione: il cliente ha accesso alle sole operazioni dell'interfaccia, mentre l'implementazione vera e propria è *incapsulata* in opportuni costrutti opachi.

In un ADT ci sono in generale:

- *costruttori*: operazioni per costruire gli elementi del tipo (usando eventualmente dati di altri tipi): `creapila`, che crea una pila vuota,
- *trasformatori* (o operatori): operazioni che calcolano valori di quel tipo ed eventualmente lo restituiscono: `push` oppure `pop`,
- *osservatori*: operazioni che restituiscono valori di altri tipi basati su proprietà dell'elemento: `top`.

Con gli ADT c'è un importante principio: indipendenza dalla rappresentazione. Infatti *due implementazioni corrette di un tipo (astratto) non sono distinguibili dai clienti di quel tipo*, quindi le implementazioni sono modificabili senza con ciò interferire con alcun cliente, e il cliente non ha alcun modo per accedere all'implementazione

Il *modulo* è il costrutto generale per l'*information hiding*, ed è disponibile sia in linguaggi imperativi e funzionali che, ovviamente, in quelli object oriented. Il modulo ha due parti:

- *interfaccia*: un insieme di nomi e relativi tipi
- *implementazione*: dichiarazioni (di tipi e funzioni) per ogni nome dell'interfaccia ed eventuali dichiarazioni aggiuntive (nascoste al cliente)

I costrutti sono, per esempio, i moduli di **Modula**, packages di **Ada**, strutture di **ML**.

Gli ADT hanno un importante limite: sono *rigidi* (difficilmente estendibili), dato che le aggiunte modificano l'ADT senza alcuna relazione codificate con l'ADT di partenza.

Invece c'è bisogno di costrutti che permettano la definizione di nuovi tipi e operazioni in modo astratto, che rispettino i principi d'incapsulamento (dell'information hiding). Quindi di costrutti per

- information hiding e incapsulamento
- riuso del codice (ereditarietà)
- compatibilità tra tipi (sottotipi)
- selezione dinamica delle operazioni

La soluzione proposta è quella di *oggetto*. Un oggetto è una *capsula* che contiene

- dati nascosti: variabili, valori (talvolta anche operazioni)
- operazioni pubbliche: operazioni, dette metodi

In un programma orientato agli oggetti si mandano *messaggi* agli oggetti (il messaggio è l'invocazione del metodo di quell'oggetto), e l'oggetto risponde ai messaggi (lo stato è confinato negli oggetti). Principi di organizzazione permettono di raggruppare gli oggetti con la stessa struttura (classi) e questi stessi principi consentono estensibilità e riuso. Va sottolineato che l'*astrazione* sui dati e sul controllo, l'*information hiding* e l'*incapsulamento* sono presenti nel progetto del linguaggio di programmazione sin dall'inizio.

Definizione 5.10 *Una classe è un modello di un insieme di oggetti, quali dati, nome, segnatura e visibilità delle operazioni (metodi).*

Gli oggetti sono creati dinamicamente per istanziazione di una classe, mediante costruttori

```
class Counter {
    private int x;
    public void reset () {
        x = 0
    }
    public int get () {
        return x;
    }
    public void inc () {
        x = x + 1;
    }
}
```

Le classi garantiscono l'incapsulamento (opportuni modificatori assicurano che determinati campi siano *pubblici*, *privati* o *protetti*), la parte pubblica è l'*interfaccia* della classe, mentre la parte privata è l'implementazione. Altri meccanismi assicurano che l'incapsulamento sia compatibile con l'estensibilità e la modificabilità del codice e con i modi per il riuso (**extending**) e ridefinizione (**overriding**) di metodi.

Ad esempio, **NewCounter** estende l'interfaccia di **Counter** con nuovi campi e ridefinisce il metodo **reset** (e il messaggio **reset** inviato ad un'istanza di **NewCounter** causa l'invocazione del nuovo codice...):

```
class NamedCounter extending Counter {
    private string Name;
    public SetName (String n) {
        Name = n;
    }
}
```

Con la keyword **extending** si creano nuovi tipi (sottotipi di uno più generale). Un tipo può avere più di un sovratipo immediato (ad esempio, in **Java** quando i sovratipi sono classi totalmente astratte, senza implementazioni, che nel gergo **Java** sono chiamate implementazioni), per cui la gerarchia dei sottotipi non è un albero. Il *sottotipo* fornisce la possibilità di usare un oggetto in un altro contesto (sostanzialmente è una relazione tra le *interfacce* di due classi), mentre l'*ereditarietà* dà la possibilità di riusare il codice che manipola un oggetto (quindi è una relazione tra le *implementazioni* di due classi).

L'ereditarietà permette il riuso del codice in un contesto estendibile (ogni modifica all'implementazione di un metodo in una classe è automaticamente disponibile in tutte le sottoclassi): **NewCounter** eredita da **Counter** i metodi (e i campi) non ridefiniti.

Le nozioni di sottotipo ed ereditarietà forniscono due meccanismi che sono logicamente del tutto indipendenti ma (spesso) linguisticamente collegati nei linguaggi object oriented. Ad esempio, sia **C++** che **Java** hanno costrutti che introducono contemporaneamente entrambe le relazioni tra due classi: **extends** in **Java** introduce un sottotipo e può introdurre ereditarietà se ci sono metodi della superclasse che non sono ridefiniti nella sottoclasse. In **Java** l'ereditarietà è singola: ogni classe eredita al più da una sola superclasse immediata, mentre in **C++** l'ereditarietà può essere multipla: una classe può ereditare da più di una superclasse immediata.

Capitolo 6

Lexer and parser

In questo capitolo vedremo parte della teoria che consente di costruire i *lexer* ed i *parser*.

Un *lexer* è un programma che riceve in input una parola (il programma da analizzare) e restituisce una sequenza di *tokens*, sequenza che poi verrà analizzata dal *Parser*, che a sua volta restituisce un albero di *parsing* che, opportunamente annotato, costituisce la base per costruire l'albero di sintassi astratta che noi effettivamente *interpreteremo*, cioè eseguiremo. Consideriamo la parola “**if** $x > 0$ **then** 0 **else** 1”, il lexer la suddividerà nella sequenza di token `[if;  $x > 0$ ; then; 0; else; 1]` (che è stata scritta come una lista in OCaml) che poi potrà essere analizzata dal parser per verificare che sia una frase corretta secondo la grammatica di un qualche linguaggio di programmazione.

La parola “**if** $x > 0$ **then** 0 **else** 1”, per poter essere *scomposta* in token, deve comunque essere riconosciuta come parola appartenente ad un linguaggio, cioè essere una stringa valida. Per chiarire meglio questo punto, vale la pena di osservare che ogni *token* in cui la parola deve essere scomposta appartiene ad un linguaggio (in generale finito). Per questo la parola da riconoscere è spesso la concatenazione di parole di vari linguaggi.

Per fare questi due passi (lexing e parsing) dobbiamo prima di tutto dire cosa è un linguaggio *formale*, poi capire come generiamo un linguaggio, o lo denotiamo, e come riconosciamo che una parola appartiene ad un linguaggio. Sarà anche utile dare proprietà di chiusura delle classi di linguaggi che vedremo.

Vedremo prima la classe dei linguaggi *regolari* per poi concentrarci sui linguaggi *context-free*. Sui linguaggi regolari è basato il lexer, mentre sui linguaggi context-free il parser.

6.1 Linguaggi

Si consideri un insieme finito A , che chiamiamo *alfabeto*. Con A^* indichiamo tutte le parole di lunghezza finita sull'alfabeto A . Le parole sono una *sequenza*, eventualmente vuota, di simboli dell'alfabeto. Sia $A = \{a, b\}$, allora a è una parola, così come *abbab*. Con ϵ indichiamo la parola vuota, quella cioè in cui non c'è alcun simbolo dell'alfabeto. Date due parole $w, w' \in A^*$, con $w \cdot w'$ indichiamo la *concatenazione* delle due parole, che significa che scriviamo la parola w' alla destra della parola w , formando così una nuova parola. Con $|w|$ indichiamo la lunghezza di una parola, che è definita nel seguente modo: $|\epsilon| = 0$ e $|a \cdot w| = |w| + 1$, dove a è la parola di una singola lettera $a \in A$. Sia data una parola $w \in A^*$, un *prefisso* di w è una parola $w' \in A^*$ tale che esiste una parola $w'' \in A^*$ e $w = w' \cdot w''$. Un *uffisso* di una parola $w \in A^*$ è una parola $w'' \in A^*$ tale che esiste una parola $w' \in A^*$ e $w = w' \cdot w''$. Un *infisso* di una parola $w \in A^*$ è una parola $\bar{w} \in A^*$ tale che esistono due parole $w', w'' \in A^*$ e $w = w' \cdot \bar{w} \cdot w''$.

Possiamo ora formalizzare cosa è un linguaggio su un alfabeto A .

Definizione 6.1 Sia A un alfabeto. Un linguaggio $\mathcal{L}$ su A è un sottoinsieme di A^* .

Consideriamo $A = \{a, b\}$, allora un linguaggio può essere l'insieme delle parole di lunghezza pari in cui compaiono il doppio di a rispetto alle b . Alcune parole di questo linguaggio sono $aaaabb$, $abbaaaa$, $baabaa$ e così via. È bene ricordare che un linguaggio può contenere infinite parole, ma tutte le parole sono di lunghezza finita. In generale un linguaggio sarà un insieme di parole che soddisfano una certa proprietà, ad esempio, se scriviamo $\mathcal{L} = \{w \in A^* \mid \exists k \in \mathbb{N}. |w| = 2k + 1\}$ indichiamo il linguaggio di tutte le parole che hanno lunghezza dispari. Se è chiaro dal contesto spesso indichiamo una proprietà con la lettera P , quindi scriviamo $\mathcal{L} = \{w \in A^* \mid P(w)\}$.

Sui linguaggi sono definite varie operazioni, alcune insiemistiche, altre che riguardano le parole. Le più importanti sono l'unione, l'intersezione, il complemento, la differenza (per quanto riguarda le operazioni insiemistiche), la concatenazione e lo *star di Kleene* (per quelle che sono operazioni sulle parole). Ne diamo una breve descrizione facendo riferimento alle varie proprietà. Assumiamo che $\mathcal{L} = \{w \in A^* \mid P_i(w)\}$. Abbiamo che

$$\begin{aligned}\mathcal{L}_1 \cup \mathcal{L}_2 &= \{w \in A^* \mid P_1(w) \text{ or } P_2(w)\} \\ \mathcal{L}_1 \cap \mathcal{L}_2 &= \{w \in A^* \mid P_1(w) \text{ and } P_2(w)\} \\ \overline{\mathcal{L}} &= \{w \in A^* \mid \neg P(w)\} \\ \mathcal{L}_1 \setminus \mathcal{L}_2 &= \{w \in A^* \mid P_1(w) \text{ and } \neg P_2(w)\} \\ \mathcal{L}_1 \cdot \mathcal{L}_2 &= \{w \in A^* \mid \exists w_1, w_2 \in A^*. w = w_1 \cdot w_2, w_1 \in \mathcal{L}_1 \text{ and } w_2 \in \mathcal{L}_2\}\end{aligned}$$

infine, per lo star di Kleene, prima poniamo $\mathcal{L}^0 = \{\epsilon\}$ ed $\mathcal{L}_{i+1} = \mathcal{L}^i \cdot \mathcal{L}$, poi definiamo $\mathcal{L}^*$ come l'unione di $\mathcal{L}^i$, quindi $\mathcal{L}^* = \bigcup_{i=0}^{\infty} \mathcal{L}^i$. Con $\mathcal{L}^+$ indichiamo $\mathcal{L} \cdot \mathcal{L}^*$. Osserviamo che il linguaggio $\mathcal{L}^+$ contiene la parola vuota solo se $\mathcal{L}$ la contiene.

6.2 Linguaggi regolari

Definiamo un modo per denotare i linguaggi regolari, che sono le *espressioni regolari* ed un modo per riconoscere che una parola appartiene ad un linguaggio regolare, che saranno gli *automi a stati finiti*.

6.2.1 Espressioni regolari

Consideriamo un alfabeto A . Ad ogni simbolo dell'alfabeto $a \in A$ associamo $\underline{a}$.

Definizione 6.2 Sia A un alfabeto. Una espressione regolare (e.r.) su A è definita induttivamente come segue:

- $\emptyset$, ϵ e $\underline{a}$ sono espressioni regolari, dove $a \in A$;
- se r_1 ed r_2 sono espressioni regolari allora anche $r_1 + r_2$ ed $r_1 \cdot r_2$ sono espressioni regolari;
- se r è una espressione regolare anche r^* e (r) sono espressioni regolari; e
- niente altro è una espressione regolare.

Avremmo potuto scrivere anche che una e.r. è generata dalla seguente grammatica BNF (che per ora non abbiamo formalizzato, ma che formalizzeremo in seguito):

$$r ::= \emptyset \mid \epsilon \mid \underline{a} \mid r + r \mid r \cdot r \mid (r)$$

Se consideriamo $A = \{0, 1\}$, una espressione regolare è, ad esempio, la seguente $(\underline{0} \cdot \underline{1} + \underline{0})^* \cdot \underline{1}$. Spesso scriveremo $(\underline{01} + \underline{0})^* \cdot \underline{1}$ omettendo, quando è chiaro dal contesto, il ‘ $\cdot$ ’, cioè la concatenazione, e ometteremo anche la sottolineatura, a meno che non si creino ambiguità.

Le espressioni regolari *denotano* un linguaggio. Consideriamo, ad esempio, l’espressione regolare $(0 + 1)^*00$. Il linguaggio denotato da questa espressione regolare è il linguaggio di tutte le parole che terminano con due 0. Per capire questo dobbiamo associare ad ogni espressione regolare un linguaggio.

Definizione 6.3 *Il linguaggio $\mathcal{L}$ associato ad una espressione regolare r è definito induttivamente come segue:*

1. se $r = \emptyset$ allora $\mathcal{L}(r) = \emptyset$;
2. se $r = \epsilon$ allora $\mathcal{L}(r) = \{\epsilon\}$;
3. se $r = \underline{a}$ allora $\mathcal{L}(r) = \{a\}$;
4. se $r = r_1 + r_2$ allora $\mathcal{L}(r) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$;
5. se $r = r_1 \cdot r_2$ allora $\mathcal{L}(r) = \mathcal{L}(r_1) \cdot \mathcal{L}(r_2)$;
6. se $r = r_1^*$ allora $\mathcal{L}(r) = \mathcal{L}(r_1)^*$; e
7. se $r = (r_1)$ allora $\mathcal{L}(r) = \mathcal{L}(r_1)$.

Osserviamo che il linguaggio di tutte le parole su un certo alfabeto A è dato dall’espressione regolare $(\sum_{a \in A} \underline{a})^*$, dove $\sum$ denota la somma. Consideriamo $A = \{0, 1\}$. Allora abbiamo che $\mathcal{L}((\underline{0} + \underline{1})^*) = (\mathcal{L}(\underline{0} + \underline{1}))^* = (\mathcal{L}(\underline{0}) + \mathcal{L}(\underline{1}))^* = (\{0\} \cup \{1\})^* = \{0, 1\}^*$.

Definizione 6.4 *Un linguaggio $\mathcal{L}$ è detto regolare se esiste una espressione regolare r tale che $\mathcal{L}(r) = \mathcal{L}$.*

La classe dei linguaggi regolari è quindi quella per cui la proprietà del linguaggio può essere in qualche modo espressa con una espressione regolare. Non esiste un modo unico per denotare un linguaggio regolare con una espressione regolare, abbiamo quindi bisogno di dire quando due espressioni regolari sono *equivalenti*.

Definizione 6.5 *Due espressioni regolari r ed r' sono equivalenti, scritto come $r \equiv r'$, se e solo se $\mathcal{L}(r) = \mathcal{L}(r')$.*

Ad esempio, si vede chiaramente che $\underline{a} + \underline{a} \equiv \underline{a}$. Tra le equivalenze note abbiamo:

$$\begin{array}{llll}
 r_1 + r_2 & \equiv & r_2 + r_1 & r_1 + (r_2 + r_3) \equiv (r_1 + r_2) + r_3 \\
 r_1 \cdot (r_2 \cdot r_3) & \equiv & (r_1 \cdot r_2) \cdot r_3 & r_1 \cdot (r_2 + r_3) \equiv r_1 \cdot r_2 + r_1 \cdot r_3 \\
 r + \emptyset & \equiv & r & r \cdot \epsilon \equiv r \\
 \epsilon \cdot r & \equiv & r & r \cdot \emptyset \equiv \emptyset \\
 \emptyset \cdot r & \equiv & \emptyset & (r^*)^* \equiv r^*
 \end{array}$$

Osserviamo anche che se $\mathcal{L}(r) \subseteq \mathcal{L}(r')$ allora $r + r' \equiv r'$.

Le espressioni regolari vengono usate anche per stabilire se una certa parola soddisfa un certo pattern, ad esempio, se si vuole verificare che la parola w contiene la sequenza 001 possiamo vedere se $w \in \mathcal{L}((0+1)^*001(0+1)^*)$, dato che il linguaggio denotato dall’espressione regolare $(0+1)^*001(0+1)^*$ è fatto da tutte le parole su $\{0, 1\}$ dove appare il pattern 001.

Possiamo infine osservare che un linguaggio *finito* è certamente regolare, dato che ogni sua parola può essere descritta da un’espressione regolare e l’unione di tutte queste caratterizza precisamente il linguaggio. Un linguaggio finito è, ad esempio, quello di tutte le keywords di un linguaggio di programmazione.

6.2.2 Automi a stati finiti

Fino ad ora abbiamo visto come denotare un linguaggio regole e come descriverlo con le espressioni regolari, ma non abbiamo considerato come riconoscere che una parola appartiene al linguaggio. Per questo introduciamo la nozione di *automa a stati finiti* che abbrevieremo spesso con FA (da *Finite Automaton*), premettendo eventualmente o la D di deterministico o la N di non-deterministico.

Definizione 6.6 Un automa a stati finiti non deterministico (NFA) è la quintupla $\mathcal{A} = (Q, A, \delta, q_0, F)$ dove

- Q è un insieme finito di stati;
- A è un alfabeto;
- $\delta : Q \times A \rightarrow 2^Q$ è la funzione di transizione;
- $q_0 \in Q$ è lo stato iniziale; e
- $F \subseteq Q$ è un insieme di stati finali.

Un automa ha anche un nastro di input dove viene scritta la parola che deve essere accettata. Se un automa si trova in uno stato q e la testina del nastro d'input legge il simbolo a appartenente all'alfabeto, allora l'automa va nello stato q' e la testina si sposta sul simbolo successivo, dove $q' \in \delta(q, a)$. L'automa accetta la parola se la legge tutta e termina in uno stato finale. Formalmente estendiamo la funzione δ in modo che il suo dominio sia non $Q \times A$ ma $Q \times A^*$, chiamandola $\hat{\delta}$ in questo modo: $\hat{\delta}(q, \epsilon) = \{q\}$ e $\hat{\delta}(q, aw) = \bigcup_{q' \in \delta(q, a)} \hat{\delta}(q', w)$.

Definizione 6.7 Sia $\mathcal{A} = (Q, A, \delta, q_0, F)$ un NFA. Il linguaggio accettato da $\mathcal{A}$, che chiamiamo $\mathcal{L}(\mathcal{A})$, è l'insieme $\{w \in A^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$.

Un FA $\mathcal{A} = (Q, A, \delta, q_0, F)$ può anche essere *deterministico*, quando $|\delta(q, a)| = 1$ per ogni $q \in Q$ ed ogni $a \in A$. La funzione $\hat{\delta}$ cambia leggermente dato che ora deve restituire uno stato, quindi $\hat{\delta}(q, \epsilon) = q$ e $\hat{\delta}(q, aw) = \hat{\delta}(\delta(q, a), w)$. Infine abbiamo che il linguaggio accettato è l'insieme $\{w \in A^* \mid \hat{\delta}(q_0, w) \in F\}$.

Il risultato importante è che ogni NFA può essere trasformato in un DFA che accetta lo stesso linguaggio.

Teorema 6.1 Dato un automa NFA $\mathcal{A}$, esiste un DFA $\mathcal{A}'$ effettivamente costruibile da $\mathcal{A}$ tale che $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$.

Osservando che un DFA può essere trasformato in un NFA semplicemente $\delta(q, a) = \{q'\}$, abbiamo che il non-determinismo non aggiunge potere espressivo. La prova di questo teorema è semplice. Partiamo dall'NFA $\mathcal{A} = (Q, A, \delta, q_0, F)$, costruiamo un DFA $\mathcal{D}(\mathcal{A}) = (2^Q, A, \delta', \{q_0\}, \hat{F})$ dove $\delta'(X, a) = \bigcup_{q \in X} \delta(q, a)$ e $\hat{F} = \{X \subseteq Q \mid X \cap F \neq \emptyset\}$. Questo automa è certamente deterministico ed è facile vedere che, per ogni parola $w \in A^*$, se $\hat{\delta}(q_0, w) \cap F \neq \emptyset$ allora $\delta'^*(\{q_0\}, w) \in \hat{F}$.

Dato un DFA $\mathcal{A}$, ne esiste uno che accetta il medesimo linguaggio e che ha almeno meno stati di $\mathcal{A}$. Questo implica che se la misura è il numero degli stati, allora ne esiste uno minimo.

Infine citiamo i risultati che collegano FA ed espressioni regolari.

Teorema 6.2 Sia r una espressione regolare e $\mathcal{L}(r)$ il linguaggio ad essa associato. Allora esiste un DFA $\mathcal{A}$ tale che $\mathcal{L}(\mathcal{A}) = \mathcal{L}(r)$.

Teorema 6.3 Sia $\mathcal{A}$ un DFA e il linguaggio ad esso associato. Allora esiste una espressione regolare r tale che $\mathcal{L}(\mathcal{A}) = \mathcal{L}(r)$.

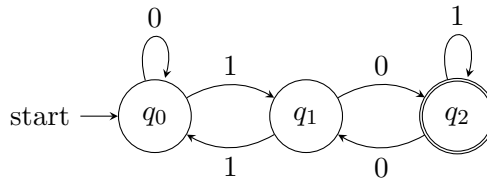
Con l'aiuto di questi due risultati possiamo caratterizzare la classe dei linguaggi regolari.

Definizione 6.8 Sia dato un alfabeto A . La classe dei linguaggi regolari su A , che indichiamo con $\mathcal{L}_{reg}^A$, è l'insieme $\mathcal{L}_{reg}^A = \{\mathcal{L} \subseteq A^* \mid \mathcal{L} \text{ è regolare}\}$.

Chiaramente abbiamo che $\mathcal{L}_{reg}^A = \{\mathcal{L} \subseteq A^* \mid \mathcal{L} \text{ è accettato da un DFA}\}$ è uguale a $\{\mathcal{L} \subseteq A^* \mid \mathcal{L} \text{ è denotato da una espressione regolare}\}$.

Rappresentazione grafica di un FA

Un automa a stati finiti viene rappresentato graficamente nel seguente modo:



Gli archi dallo stato q allo stato q' hanno come annotazione a se $\delta(q, a)$ contiene q' . Lo stato iniziale è indicato con la freccia entrante e gli stati accettanti sono quelli indicati con il doppio bordo. È facile vedere che il linguaggio accettato da questo automa è quello di tutte le parose su $\{0, 1\}$ che, interpretate come un numero naturale e divise per 3 danno resto 2.

6.2.3 Proprietà di chiusura dei linguaggi regolari

Dato che i linguaggi regolari sono denotabili come espressioni regolari è abbastanza facile vedere che sono chiusi per unione, concatenazione e star di Kleene.

Sui linguaggi sono definibili altre proprietà di chiusura, come la sostituzione e l'omomorfismo. Una *sostituzione* è una funzione che associa ad ogni elemento di un alfabeto un linguaggio (possibilmente su un altro alfabeto). Dato un alfabeto A e dato un insieme di linguaggi (non necessariamente tutti sullo stesso alfabeto), una sostituzione è una funzione $s : A \rightarrow \{\mathcal{L} \mid \mathcal{L} \text{ è un linguaggio}\}$. Con $s(a)$ indichiamo il linguaggio $\mathcal{L}_a$ associato ad a . Se consideriamo una parola $w \in A^*$, $s(w)$ è il linguaggio ottenuto nel seguente modo: $w = a_1 a_2 \dots a_n$, allora $s(w) = s(a_1) s(a_2) \dots s(a_n)$, cioè la concatenazione di tutti i linguaggi associati ad ogni terminale. Dato $\mathcal{L} \subseteq A^*$, $s(\mathcal{L}) = \bigcup_{w \in \mathcal{L}} s(w)$.

Dato un linguaggio regolare $\mathcal{L}$ ed una sostituzione $s : A \rightarrow 2^{B^*}$ tale che $\forall a \in A. s(a)$ è regolare, abbiamo che $s(\mathcal{L})$ è regolare, e questo è facile vederlo usando le espressioni regolari dove ad ogni a associamo l'espressione regolare che denota il linguaggio $s(a)$. Dato un linguaggio regolare $\mathcal{L}$ sull'alfabeto A , ed un omomorfismo $h : A \rightarrow B^*$, è facile vedere che anche $h(\mathcal{L}) = \{w \in B^* \mid \exists x \in \mathcal{L}. h(x) = w\}$ è regolare dato che un omomorfismo è banalmente una sostituzione.

Per vedere altre proprietà di chiusura è più semplice ricorrere agli automi. Dato un linguaggio regolare $\mathcal{L}$ e l'automa che lo accetta $\mathcal{A} = (Q, A, \delta, q_0, F)$ è facile vedere che l'automa $(Q, A, \delta, q_0, Q \setminus F)$ accetta il complementare di $\mathcal{L}$ mentre dati due automi, che per semplicità assumiamo deterministici, $\mathcal{A} = (Q, A, \delta, q_0, F)$ e $\mathcal{A}' = (Q', A, \delta', q'_0, F')$ è facile vedere che l'automa $(Q \times Q', A, \delta'', (q_0, q'_0), F \times F')$ con $\delta''((q, q'), a) = (\delta(q, a), \delta'(q', a))$ accetta l'intersezione.

Dato un omomorfismo $h : A \rightarrow B^*$, con $h^{-1}(\mathcal{L})$ indichiamo il linguaggio $\{w \in A^* \mid h(w) \in \mathcal{L}\}$. Se $\mathcal{L}$ è regolare e $h : A \rightarrow B^*$ è un omomorfismo, è facile vedere che $h^{-1}(\mathcal{L})$ è un linguaggio regolare,

quindi i linguaggi regolari sono chiusi anche per omomorfismo inverso. L'intuizione dietro la prova è ancora basata su automi. Supponiamo di avere un automa per $\mathcal{L}$, costruiamo un automa che per ogni simbolo di A simula $h(a)$ usando l'automa per $\mathcal{L}$.

6.2.4 Pumping Lemma per linguaggi regolari

Il prossimo risultato è particolarmente importante perché ci aiuta a capire che ci esistono linguaggi che non sono regolari.

Lemma 6.1 *Sia $\mathcal{L}$ un linguaggio regolare su A , allora esiste una costante $k \in \mathbb{N}$ propria del linguaggio tale che, per ogni parola $w \in \mathcal{L}$ con $|w| \geq k$, esistono $x, y, z \in A^*$ tali che $w = x \cdot y \cdot z$, $|x \cdot y| \leq k$, $y \neq \epsilon$, e $\forall i \in \mathbb{N}. x \cdot y^i \cdot z \in \mathcal{L}$.*

Questo viene chiamato *pumping lemma* perché possiamo pompare la string y quante volte vogliamo.

Il pumping lemma lo possiamo usare per capire che un linguaggio non è regolare. Consideriamo il linguaggio $\mathcal{L} = \{a^n b^n \mid n \in \mathbb{N}\}$. Mostriamo che non è regolare. Se lo fosse, esisterebbe la costante propria del linguaggio, e supponiamo che questa sia N . Consideriamo la parola $a^N b^N$ che certamente sta in $\mathcal{L}$ ed è più lunga di N . Questa la possiamo suddividere in x, y e z con la concatenazione di queste tre parole che dà $a^N b^N$. È facile ora vedere che necessariamente deve essere che $x = a^i$, $y = a^j$ e $z = a^l b^N$ con $i + j + l = N$ e $j \neq 0$. Ma, se $\mathcal{L}$ fosse regolare, anche $a^i a^l b^N$ appartenerrebbe al linguaggio che è falso dato che $i + l < N$, visto che $j \neq 0$.

6.3 Linguaggi context-free

Analogamente a quanto abbiamo visto per i linguaggi regolari, vediamo prima un modo per generare le parole di un linguaggio context-free e poi come riconoscere tali parole.

6.3.1 Grammatiche

Diamone prima una definizione informale: una grammatica è sostanzialmente un insieme di *regole* che descrivono o prescrivono la formazione delle parole di un linguaggio, che, dal punto di vista della programmazione, sono le frasi sintatticamente corrette di un linguaggio di programmazione. Quindi, per noi, una grammatica non è altro che un insieme di regole per *generare* le parole del linguaggio.

Prima di tutto formalizziamo la nozione di *produzione*.

Definizione 6.9 *Sia A un alfabeto. Una produzione su A è una coppia (α, β) tale che $\alpha, \beta \in A^*$, e $\alpha \neq \epsilon$.*

L'idea che sta dietro a questa nozione è banale. Devo *riscrivere* il lato sinistro della produzione (α) con in lato destro (β), all'interno di una parola dove compare, come *infisso*, il lato sinistro della produzione. La seconda condizione impone che non si possa riscrivere la parola vuota, come sarà chiaro nel seguito. La produzione (α, β) viene normalmente scritta come $\alpha \rightarrow \beta$.

Osserviamo che le produzioni su A non sono altro che una relazione su $A^* \times A^*$. Quelle che considereremo saranno in più tali che $|\alpha| = 1$, cioè si riscrive un solo simbolo.

Definizione 6.10 *Una grammatica context-free (CFG) su A è la quadrupla $G = (\mathbf{V}, A, \mathcal{P}, S)$ dove*

1. $\mathbf{V}$ è un insieme finito di simboli non terminali, e A è un insieme finito di simboli terminali tali che $\mathbf{V} \cap A = \emptyset$,

2. $\mathcal{P}$ è un insieme di produzioni su $\mathbf{V} \cup A$ tali che il primo elemento della coppia appartiene sempre a $\mathbf{V}$, quindi $\mathcal{P} \subseteq \mathbf{V} \times (\mathbf{V} \cup A)^*$, e
3. $S \in V$ è il simbolo iniziale.

Prima di formalizzare cosa è una derivazione in una grammatica, diamone un esempio. Consideriamo la CFG $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S0, S \rightarrow 1S1, S \rightarrow \epsilon\}, S)$, come la usiamo? l'idea è di *derivare* parole su A^* applicando le regole definite dalle produzioni, partendo dal simbolo iniziale S . Supponiamo di voler derivare la parola 011110. Dato che le produzioni delle grammatiche che stiamo considerando riscrivono solo simboli non terminali, dobbiamo trovare all'interno della parola che stiamo riscrivendo il simbolo non terminale che è il lato sinistro di una qualche produzione. Quindi da S deriviamo $0S0$ usando la produzione $S \rightarrow 0S0$, poi, da questa, deriviamo $01S10$ usando la produzione $S \rightarrow 1S1$, poi deriviamo $011S110$ usando la medesima produzione $S \rightarrow 1S1$ ed infine deriviamo 011110 usando la produzione $S \rightarrow \epsilon$.

Data una CFG $G = (\mathbf{V}, A, \mathcal{P}, S)$, un passo di derivazione è definito come segue:

Definizione 6.11 Sia $G = (\mathbf{V}, A, \mathcal{P}, S)$ una CFG e sia α una parola diversa da ϵ ($\alpha \in (\mathbf{V} \cup A)^+$). Un passo di derivazione $\alpha \Rightarrow \gamma$ usando una produzione $V_i \rightarrow \alpha_i$ è definito come segue:

1. esistono $\beta, \delta \in (\mathbf{V} \cup A)^*$ tali che $\alpha = \beta V_i \delta$, e
2. $\gamma = \beta \alpha_i \delta$.

Una derivazione non è altro che la chiusura riflessiva e transitiva della relazione $\Rightarrow$:

Definizione 6.12 Sia $G = (\mathbf{V}, A, \mathcal{P}, S)$ una CFG e $\alpha \in (\mathbf{V} \cup A)^+$. Deriviamo da $\alpha \in (\mathbf{V} \cup A)^+$ la parola $\beta \in (\mathbf{V} \cup A)^*$ se e solo se $\alpha \Rightarrow^* \beta$.

Per indicare che deriviamo β da α scriveremo $\alpha \xRightarrow{*} \beta$. Chiaramente questa definizione implica che esiste una sequenza finita di passi che consentono di arrivare da α a β con passi di derivazione successivi.

Vediamo un altro esempio. Data la CFG $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \epsilon\}, S)$, deriviamo a partire da S , la parola 000111. Abbiamo $S \Rightarrow 0S1$, $0S1 \Rightarrow 00S11$, $00S11 \Rightarrow 000S111$, sempre usando la produzione $S \rightarrow 0S1$, ed infine $000S111 \Rightarrow 000111$, usando $S \rightarrow \epsilon$. Osservando che abbiamo derivato la parola 0^31^3 , possiamo immaginare che il linguaggio generato da questa grammatica sia il linguaggio di tutte le parole che contengono tanti 0 quanti 1 e tutti gli 0 stanno a sinistra degli 1.

Possiamo ora formalizzare quale è il linguaggio generato da una CFG G .

Definizione 6.13 Sia $G = (\mathbf{V}, A, \mathcal{P}, S)$ una CFG. Il linguaggio generato da G è l'insieme delle parole fatte da soli simboli terminali derivabili da S . Quindi $\mathcal{L}(G) = \{w \in A^* \mid S \xRightarrow{*} w\}$.

Sia $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \epsilon\}, S)$ la grammatica. Si può facilmente vedere che $\mathcal{L}(G) = \{0^i1^i \mid i \geq 0\}$.

Abbiamo formalizzato cosa è un passo di derivazione, e tramite questa nozione abbiamo anche stabilito cosa è un linguaggio. Vediamo ora una caratterizzazione alternativa alla nozione di derivazione, che sono gli *alberi sintattici*. Gli alberi sintattici sono anche chiamati *alberi di sintassi astratta*. Ricordiamo che un albero è un grafo etichettato con un vertice designato, detto *radice* e tale che ogni vertice è raggiungibile dalla radice con un unico cammino.

Definizione 6.14 Un grafo etichettato su un insieme di etichette $\mathcal{E}$ è la quadrupla $\mathcal{V} = (V, A, v_{in}, \ell)$ dove

- V è un insieme di vertici;
- $A \subseteq V \times V$ è un insieme di archi;
- $v_{in} \in V$ è un vertice detto radice; e
- $\ell : V \rightarrow \mathcal{E}$ è una funzione totale di etichettatura.

Definiamo cosa è un cammino in un albero:

Definizione 6.15 Sia (V, A, v_{in}, ℓ) un grafo, un cammino ρ su tale grafo è una sequenza di vertici $\rho = v_0 \dots v_n$ tale che $\forall i \in \{1, \dots, n\}$ abbiamo che $(v_{i-1}, v_i) \in A$. Del cammino ρ diciamo che parte da v_0 ed arriva a v .

Ricordiamo la nozione di albero.

Definizione 6.16 Sia (V, A, v_{in}, ℓ) un grafo, allora (V, A, v_{in}, ℓ) è un albero se e solo se per tutti i vertici $v \in V$ esiste un unico cammino da v_{in} a v .

Di un albero diciamo che un vertice v è un nodo interno se esiste un vertice v' tale che $(v, v') \in A$, mentre v è una foglia se non è un nodo interno. Dato un nodo interno v , allora chiamiamo v' nodo figlio. Dato un vertice v , i suoi figli sono l'insieme $\{v' \mid (v, v') \in A\}$, e questo insieme è indicato con $\text{succ}(v)$. Osserviamo che una foglia v è tale che $\text{succ}(v) = \emptyset$.

Per noi sarà importante sottolineare che per ogni nodo interno (che quindi ha dei figli) l'ordine con cui sono elencati i figli (scritti da sinistra verso destra) è rilevante.

Possiamo formalizzare la nozione di albero sintattico.

Definizione 6.17 Sia $G = (\mathbf{V}, A, \mathcal{P}, S)$ una CFG. Un albero sintattico di G è un albero (V, A, v_{in}, ℓ) etichettato su $\mathbf{V} \cup A \cup \{\epsilon\}$ tale che

- esiste una relazione $\prec \subseteq V \times V$;
- ogni vertice interno è etichettato con una variabile in $\mathbf{V}$;
- ogni foglia è etichettata con un simbolo da $\mathbf{V} \cup A \cup \{\epsilon\}$; e
- se un vertice interno v è etichettato con X ed i suoi figli $\text{succ}(v) = \{v_1, \dots, v_n\}$ sono etichettati con $X_1, X_2, \dots, X_n$ allora
 - $\prec^* \cap (\text{succ}(v) \times \text{succ}(v))$ è un ordine totale; e
 - c'è una produzione $X \rightarrow X_1 X_2 \dots X_n$ tale che $\ell(v_i) = X_i$ e $\forall i \leq j \leq n$ $v_i \prec^* v_j$.

La relazione $\prec$ ci serve per poter distinguere alberi apparentemente isomorfi imponendo un ordine nella sequenza dei nodi come li leggiamo.

Perché abbiamo introdotto la nozione di albero sintattico? L'idea è che se leggiamo le foglie da sinistra a destra possiamo trovare una stringa derivabile dal simbolo non terminale che è nella radice. Colleghiamo gli alberi sintattici alle derivazioni.

Teorema 6.4 Sia $G = (\mathbf{V}, A, \mathcal{P}, S)$. Allora w è derivabile da $X \in \mathbf{V}$ se e solo se esiste un albero sintattico in cui leggiamo w e che ha come etichetta del nodo radice X .

Avendo mostrato che sia l'albero sintattico che le derivazioni sono equivalenti abbiamo che, data una CFG, ci sono vari modi per ottenere le parole appartenenti al linguaggio generato dalla grammatica: o costruiamo una derivazione oppure un albero sintattico. I due metodi sono equivalenti.

6.3.2 Derivazioni left-most, right-most e ambiguità

Diamo ora alcune definizioni che saranno importanti quando parleremo dei modi di fare parsing.

Definizione 6.18 Data la grammatica CFG $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ e la derivazione $\alpha \xRightarrow{*} \beta$. Diciamo che la derivazione è *leftmost* se e solo se sono stati riscritti i non terminali più a sinistra. In questo caso indichiamo $\alpha \xRightarrow{*}_{lm} \beta$

Analogamente:

Definizione 6.19 Data la grammatica CFG $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ e la derivazione $\alpha \xRightarrow{*} \beta$. Diciamo che la derivazione è *rightmost* se e solo se sono stati riscritti i non terminali più a destra. In questo caso indichiamo $\alpha \xRightarrow{*}_{rm} \beta$

Vediamo un esempio. Consideriamo la grammatica $G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a}, E \rightarrow \mathbf{b}, E \rightarrow \emptyset, E \rightarrow \epsilon, E \rightarrow (E), E \rightarrow E^*, E \rightarrow E + E, E \rightarrow E \cdot E\}, E)$. Deriviamo $\mathbf{a} + (\mathbf{b}^* + \mathbf{a})$ in due modi:

- (leftmost) $E \Rightarrow E + E \Rightarrow \mathbf{a} + E \Rightarrow \mathbf{a} + (E) \Rightarrow \mathbf{a} + (E + E) \Rightarrow \mathbf{a} + (E^* + E) \Rightarrow \mathbf{a} + (\mathbf{b}^* + E) \Rightarrow \mathbf{a} + (\mathbf{b}^* + \mathbf{a})$
- (rightmost) $E \Rightarrow E + E \Rightarrow E + (E) \Rightarrow E + (E + E) \Rightarrow E + (E + \mathbf{a}) \Rightarrow E + (E^* + \mathbf{a}) \Rightarrow E + (\mathbf{b}^* + \mathbf{a}) \Rightarrow \mathbf{a} + (\mathbf{b}^* + \mathbf{a})$

È facile vedere che le due derivazioni precedenti hanno alberi isomorfi.

6.3.3 Ambiguità

Consideriamo la grammatica

$$G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid (E) \mid E^* \mid E + E, \mid E \cdot E\}, E)$$

e la parola $\mathbf{a} + \mathbf{a} \cdot \mathbf{b}$ ha due derivazioni leftmost:

- $E \Rightarrow E + E \Rightarrow \mathbf{a} + E \Rightarrow \mathbf{a} + E \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot \mathbf{b}$
- $E \Rightarrow E \cdot E \Rightarrow E + E \cdot E \Rightarrow \mathbf{a} + E \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot \mathbf{b}$

Definizione 6.20 Una grammatica G è *ambigua* se per una stessa parola esistono due derivazioni leftmost diverse.

Diverse significa che gli alberi sintattici associati non sono isomorfi.

Molte ambiguità sono eliminabili. Consideriamo la grammatica

$$G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid (E) \mid E^* \mid E + E, \mid E \cdot E\}, E)$$

sappiamo che gli operatori hanno precedenze e legano più di altri, o godono di proprietà... possiamo tenerne conto. Sappiamo che il ‘ $\cdot$ ’ lega di più del ‘ $+$ ’, ed il ‘ $*$ ’ più del ‘ $\cdot$ ’: teniamone conto cercando di *non confondere* le derivazioni. Introduciamo dei nuovi simboli non terminali:

- $E \rightarrow T \mid E + T$ - un’espressione regolare è la somma di due espressioni generate da T ed E oppure da una sola in cui *all’esterno* non c’è un $+$,

- $T \rightarrow F \mid T \cdot F$ - un'espressione regolare è la concatenazione di due espressioni generate da T ed F oppure da una sola in cui *all'esterno* non c'è un $*$,
- $F \rightarrow K \mid F^*$ - un'espressione regolare ha come operatore principale lo star di Kleene,
- $K \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon$ - Il resto. Tengo anche conto che tutte le operazioni associative in effetti associano a sinistra.

Deriviamo $\mathbf{a} + \mathbf{a} \cdot \mathbf{b}$

$$E \Rightarrow E + T \xRightarrow{*} \mathbf{a} + T \Rightarrow \mathbf{a} + T \cdot F \Rightarrow \mathbf{a} + F \cdot F \Rightarrow \mathbf{a} + K \cdot F \Rightarrow \mathbf{a} + \mathbf{a} \cdot F \Rightarrow \mathbf{a} + \mathbf{a} \cdot K \Rightarrow \mathbf{a} + \mathbf{a} \cdot \mathbf{b}$$

Se provassimo a derivare dal $\cdot$ avremmo

$$E \Rightarrow T \Rightarrow T \cdot F \Rightarrow F \cdot F \Rightarrow K \cdot F$$

ma da qui o metto parentesi oppure un terminale. In effetti introduciamo nuove variabili per *forzare* la corretta generazione dell'espressione.

Possiamo eliminare qualsiasi ambiguità, ovvero esiste per ogni linguaggio generato da una grammatica ambigua, una grammatica che genera lo stesso linguaggio e che non è ambigua? La risposta è che esistono linguaggi che sono inerentemente ambigui, un esempio è $\mathcal{L} = \{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$.

6.3.4 Push Down Automata

Abbiamo bisogno di un dispositivo per riconoscere le parole di un linguaggio context-free. Sappiamo che non ci basta un automa a stati finiti per riconoscere un linguaggio come $\{0^n 1^n \mid n \geq 0\}$, che è context-free. Quale è la *debolezza* degli automi a stati finiti? Hanno una *memoria* limitata (dal numero degli stati). Per riconoscere $0^n 1^n$ devo ricordarmi quanti 0 ho visto, quindi non posso codificare tale memoria con gli stati. Quale è la memoria più semplice che abbiamo a disposizione? La *pila*.

Diamo una descrizione informale di un automa con una memoria a pile: ho un dispositivo di controllo (un automa a stati finiti) e una memoria a pila. Leggo sul nastro d'ingresso un simbolo (che può essere anche la parola vuota) e *leggo* sulla pila il simbolo che c'è in cima. A seconda dello stato si leva il simbolo dalla cima della pila e lo si sostituisce con una parola, si consuma il simbolo d'input (tranne che se si è letta la parola vuota) e si cambia eventualmente stato.

Definizione 6.21 *Un automa a pila (o PDA, Push-Down Automaton) è la 7-upla $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ dove*

1. Q è un insieme finito di stati;
2. Σ è l'alfabeto d'input, ed è finito;
3. Γ è l'alfabeto della pila, ed è finito;
4. $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$ è la funzione di transizione
5. q_0 è lo stato iniziale;
6. Z_0 è il simbolo iniziale che è presente sulla pila all'inizio della computazione
7. $F \subseteq Q$ è l'insieme degli stati accettanti o finali.

Non è necessario che Σ e Γ siano disgiunti, ed il PDA è non deterministico: vedremo che qui il non determinismo conta. Vediamo un esempio:

$$P = (\{q_0, q, q_f\}, \{0, 1\}, \{Z_0, 0\}, \delta, q_0, Z_0, \{q_f\})$$

Le mosse sono:

- $\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$. Se leggo uno 0 e sulla pila c'è Z_0 , lo consumo e memorizzo sulla pila che ho trovato il primo zero
- $\delta(q_0, 0, 0) = \{(q_0, 00)\}$. Se leggo uno 0 e sulla pila c'è 0 (significa che ho già trovato almeno uno 0), lo consumo e memorizzo sulla pila che ho trovato un altro zero
- $\delta(q_0, 1, Z_0) = \emptyset$. Se leggo uno 1 e sulla pila c'è Z_0 , vuol dire che la parola inizia per 1
- $\delta(q_0, 1, 0) = (q, \epsilon)$. Se sono nello stato q_0 , leggo un 1 e sulla pila c'è 0, vuol dire che ho visto fino ad ora solo 0 ed ora inizio a vedere 1. Levo lo 0 dalla pila....
- $\delta(q, 1, 0) = (q, \epsilon)$. Se sono nello stato q , leggo un 1 e sulla pila c'è 0, vuol dire che ho iniziato a vedere 1. Levo lo 0 dalla pila e continuo
- $\delta(q, 0, a) = \emptyset$. Se sono nello stato q e leggo uno 0 allora qualsiasi simbolo c'è sulla pila ($a \in \Gamma$), la parola non appartiene al linguaggio
- $\delta(q, \epsilon, Z_0) = (q_f, \epsilon)$, se ho consumato tutto l'input, ho messo tanti 0 sulla pila quanti ne ho tolto, allora posso accettare la parola
- le altre danno sempre l'insieme vuoto.

Se nei DFA la configurazione era una coppia *stato* e *parola da leggere*, ora è più complessa: è una tripla *stato*, *parola da leggere* e *pila*. Lo formalizziamo dando la nozione di *descrizione istantanea* che ci serve per descrivere ogni *snapshot* del sistema.

Definizione 6.22 Dato un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, una *descrizione istantanea* è la tripla (q, w, γ) dove $q \in Q$ indica in quale stato si trova l'automa a pila P , $w \in \Sigma^*$ indica cosa ancora devo leggere e $\gamma \in \Gamma^*$ indica cosa è presente sulla pila. La *descrizione istantanea iniziale* è (q_0, w, Z_0) .

Come passo da una descrizione istantanea ad un'altra?

Definizione 6.23 Dato un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ e una *descrizione istantanea* (q, w, γ) , la *descrizione* (q', w', γ') è *raggiungibile* da (q, w, γ) se esiste in $\delta(q, a, A)$ la coppia (q', α) tale che $w = aw'$ e $\gamma' = \alpha\beta$ posto che $\gamma = A\beta$.

La *transizione* viene indicata con $(q, w, \gamma) \vdash (q', w', \gamma')$.

Le descrizioni istantanee hanno delle proprietà. Abbiamo una relazione di transizione. Osserviamo che una sequenza di descrizioni istantanee assomiglia molto ad una *computazione*.

Definizione 6.24 Dato un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ e una *descrizione istantanea* (q, w, γ) , la *descrizione* (q', w', γ') è *raggiungibile* da (q, w, γ) se esiste in $\delta(q, a, A)$ la coppia (q', α) tale che $w = aw'$ e $\gamma' = \alpha\beta$ posto che $\gamma = A\beta$. La *transizione* viene indicata con $(q, w, \gamma) \vdash (q', w', \gamma')$.

Facciamone la chiusura transitiva e riflessiva e indichiamola con $\vdash^*$ (mi concentro su cosa ottengo e non solo sui passi). Se una sequenza di descrizioni istantanee (cioè una computazione) è lecita per P allora lo è anche quella sequenza in cui a tutte le ID abbiamo aggiunto lo stesso suffisso alla stringa d'input: quella parte d'input non l'ho ancora vista, e non so se la dovrò vedere.

Lemma 6.2 *Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA e sia $(q, x, \alpha) \vdash^* (p, y, \beta)$, allora per ogni $w \in \Sigma^*$, $\gamma \in \Gamma^*$ vale che $(q, xw, \alpha\gamma) \vdash^* (p, yw, \beta\gamma)$*

Vale anche il vice versa. Se una sequenza di descrizioni istantanee (cioè una computazione) è lecita per P , e resta una coda d'input non consumata, allora lo è anche quella sequenza in cui a tutte le ID abbiamo levato lo stesso residuo alla stringa d'input: come l'aggiungiamo, possiamo anche levarla (attenzione, leviamo solo una parte che nella computazione non è mai stata, per così dire, *coinvolta*).

Infine, se una sequenza di descrizioni istantanee (cioè una computazione) è lecita per P allora lo è anche quella sequenza in cui a tutte le ID abbiamo aggiunto lo stesso suffisso alla stringa che rappresenta la pila: nello stack, nella parte che non vedo, ci può essere qualsiasi cosa.

Lemma 6.3 *Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA e $(q, xw, \alpha) \vdash^* (p, yw, \beta)$, allora anche $(q, x, \alpha) \vdash^* (p, y, \beta)$*

Fissiamo ora alcuni concetti, che giocano un ruolo essenziale:

- l'automa a pila è non deterministico (scelgo tra varie possibili mosse),
- ho una pila sulla quale, per continuare, devo leggere qualche simbolo,
- ho degli stati finali o accettanti, e
- ho una ID iniziale.

Definizione 6.25 *Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA, allora il linguaggio riconosciuto dall'automa per stato finale è*

$$\mathcal{L}_{\text{PDA}}(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \gamma) \text{ con } p \in F\}$$

Sono quindi tutte le parole che l'automa a pila riconosce perchè, consumato tutto l'input, sono in uno stato finale. Sulla pila ci possono essere moltissimi simboli.

Diamo una definizione alternativa di automa a pila in cui cambia la regola d'accettazione.

Definizione 6.26 *Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA, allora il linguaggio riconosciuto dall'automa per stack vuoto è*

$$\mathcal{N}_{\text{PDA}}(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon)\}$$

Sono quindi tutte le parole che l'automa a pila riconosce perchè, consumato tutto l'input, anche sulla pila non c'è più alcun simbolo. La differenza è che non si pongono condizioni sul fatto che p possa o no essere uno stato finale, e quindi possiamo omettere gli stati accettanti: abbiamo quindi che un automa a pila con questa caratteristica sarà $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$

Vediamone un esempio. Costruiamo un PDA che accetta $\{a^i b^{2i} \mid i \geq 1\}$

$$(\{q_0, q, q_f\}, \{a, b\}, \{A, Z_0\}, \delta, q_0, Z_0)$$

- $\delta(q_0, a, Z_0) = \{(q_0, AA)\}$ leggo una a , metto AA sulla pila e proseguo

- $\delta(q_0, a, A) = \{(q_0, AAA)\}$ leggo una a , metto AAA sulla pila e proseguo
- $\delta(q_0, b, A) = \{(q, \epsilon)\}$ leggo una b e levo il simbolo sulla pila (A) e proseguo
- $\delta(q, b, A) = \{(q, \epsilon), (q_f, \epsilon)\}$ leggo una b e levo il simbolo sulla pila (A) e proseguo oppure immagino di aver terminato

Ogni volta che leggo una a metto due AA sulla pila, se invece leggo una b ne levo una. Una computazione è

$$(q_0, abb, Z_0) \vdash (q_0, bb, AA) \vdash (q, b, A) \vdash (q, \epsilon, \epsilon)$$

L'automa che accetta per stato finale non è più *potente* di quello che accetta per stack vuoto.

Teorema 6.5 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA che accetta per stato finale. Allora esiste $P^N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta^N, p_0, X_0)$, PDA che accetta lo stesso linguaggio per pila vuota, i.e., $\mathcal{L}_{\text{PDA}}(P) = \mathcal{N}_{\text{PDA}}(P^N)$.

Anche il vice versa vale.

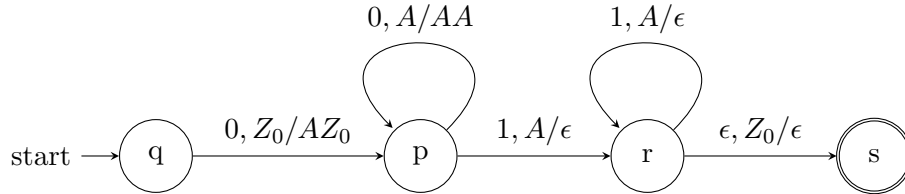
Teorema 6.6 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ un PDA che accetta per stack vuoto. Allora esiste $P^L = (Q \cup \{p_0, p_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta^L, p_0, X_0, \{p_f\})$, PDA che accetta lo stesso linguaggio per stato finale, i.e., $\mathcal{L}_{\text{PDA}}(P^L) = \mathcal{N}_{\text{PDA}}(P)$.

6.3.5 Rappresentazione grafica di un pda

Per rappresentare graficamente un PDA si usa il diagramma delle transizioni come per gli automi a stati finiti, solo che gli archi sono annotati in modo più complesso

$$P = (\{q, p, r, s\}, \{0, 1\}, \{Z_0, A\}, \delta, q, Z_0, \{r\})$$

mettiamo il simbolo letto (se c'è), il simbolo in cima alla pila e quello che viene messo sulla pila al posto del simbolo.



6.3.6 Relazione tra push down automata e grammatiche context-free

È abbastanza intuitivo immaginare che i PDA sono proprio il dispositivo che accetta un linguaggio generato da una grammatica context-free.

Teorema 6.7 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Allora esiste un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ tale che $\mathcal{L}(G) = \mathcal{N}_{\text{PDA}}(P)$.

Teorema 6.8 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ un PDA che accetta per pila vuota. Allora esiste una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ tale che $\mathcal{L}(G) = \mathcal{N}_{\text{PDA}}(P)$.

A questo punto possiamo, come per i linguaggi regolari, definire la classe dei linguaggi context-free.

Definizione 6.27 Sia dato un alfabeto A . La classe dei linguaggi context-free su A , che indichiamo con $\mathcal{L}_{cf}^A$, è l'insieme $\mathcal{L}_{cf}^A = \{\mathcal{L} \subseteq A^* \mid \mathcal{L} \text{ è generato da una CFG}\}$.

Chiaramente abbiamo che $\mathcal{L}_{cf}^A = \{\mathcal{L} \subseteq A^* \mid \mathcal{L} \text{ è accettato da un PDA}\}$.

6.3.7 Determinismo nei push down automata

Consideriamo il linguaggio $\{ww^R \mid w \in \{a,b\}^*\}$. Osserviamo che l'abbiamo già visto: infatti se applichiamo la reverse ad una parola del linguaggio abbiamo: $(ww^R)^R = (w^R)^R w^R$ e sapendo che $(w^R)^R = w$ abbiamo che $(ww^R)^R = (w^R)^R w^R = ww^R$.

Chiaramente l'automa ad un certo punto deve decidere *sono in mezzo alla parola*. Non ho modo di evitare questa *guess*.

6.3.8 Forme normali per le grammatiche context-free

Se un linguaggio è context free, allora ha una grammatica, e in particolare ha una grammatica le cui produzioni hanno tutte una forma particolare. Quelle che vediamo sono due, la forma normale di *Chomsky* e quella di *Greibach*. La prima forma normale è tale che c'è una precisa relazione tra la lunghezza della derivazione e l'albero sintattico della parola derivata, mentre la forma normale di Greibach è usata soprattutto dagli algoritmi di parsing.

Forma normale di Chomsky

Consideriamo solo linguaggi context free che non contengono la parola vuota. Questa non è una limitazione, come subito vediamo.

Teorema 6.9 *Sia $\mathcal{L}$ un linguaggio context free tale che $\epsilon \notin \mathcal{L}$. Allora esiste una grammatica G che genera il linguaggio $\mathcal{L} \cup \{\epsilon\}$*

Definizione 6.28 *Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. G è in forma normale di Chomsky (CNF) se e solo se tutte le produzioni sono della forma $A \rightarrow BC$ o $A \rightarrow a$, dove $A, B, C \in \mathbf{V}$ e $a \in \Sigma$.*

Il teorema che vogliamo provare dice che ogni grammatica che genera un linguaggio context free senza ' ϵ ' può essere messa in CFN. Per far questo dobbiamo:

- Eliminare i simboli inutili, cioè quelli che non appaiono in nessuna derivazione $S \xRightarrow{*} w$, per S simbolo iniziale e $w \in \Sigma^*$
- Eliminare le produzioni della forma $A \rightarrow \epsilon$
- Eliminare le produzioni unità (i.e., della forma $A \rightarrow B$, dove $A, B \in \mathbf{V}$)
- Eliminare le produzioni con più di due non terminali sulla destra

Eliminazione dei simboli inutili

Definizione 6.29 *Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Un simbolo $V \in \mathbf{V}$ è inutile se e solo se $\forall w \in \mathcal{L}(G)$ e per ogni derivazione $S \xRightarrow{*} w$, V non compare nella derivazione.*

Possiamo eliminare i simboli inutili:

Teorema 6.10 *Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica, e sia $\mathcal{L} = \mathcal{L}(G) \neq \emptyset$. Allora esiste una grammatica G' senza simboli inutili tale che $\mathcal{L} = \mathcal{L}(G')$*

Come troviamo i simboli inutili? Partiamo dalla grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$. Cerchiamo i simboli *utili*, che sono facili da trovare, intuitivamente sono quelli che troviamo all'interno di una derivazione.

- $X \in \mathbf{V}$ è un *generatore* se esiste una parola $w \in \Sigma^*$ tale che $X \xRightarrow{*} w$
- X è raggiungibile se esiste una derivazione $S \xRightarrow{*} \alpha X \beta$

I simboli utili sono tutti quelli che sono generatori e raggiungibili. Per trovarli procediamo a ritroso: tutte le produzioni che riscrivono in stringhe di soli terminali sono generatori. Se $A \rightarrow \alpha$ è una produzione e tutti i simboli di α sono terminali o generatori, allora A è un generatore. Infine basta osservare che un simbolo X (generatore) è raggiungibile se e solo se S è un generatore.

Eliminare le produzioni della forma $A \rightarrow \epsilon$

Cerchiamo tutti i simboli di variabile di una grammatica che sono *annullabili*. Diamo la definizione.

Definizione 6.30 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Un simbolo $A \in \mathbf{V}$ è annullabile se

- esiste una produzione $A \rightarrow \epsilon$, o
- esiste una produzione $A \rightarrow Y_1 Y_2 \dots Y_k$ e ogni Y_i è annullabile

Non deve sfuggire che questa definizione ci dà un modo per calcolare quali sono i simboli annullabili. Ricordiamoci che stiamo considerando linguaggi senza la parola vuota.

Teorema 6.11 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica senza simboli inutili, e sia $\mathcal{L} = \mathcal{L}(G)$. Allora esiste una grammatica G' dove non ci sono produzioni $A \rightarrow \epsilon$ tale che $\mathcal{L} = \mathcal{L}(G')$.

Eliminare le produzioni della forma $A \rightarrow B$, dove $A, B \in \mathbf{V}$

Iniziamo con la definizione di coppia unitaria.

Definizione 6.31 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Una coppia unitaria è un elemento $(X, Y) \in \mathbf{V} \times \mathbf{V}$ tale che $X \xRightarrow{*} Y$.

Una coppia unitaria è una coppia di non terminali che sono derivabili uno dall'altro. Come calcoliamo le coppie unitarie?

- (A, A) è una coppia unitaria
- se (A, B) è una coppia unitaria e $B \rightarrow C$ con $B, C \in \mathbf{V}$ è una produzione, allora anche (A, C) è una coppia unitaria

Per ogni coppia unitaria (A, B) si aggiunge la produzione $A \rightarrow \alpha$, dove $B \rightarrow \alpha$ è una produzione non unitaria in $\mathcal{P}$.

Teorema 6.12 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica con soli simboli utili, e sia $\mathcal{L} = \mathcal{L}(G)$. Allora esiste una grammatica G' dove non ci sono produzioni unitarie tale che $\mathcal{L} = \mathcal{L}(G')$.

Produzioni con soli non terminali o soli terminali

Prendiamo una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ e una produzione $Y \rightarrow X_1 X_2 \dots X_n$. Se X_i è una variabile, la lasciamo. Se invece X_i è un simbolo terminale, introduciamo una nuova variabile $\bar{X}_i$, che sostituiamo a X_i nel lato destro della produzione, e introduciamo una nuova produzione $\bar{X}_i \rightarrow X_i$.

Chiaramente la nuova grammatica G' così ottenuta è tale che $\mathcal{L}(G) = \mathcal{L}(G')$. Vediamo quindi come eliminare le produzioni con più di due non terminali sulla destra: l'idea è ovvia: rimpiazzo la produzione con una serie di produzioni che simulano l'effetto della produzione che devo rimpiazzare, introducendo nuove variabili.

Teorema 6.13 *Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica e $A \rightarrow \alpha$ una produzione in $\mathcal{P}$ tale che $\alpha \in \mathbf{V}^*$ e $|\alpha| > 2$. Sia $\alpha = X\beta$, Allora possiamo introdurre una nuova variabile $[\beta]$ e la produzione $[\beta] \rightarrow \beta$ e sostituire $A \rightarrow \alpha$ con $A \rightarrow X[\beta]$ tali che $A \Rightarrow_G \alpha$ se e solo se $A \Rightarrow_{G'} X[\beta] \Rightarrow_{G'} X\beta$.*

Forma normale di Greibach

Esiste anche un'altra forma normale. In questo caso le produzioni sono del tipo $A \rightarrow a\alpha$ dove $a \in \Sigma$ e $\alpha \in \mathbf{V}^*$.

Osserviamo prima alcuni fatti:

Lemma 6.4 *Consideriamo una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ in cui tutti i simboli sono utili. Consideriamo una produzione $A \rightarrow \alpha_1 B \alpha_2$ e le produzioni $B \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$. Allora possiamo sostituire $A \rightarrow \alpha_1 B \alpha_2$ con $A \rightarrow \alpha_1 \beta_1 \alpha_2 \mid \alpha_1 \beta_2 \alpha_2 \mid \dots \mid \alpha_1 \beta_n \alpha_2$ e la nuova grammatica $G' = (\mathbf{V}, \Sigma, \mathcal{P}', S)$ così ottenuta genera lo stesso linguaggio.*

Lemma 6.5 *Consideriamo una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ in cui tutti i simboli sono utili. Consideriamo tutte le produzioni $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_k$ e siano tutte le altre produzioni $A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ che riscrivono A . Definiamo $G' = (\mathbf{V} \cup \{B\}, \Sigma, \mathcal{P}', S)$, con $B \notin \mathbf{V}$ e $\mathcal{P}'$ ottenuto rimpiazzando tutte le produzioni che riscrivono A con le seguenti: $A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ e $A \rightarrow \beta_1 B \mid \beta_2 B \mid \dots \mid \beta_n B$ e per la nuova variabile B si introducono le produzioni $B \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$ e $B \rightarrow \alpha_1 B \mid \alpha_2 B \mid \dots \mid \alpha_k B$. Allora $\mathcal{L}(G') = \mathcal{L}(G)$.*

Teorema 6.14 *Ogni linguaggio $\mathcal{L}$ context free senza la parola vuota può essere generato da una grammatica in cui le produzioni sono della forma $A \rightarrow a\alpha$, dove a è un terminale e α è una stringa, eventualmente vuota, di variabili.*

6.3.9 Operazioni sui linguaggi context free

Vediamo ora le proprietà di chiusura. Osserviamo che alcune delle proprietà di chiusura dei linguaggi regolari valgono anche per quelli context free: unione, concatenazione, star di Kleene, intersezione con un linguaggio regolare, omomorfismo e omomorfismo inverso, vedremo anche perché.

I linguaggi context-free sono chiusi per sostituzione.

Teorema 6.15 *Sia $\mathcal{L}$ un linguaggio context free su Σ , e sia s una sostituzione tale che $s(a)$ è un linguaggio context free per ogni $a \in \Sigma$. Allora $s(\mathcal{L})$ è un linguaggio context free.*

Basta sostituire ad ogni terminale a il simbolo di start della grammatica associata al linguaggio immagine di a .

I linguaggi context-free, grazie al fatto che sono chiusi per sostituzione, sono chiusi anche rispetto alle operazioni di unione, concatenazione, star di Kleene, chiusura positiva e omomorfismo. Diamo

l'intuizione per la concatenazione. Consideriamo i linguaggi context free $\mathcal{L}_0$ e $\mathcal{L}_1$ che saranno generati dalle grammatiche $G_0 = (V_0, \Sigma, \mathcal{P}_0, S_0)$ ed $G_1 = (V_1, \Sigma, \mathcal{P}_1, S_1)$ e assumiamo senza perdere generalità che $V_0 \cap V_1 = \emptyset$. Consideriamo la grammatica $G = (\{S\}, \{0, 1\}, \{S \rightarrow 01\}, S)$, è facile vedere che, ponendo $s(0) = \mathcal{L}_0$ ed $s(1) = \mathcal{L}_1$, che $s(01) = \mathcal{L}_0\mathcal{L}_1$.

Teorema 6.16 *I linguaggi context free sono chiusi rispetto all'unione, concatenazione, star di Kleene, chiusura positiva e omomorfismo.*

Data una parola, la sua reverse la si ottiene scrivendo la parola da al contrario. Se generalizziamo ad un insieme, abbiamo che i linguaggi context free sono chiusi rispetto all'inversione. Per questo basta sostituire tutte le produzioni $A \rightarrow \alpha$ con $A \rightarrow \alpha^R$.

Usando invece gli automi a pila e gli automi a stati finiti possiamo vedere che i linguaggi context free sono chiusi rispetto all'intersezione con i linguaggi regolari ed anche che i linguaggi context free sono chiusi rispetto all'omomorfismo inverso.

6.3.10 Pumping lemma per linguaggi context-free

L'intuizione nel pumping lemma per i linguaggi regolari era: per parole sufficientemente lunghe l'automa a stati finiti deve ripassare per due volte nello stesso stato, e la parola che ho visto dal primo al secondo attraversamento la posso pompare. Per il linguaggi context free ragiono sull'albero sintattico. Se nell'albero di una derivazione di una grammatica in forma normale di Chomsky c'è un cammino abbastanza lungo, allora devo vedere almeno due volte lo stesso simbolo non terminale. Le parole derivate derivate da quel simbolo le posso pompare.

Vediamo prima la relazione tra $|w|$ e l'albero sintattico.

Proposizione 6.1 *Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica in forma normale di Chomsky e sia $w \in \mathcal{L}(G)$. Se la lunghezza del cammino più lungo del suo albero di derivazione è n allora $|w| \leq 2^{n-1}$.*

Lemma 6.6 *Sia $\mathcal{L}$ un linguaggio context free. Allora esiste una costante n tale che se $z \in \mathcal{L}$ e $|z| \geq n$ allora z può essere scomposta in cinque parti $z = uvwxy$ tali che*

1. $|vwx| \leq n$,
2. $vx \neq \emptyset$,
3. $\forall i \geq 0$ la parola $uv^iwx^iy \in \mathcal{L}$.

Mostriamo che $\{0^n 1^n 2^n \mid n \geq 1\}$ non è context free. Assumiamo lo sia, esiste una costante m tale che possiamo scomporre ogni parola in cinque parti $uvwxy$ in modo tale $|vwx| \leq m$, $vx \neq \emptyset$ e che sia possibile *pompare* le v e x in modo che anche la parola uv^iwx^iy sia nel linguaggio. Prendiamo come parola $0^m 1^m 2^m$, allora qualsiasi scomposizione di $0^m 1^m 2^m$ in $uvwxy$ tale che $|vwx| \leq m$ deve essere tale che vwx contiene o 0 ed 1, o 1 e 2. Comunque io divida vwx , dato che $vx \neq \epsilon$ vale che o v contiene solo 0 e x contiene solo 1, oppure v contiene solo 1 e x contiene solo 2. Comunque le pompi non sono in grado di avere parole che ancora stanno nel linguaggio, quindi la tesi.

Lo schema è sempre lo stesso: prendo una parola, devo mostrare che qualsiasi suddivisione che soddisfa le ipotesi del pumping lemma, porta alla falsificazione del fatto che la parola *pompata* appartiene al linguaggio.

Usando il pumping lemma possiamo vedere che i linguaggi context-free non sono chiusi per intersezione.

Teorema 6.17 *I linguaggi context free non sono chiusi rispetto all'intersezione.*

Consideriamo $\mathcal{L}_1 = \{a^n b^n c^m \mid n, m \geq 1\}$ e $\mathcal{L}_2 = \{a^n b^m c^m \mid n, m \geq 1\}$. Chiaramente sono entrambi context free ma la loro intersezione $\{a^n b^n c^n \mid n \geq 1\}$ non lo è.

6.3.11 Gerarchia di linguaggi

Prima di parlare nuovamente di lexer e parser, illustriamo brevemente i risultati principali che riguardano la gerarchia delle classi di linguaggi che abbiamo visto.

Introduciamo le grammatiche regolari.

Definizione 6.32 Una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ è detta regolare se e solo se tutte le sue produzioni sono della forma $A \rightarrow aB$, con $a \in \Sigma$ e $B \in \mathbf{V}$, oppure $A \rightarrow a$, con $a \in \Sigma \cup \{\epsilon\}$, e se $A \rightarrow \epsilon$ allora $A = S$.

Teorema 6.18 Un linguaggio è regolare se e solo se è generato da una grammatica regolare.

Corollario 6.1 Ogni linguaggio regolare è anche context free.

Un linguaggio regolare viene generato da una grammatica con opportune restrizioni (viene chiamata lineare destra). Abbiamo percorso il primo passo della gerarchia di Chomsky: dai linguaggi regolari a quelli context free.

Teorema 6.19 $L_{reg} \subset L_{CFG}$.

Chiaramente $L_{reg} \subseteq L_{cf}$ dato che abbiamo visto che un linguaggio regolare viene generato da una grammatica regolare. Per il contenimento proprio osserviamo che $\{0^n 1^n \mid n \in \mathbb{N}\}$ sta in L_{CFG} ma non in L_{reg} .

6.4 Linguaggi context-free deterministici

Abbiamo visto che gli automi a pila possono anche essere deterministici. Questo suggerisce che possiamo caratterizzare una classe di linguaggi context-free deterministici.

Definizione 6.33 Un linguaggio context-free $\mathcal{L}$ è deterministico se esiste un automa a pila deterministico P che accetta tale linguaggio.

Il linguaggio $\mathcal{L} = \{w \mid w = w^R \text{ e } |w| \text{ pari}\}$ non è accettato da alcun automa a pila deterministico. Infatti l'automato deve fare la guess di essere arrivata a centro della parola. Esistono quindi linguaggi context-free che non sono deterministici.

I linguaggi context-free deterministici godono di varie proprietà di chiusura, che sono differenti da quelle dei linguaggi context-free. Consideriamo i linguaggi $\mathcal{L} = \{a^n b^n c^m \mid n, m \geq 1\}$ e $\mathcal{L}' = \{a^n b^m c^m \mid n, m \geq 1\}$, sono entrambi deterministici, ma la loro unione non lo è. I linguaggi context-free deterministici non sono chiusi per unione, concatenazione, star di Kleene e omomorfismo. Sono invece chiusi per intersezione, quoziente con linguaggi regolari e $\min$ e $\max$ dove

$$\max(\mathcal{L}) = \{x \in \mathcal{L} \mid x \text{ non è il prefisso proprio di alcuna parola } y \in \mathcal{L}\}$$

e

$$\min(\mathcal{L}) = \{x \in \mathcal{L} \mid \text{non c'è alcun prefisso proprio } y \text{ di } x \text{ in } \mathcal{L}\}$$

Bisogna infine osservare che se un linguaggio è deterministico allora non è ambiguo.

Fortunatamente i linguaggi che consideriamo sono in generale deterministici. In particolare ci interessano i linguaggi LL e quelli LR.

Diamone una definizione.

Definizione 6.34 *Un linguaggio context-free è LL se per ogni derivazione esiste un parser top down che costruisce una derivazione left most.*

Un linguaggio context-free è LR se per ogni derivazione esiste un parser bottom up che costruisce una derivazione right most.

6.5 Lexer

Dovrebbe oramai essere chiaro che un lexer altro non è che un automa a stati finiti che controlla se la sequenza di caratteri è corretta e restituisce una sequenza di token, che verranno poi usati dal parser.

6.6 Parser

Vedremo ora le due principali tecniche per scrivere un parser. Il risultato del parser sarà una derivazione o leftmost o rightmost che corrisponde ad un albero di derivazione in modo univoco, e questo albero di derivazione potrà essere tradotto direttamente in un albero di sintassi astratta che potremo valutare.

6.6.1 Parser Top Down

Un parser top down costruisce un albero di parsing per una parola, iniziando dalla radice e creando i nodi in ordine depth-first (dato un nodo, prima lui e poi i figli da destra a sinistra, ricorsivamente) e trova la derivazione leftmost per tale parola. Ad ogni passo, una produzione viene usata per trasformare una variabile e, una volta scelta la produzione, bisogna individuare i simboli terminali della parte destra della produzione nella stringa in input.

Il pseudocodice non deterministico per una variabile A della grammatica è:

1. si sceglie una produzione per il simbolo di variabile: $A \rightarrow X_1 X_2 \cdots X_k$,
2. per ogni $1 \leq i \leq k$
 - se X_i è una variabile chiamo una delle procedure associate al simbolo di variabile,
 - se X_i è un terminale ed è uguale al simbolo corrente, si passa al prossimo simbolo della parola, e
 - se non ho procedure o il terminale non è il simbolo corrente, ho un errore
3. la produzione (procedura) scelta potrebbe non essere quella giusta
 - bisogna permettere il backtracking
 - errore vuol solo dire che bisogna tornare al passo 1 per scegliere un'altra produzione (procedura) per A ,
 - se non ce ne siano più si fallisce,
 - e anche il puntatore al simbolo corrente deve tornare indietro

La grammatica potrebbe avere delle produzioni $A \rightarrow A\gamma$, quindi richiamerebbe sempre se stessa e potrebbe dare luogo ad una catena infinita (ricordiamo che la derivazione è leftmost), ma questo si può evitare sfruttando il seguente risultato.

Lemma 6.7 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica in cui tutti i simboli sono utili, siano $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_k$ delle produzioni della grammatica e siano $A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ tutte le altre produzioni che riscrivono A . Sia $G' = (\mathbf{V} \cup \{B\}, \Sigma, \mathcal{P}', S)$, con $B \notin \mathbf{V}$ e $\mathcal{P}'$ ottenuto da $\mathcal{P}$ rimpiazzando tutte le produzioni che riscrivono A con le produzioni $A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ e $A \rightarrow \beta_1 B \mid \beta_2 B \mid \dots \mid \beta_n B$, e aggiungendo, per la nuova variabile B , le produzioni $B \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$ e $B \rightarrow \alpha_1 B \mid \alpha_2 B \mid \dots \mid \alpha_k B$, una grammatica ottenuta da G . Allora $\mathcal{L}(G') = \mathcal{L}(G)$.

Questo lemma implica che la ricorsione sinistra può essere eliminata.

Il parser top down è sostanzialmente un automa a pila che ha sulla pila, all'inizio, i seguenti simboli: S e $\$$ ed è ottenuto dalla grammatica con la costruzione canonica. Il problema è capire quale produzione applicare.

Consideriamo la grammatica $E \rightarrow T + E \mid T$ e $T \rightarrow n \mid n * T \mid (E)$ (che genera alcune tra le espressioni aritmetiche non ambigue). Se uno si trova con T in cima alla pila e sull'input ho n , non so quale tra le due produzioni $T \rightarrow n$ e $T \rightarrow n * T$ applicare. Proviamo a *mettere assieme* i prefissi comuni delle produzioni. La grammatica diventa la seguente: $E \rightarrow TX$, $X \rightarrow +E \mid \epsilon$, $T \rightarrow aY \mid (E)$ e $Y \rightarrow *T \mid \epsilon$. Introduciamo due funzioni (FIRST e FOLLOW) per capire quale produzione usare.

Definizione 6.35 Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ ed una parola $\alpha \in (\mathbf{V} \cup \Sigma)^+$. Allora $\text{FIRST}(\alpha) = \{a \in \Sigma \cup \{\epsilon\} \mid \alpha \xRightarrow{*} aw \in \Sigma^*\}$.

Osserviamo che se $\alpha \xRightarrow{*} \epsilon$ allora $\epsilon \in \text{FIRST}(\alpha)$. Come usiamo questa definizione? Consideriamo due produzioni $A \rightarrow \alpha \mid \beta$. Se $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ allora basta guardare il prossimo simbolo in input. Se sta in $\text{FIRST}(\alpha)$ allora si usa la produzione $A \rightarrow \alpha$, se sta in $\text{FIRST}(\beta)$ allora si usa la produzione $A \rightarrow \beta$.

Definizione 6.36 Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ ed un simbolo di variabile $A \in \mathbf{V}$. $\text{FOLLOW}(A) = \{a \in \Sigma \cup \{\epsilon\} \mid S \xRightarrow{*} \alpha A \beta \text{ e } a \in \text{FIRST}(\beta)\}$.

Quindi $\text{FOLLOW}(A)$ è l'insieme di terminali che possono apparire alla destra di A in una derivazione da S . Se A è il simbolo più a destra in qualche derivazione, mettiamo $\$$ in $\text{FOLLOW}(A)$ (ricordiamo che sulla pila del nostro automa abbiamo messo $\$$).

Come calcoliamo il FIRST di un simbolo X ? Se $X \in \Sigma$ allora $\text{FIRST}(X) = \{X\}$, se invece X è una variabile allora $\text{FIRST}(X) = \bigcup_{X \rightarrow \alpha \in \mathcal{P}} \text{FIRST}(\alpha)$. Quindi per calcolare effettivamente $\text{FIRST}(X_1 \dots X_n)$ mettiamo in $\text{FIRST}(X_1 \dots X_n)$ tutti i simboli di $\text{FIRST}(X_1)$ tranne ϵ , e se $\text{FIRST}(X_i)$ contiene ϵ allora mettiamo anche tutti i simboli di $\text{FIRST}(X_{i+1})$ tranne ϵ , e se $\text{FIRST}(X_n)$ contiene ϵ , allora mettiamo ϵ anche in $\text{FIRST}(X_1 \dots X_n)$.

Vediamo ora come calcolare FOLLOW. $\text{FOLLOW}(S)$ contiene $\$$. Se abbiamo la produzione $A \rightarrow \alpha B \beta$ allora mettiamo in $\text{FOLLOW}(B)$ tutto ciò che sta in $\text{FIRST}(\beta)$ tranne ϵ , e se abbiamo la produzione $A \rightarrow \alpha B$ o $A \rightarrow \alpha B \beta$ e $\epsilon \in \text{FIRST}(\beta)$ allora mettiamo in $\text{FOLLOW}(B)$ tutto ciò che sta in $\text{FOLLOW}(A)$.

Vediamo un esempio di come calcolare FIRST e FOLLOW. Consideriamo la grammatica $E \rightarrow TA$, $A \rightarrow +TA \mid \epsilon$, $T \rightarrow FB$, $B \rightarrow *FB \mid \epsilon$ e $F \rightarrow (E) \mid n$. Allora:

- $\text{FIRST}(A) = \{+, \epsilon\}$, dato che abbiamo le produzioni $A \rightarrow +TA \mid \epsilon$ e chiaramente $+$ $\in \text{FIRST}(+TA)$,
- $\text{FIRST}(B) = \{*, \epsilon\}$, dato che abbiamo le produzioni $B \rightarrow *FB \mid \epsilon$ e chiaramente $*$ $\in \text{FIRST}(*FB)$,
- $\text{FIRST}(F) = \{(\text{, id})\}$, dato che abbiamo le produzioni $F \rightarrow (E) \mid \text{id}$ e chiaramente $(\in \text{FIRST}((E))$, mentre n lo mettiamo dato che $\text{FIRST}(n) = \{n\}$,

- $\text{FIRST}(T) = \{ (, n \}$, dato che abbiamo la produzione $T \rightarrow FB$ e chiaramente $\text{FIRST}(T) = \text{FIRST}(F)$, e
- $\text{FIRST}(E) = \{ (, id \}$, dato che abbiamo la produzione $E \rightarrow TA$ e chiaramente $\text{FIRST}(E) = \text{FIRST}(T)$.

Calcoliamo analogamente FOLLOW.

- $\text{FOLLOW}(E) = \{), \$ \}$, dato che abbiamo la produzione $F \rightarrow (E)$ e $\$$ lo mettiamo dato che E è il simbolo iniziale,
- $\text{FOLLOW}(A) = \{), \$ \}$, dato che abbiamo le produzioni $A \rightarrow +TA$ e $E \rightarrow TA$, quindi dobbiamo mettere in $\text{FOLLOW}(A)$ tutto ciò che sta in $\text{FOLLOW}(E)$, dato che $\text{FIRST}(A)$ contiene ϵ ,
- $\text{FOLLOW}(T) = \{ +,), \$ \}$, dato che abbiamo le produzioni $E \rightarrow TA$, $A \rightarrow +TA$, quindi dobbiamo mettere tutto quello che sta in $\text{FIRST}(A) = \{ +, \epsilon \}$ e $\$$ perché il simbolo iniziale è coinvolto,
- $\text{FOLLOW}(B) = \{ +,), \$ \}$, dato che abbiamo le produzioni $T \rightarrow FB$, $B \rightarrow *FB$, quindi bisogna mettere $\$$ e tutto quello che sta in $\text{FOLLOW}(T) = \{ +,), \$ \}$, e
- $\text{FOLLOW}(F) = \{ +, *,), \$ \}$, dato che abbiamo le produzioni $T \rightarrow FB$, $B \rightarrow *FB$, quindi dobbiamo mettere anche $\text{FIRST}(B)$ meno ϵ , cioè $\{ * \}$, poi bisogna mettere $\text{FOLLOW}(T) = \{ +,), \$ \}$ dato che ϵ sta in $\text{FIRST}(B)$.

Abbiamo ora tutti gli elementi per costruire la tabella del nostro parser, che sarebbe quella che ci dice cosa dobbiamo fare. Ricordiamo che sulla pila abbiamo $S\$$ e sull'input $w\$$.

Costruiamo una tabella **M** per trovare quale produzione usare, quindi in cima alla pila vediamo un simbolo di variabile, mentre sull'input abbiamo terminali o $\$$. La tabella avrà una riga per ogni variabile e una colonna per ogni terminale o $\$$.

L'idea è che a produzione $A \rightarrow \alpha$ viene scelta se il simbolo di input a sta in $\text{FIRST}(\alpha)$. Se $\alpha \xRightarrow{*} \epsilon$ allora la scelgo se $a \in \text{FOLLOW}(A)$.

Per ogni produzione $A \rightarrow \alpha$

- per ogni $a \in \text{FIRST}(\alpha)$ mettiamo $A \rightarrow \alpha$ in $\mathbf{M}[A, a]$,
- se $\epsilon \in \text{FIRST}(\alpha)$, mettiamo $A \rightarrow \alpha$ in $\mathbf{M}[A, b]$ per ogni $b \in \text{FOLLOW}(A)$, con b terminale,
- se $\epsilon \in \text{FIRST}(\alpha)$ e $\$ \in \text{FOLLOW}(A)$ allora mettiamo $A \rightarrow \alpha$ in $\mathbf{M}[A, \$]$.

Consideriamo la grammatica per cui abbiamo calcolato FIRST e FOLLOW ($E \rightarrow TA$, $A \rightarrow +TA \mid \epsilon$, $T \rightarrow FB$, $B \rightarrow *FB \mid \epsilon$ e $F \rightarrow (E) \mid n$). Otteniamo la tabella

	n	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TA$			$E \rightarrow TA$		
A		$A \rightarrow +TA$			$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
T	$T \rightarrow FB$			$T \rightarrow FB$		
B		$B \rightarrow \epsilon$	$B \rightarrow *FB$		$B \rightarrow \epsilon$	$B \rightarrow \epsilon$
F	$F \rightarrow n$			$F \rightarrow (E)$		

e usiamola per derivare $n + n$. Sull'input abbiamo $n + n\$$ e sulla pila $E\$$. Guardando la prima colonna abbiamo che applichiamo $E \rightarrow TA$ e sulla pila avremo $TA\$$, ora guardando ancora la tabella dobbiamo applicare $T \Rightarrow FB$, ottenendo sulla pila $FBA\$$. Ora in cima alla pila abbiamo F e $F \Rightarrow n$ sta in $\mathbf{M}[F, n]$, quindi mettiamo sulla pila n al posto di F e a questo punto possiamo consumare n

dall'input e rimaniamo con $+n\$$ sul nastro d'input. Ora sulla pila c'è $BA\$$ e sull'input il $+$, quindi, dato che $\mathbf{M}[B, +]$ contiene $B \rightarrow \epsilon$ facciamo sostanzialmente pop dalla pila, e siamo nella situazione in cui la pila è $A\$$. Grazie alla tabella avremo $+TA\$$ sulla pila e a questo punto possiamo consumare l'input e la testa della pila, quindi avremo sull'input $n\$$ e sulla pila $TA\$$. Come prima avremo sulla pila $FBA\$$ e sull'input $n\$$, e la pila poi diventa $nBA\$$. A questo punto consumiamo n sull'input e sulla pila ottenendo $\$$ sul nastro di input e $BA\$$ sulla pila. Guardando la tabella si vede che dobbiamo applicare $B \rightarrow \epsilon$ e $A \rightarrow \epsilon$ che sono nella tabella alla colonna $\$$. Facciamo sostanzialmente pop due volte ottenendo $\$$ sulla pila e $\$$ sul nastro di input. Consumiamo il $\$$ sulla pila e sull'input e accettiamo per pila vuota.

Se guardiamo la derivazione abbiamo costruito la seguente derivazione:

$$E \Rightarrow TA \Rightarrow FBA \Rightarrow nBA \Rightarrow nA \Rightarrow n + TA \Rightarrow n + FBA \Rightarrow n + nBA \Rightarrow n + nA \Rightarrow n + n$$

La costruzione della tabella di parsing in cui ogni casella ha al più una produzione funziona solo con le grammatiche LL(1) che sono tali che se ci sono due produzioni $A \rightarrow \alpha$ e $A \rightarrow \beta$ allora

- $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$,
- se $\epsilon \in \text{FIRST}(\alpha)$ allora $\text{FIRST}(\beta)$ e $\text{FOLLOW}(\beta)$ sono disgiunti ($\epsilon \in \text{FIRST}(\alpha)$ significa che $\alpha \xRightarrow{*} \epsilon$, e quindi β non deriva alcuna stringa che inizia con un terminale in $\text{FOLLOW}(\beta)$), e
- se $\epsilon \in \text{FIRST}(\beta)$ allora $\text{FIRST}(\alpha)$ e $\text{FOLLOW}(\alpha)$ sono disgiunti (simmetrica alla precedente).

Osserviamo che se la grammatica è ricorsiva sinistra o ambigua, la tabella ha caselle con più produzioni. Per esempio, la grammatica $S \rightarrow iEtSA \mid a, A \rightarrow eS \mid \epsilon, E \rightarrow b$ è ambigua. La tabella è

	a	b	i	e	$\$$
S	$S \rightarrow a$			$s \rightarrow iEtSB$	
A			$A \rightarrow \epsilon, A \rightarrow eS$		$A \rightarrow \epsilon$
E		$E \rightarrow b$			

che contiene due produzioni nella stessa casella.

6.6.2 Parser Bottom-Up

Nel parser top-down si cerca di costruire un albero sintattico a partire dalla radice, in quello *bottom-up* invece si cerca di costruire l'albero sintattico iniziando dalle foglie e salendo verso la radice. Prendiamo la grammatica $E \rightarrow T + E \mid T, T \rightarrow n * T \mid n \mid (E)$ e deriviamo $n + n * n$. La derivazione rightmost è:

$$E \Rightarrow T + E \Rightarrow T + T \Rightarrow T + n * T \Rightarrow T + n * n \Rightarrow n + n * n$$

L'algoritmo naive per il parsing bottom-up è questo Sia w la stringa in input

- prendi una sottostringa non vuota β di w
- cerca una produzione $X \rightarrow \beta$
- rimpiazza β con X in w
- se non ci fosse alcun β tale che $X \rightarrow \beta$ è una produzione, torna indietro
- ripeti fino a che non si raggiunge il simbolo S oppure non sono state provate tutte le possibilità

Come nel caso del parsing top-down vorremo fare previsioni e capire in generale quale sottostringa ridurre e quale produzione usare.

Definizione 6.37 Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ ed una parola $w \in \mathcal{L}(G)$. Una maniglia (handle) è una sottostringa β tale che $X \rightarrow \beta$ è una produzione che corrisponde ad un passo in una derivazione rightmost della stringa w .

Osserviamo che la sottostringa più a sinistra che coincide con la parte destra di una produzione non è sempre una maniglia.

L'idea di parsing è quella di visitare l'input da sinistra a destra cercando dove ridurre, quindi cercando una maniglia. Quello che faremo è dividere l'input con l'ausilio di una *barretta* che sostanzialmente indica cosa è già stato visto. Osserviamo anche che se $\alpha\beta\gamma$ è la stringa corrente durante un parsing bottom-up e la riduzione successiva è fatta usando $X \rightarrow \beta$, allora γ è una stringa di soli terminali dato che la derivazione è rightmost una derivazione rightmost. La parola viene quindi divisa in due sottostringhe dove la sottostringa di destra deve essere ancora esaminata (ed è una stringa di terminali) mentre la sottostringa di sinistra ha terminali e non terminali (ed è messa su una pila). Inizialmente, tutto l'input è *non esaminato*: $|x_1x_2 \dots x_n$

Nel parsing bottom-up abbiamo quindi due operazioni: SHIFT (scorri) e REDUCE (riduci). Lo SHIFT muove '|' di un posto verso destra e sposta un terminale nella stringa di sinistra (legge un terminale e lo mette sulla pila), quindi se abbiamo $ABC|xyz$ e faccio SHIFT ottengo $ABCx|yz$. L'operazione di REDUCE applica una produzione inversa nella parte finale della stringa di sinistra (rimpiazza una stringa di terminali in cima alla pila con la parte sinistra di una produzione). Se $A \rightarrow xy$ è una produzione e ho $CBA|ijk$ con la REDUCE ottengo $CBA|ijk$.

Come decidere quando fare shift o reduce? Per evidenziare la natura del problema consideriamo la solita grammatica $E \rightarrow T + E \mid T, T \rightarrow n * T \mid n \mid (E)$, e immaginiamo di essere in questa situazione: $n| * n + n$. Se decidessimo di usare $T \rightarrow n$ per ottenere $T| * n + n$ non riusciremmo poi ad arrivare ad E , mentre noi vogliamo ridurre solo se il risultato può ancora essere ridotto fino al simbolo iniziale.

Prendiamo una derivazione da destra $S \xRightarrow{*} \alpha X \gamma \Rightarrow \alpha\beta\gamma$, allora $\alpha\beta$ è una *maniglia* di $\alpha\beta\gamma$. Quindi nel parsing shift-reduce, le maniglie appaiono solo in cima alla pila, mai nel mezzo di questa. Vale la pena osservare che le maniglie non si trovano mai alla sinistra del non terminale più a destra, e quindi non dobbiamo mai muovere verso sinistra il simbolo '|'.

Dobbiamo quindi riconoscere le maniglie. Sappiamo che non è ovvio come trovare le maniglie, ricorriamo alla nozione di *prefisso valido*.

Definizione 6.38 Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ ed una derivazione rightmost per w , diciamo che $\alpha \in (\mathbf{V} \cup \Sigma)^*$ è un prefisso valido se $S \xRightarrow{*} \alpha\gamma \xRightarrow{*} w$. Quindi $\alpha|\gamma$ è uno stato di un parser shift-reduce.

Un prefisso valido non va oltre la fine destra di una maniglia, e un prefisso valido è un prefisso della maniglia, infine finchè ci sono prefissi validi sulla pila, non sono stati visti errori di parsing.

Il risultato che ci aiuterà è il seguente:

Teorema 6.20 Per ogni grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ che genera un linguaggio context-free deterministico, il linguaggio $\{\alpha \in (\mathbf{V} \cup \Sigma)^* \mid S \xRightarrow{*} \alpha\gamma \xRightarrow{*} w \text{ con } w \in \mathcal{L}(G)\}$ è un linguaggio regolare.

Mostriamo come costruire un FA che accetta i prefissi validi.

Definizione 6.39 Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ ed una produzione $A \rightarrow \alpha$, un item è una produzione $A \rightarrow \beta.\gamma$ con $\alpha = \beta\gamma$.

Quindi un item è una produzione con un “.” nella sua parte destra. Se abbiamo $T \rightarrow (E)$ gli item a: $T \rightarrow .(E) \mid (.E) \mid (E.) \mid (E)$.

L'intuizione è che un item indica quanto abbiamo visto di una produzione fino al “punto”. $A \rightarrow .XYZ$ indica che speriamo di vedere nell'input una stringa derivabile da XYZ , $A \rightarrow X.YZ$ indica che abbiamo già visto una stringa derivabile da X e che speriamo di vedere una derivabile da YZ nell'input, e $A \rightarrow XYZ.$ indica che abbiamo già visto una stringa derivabile da XYZ e che potrebbe essere il momento di ridurre XYZ ad A .

Gli item sono usati per costruire un NFA che viene usato per prendere decisioni durante il parsing (è l'automa LR(0)). E l'intuizione è che ogni stato dell'automa sia un insieme di items.

Definizione 6.40 Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$, con Item_G indichiamo l'insieme di items ottenuti da produzioni in $\mathcal{P}$.

Definizione 6.41 Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ ed un sia $I \subset \text{Item}_G$, Con $\text{CLOSURE}(I)$ indichiamo il più piccolo sottoinsieme di Item_G tale che $I \subset \text{CLOSURE}(I)$ e se $A \rightarrow \alpha.B\beta \in \text{CLOSURE}(I)$ e $B \rightarrow .\gamma \in \text{Item}_G$ allora $B \rightarrow .\gamma \in \text{CLOSURE}(I)$.

Se $A \rightarrow \alpha.B\beta \in \text{CLOSURE}(I)$ e pensiamo di poter vedere in futuro una sottostringa derivabile da $B\beta$ come input, questa sottostringa avrà un prefisso derivabile da B , per cui dobbiamo aggiungere gli item $B \rightarrow .\gamma$ per tutte le $B \rightarrow \gamma \in \text{Item}_G$

Consideriamo la solita grammatica $E \rightarrow T + E \mid T, T \rightarrow n * T \mid n \mid (E)$ e l'input (n) . Supponiamo che (E) sia uno stato di un parser shift-reduce. (E è un prefisso della parte destra di $T \rightarrow (E)$ e sarà ridotto dopo il prossimo shift. L'item $T \rightarrow (E.)$ dice che finora abbiamo visto $(E$ di questa produzione e speriamo di vedere $.)$. Il problema nel riconoscere prefissi validi è che la pila ha solo alcune parti della parte destra di una produzione (se avesse una parte destra completa, potremmo ridurre), ma queste parti sono sempre prefissi della parte destra di una produzione.

Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$, dato che la variabile S può apparire nel lato destro di qualche produzione, la aumentiamo aggiungendo un nuovo simbolo di start S' ed una produzione $S' \rightarrow S$ ottenendo la grammatica $(\mathbf{V} \cup \{S'\}, \Sigma, \mathcal{P} \cup \{S' \rightarrow S\}, S')$. L'insieme di items della nuova grammatica continuiamo ad indicarli con Items . Quindi nel caso della grammatica $E \rightarrow T + E \mid T, T \rightarrow n * T \mid n \mid (E)$ aggiungiamo $S \rightarrow E$. Consideriamo $I = \{S \rightarrow .E\}$. Allora $\text{CLOSURE}(I) = \{S \rightarrow .E, E \rightarrow .E + T, E \rightarrow .T, T \rightarrow .n * T, T \rightarrow .n, T \rightarrow .(E)\}$.

La pila può contenere molti prefissi di parti destre di produzioni. Siano questi prefissi $\text{pref}_1, \text{pref}_2, \dots, \text{pref}_n$ e sia pref_i un prefisso della parte destra di $X_i \rightarrow \alpha_i$, allora pref_i verrà prima o poi ridotto a X_i , la parte mancante di α_{i-1} inizia con X_i , e ci sarà un $X_{i-1} \rightarrow \text{pref}_{i-1} X_i \beta$ per qualche β . Abbiamo che ricorsivamente, $\text{pref}_{k+1} \dots \text{pref}_n$ prima o poi verrà ridotto alla parte mancante di α_k .

Consideriamo la stringa $(n * n)$ e sia $(n * | n)$ uno stato in del parser shift-reduce. Abbiamo che “(” è un prefisso della parte destra di $T \rightarrow (E)$, “ε” è un prefisso della parte destra di $E \rightarrow T$, e “ $n *$ ” è un prefisso della parte destra di $T \rightarrow n * T$.

Se uno stato dell'automa è identificato dalla chiusura di un sottoinsieme di items, dobbiamo capire come andare da uno stato ad un altro, dobbiamo insomma costruire la δ del nostro automa.

Definizione 6.42 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica, I un sottoinsieme di items di $(\mathbf{V} \cup \{S'\}, \Sigma, \mathcal{P} \cup \{S' \rightarrow S\}, S')$ ed $X \in \mathbf{V} \cup \{S'\} \cup \Sigma$ un simbolo della grammatica (terminale e non terminale), allora $\text{GOTO}(I, X) = \text{CLOSURE}(I')$ dove $I' = \{A \rightarrow \alpha.X\beta \mid A \rightarrow \alpha.X\beta \in I\}$.

Quindi se gli stati sono $\text{CLOSURE}(I)$ con I insieme di items ottenuti dalle produzioni, la transizione nell'automa sarà $\delta(\text{CLOSURE}(I), X) = \text{GOTO}(\text{CLOSURE}(I), X)$. L'automa deve riconoscere prefissi quindi è ovvio quale è il suo alfabeto: $\mathbf{V} \cup \Sigma$.

Consideriamo la solita grammatica $E \rightarrow T + E \mid T, T \rightarrow n * T \mid n \mid (E)$. $\text{GOTO}(\{T \rightarrow \cdot(E)\}, () = \text{CLOSURE}(\{T \rightarrow \cdot(E)\}) = \{T \rightarrow \cdot(E), E \rightarrow \cdot T + E, E \rightarrow \cdot T, T \rightarrow \cdot n * T, T \rightarrow \cdot n, T \rightarrow \cdot(E)\}$.

Possiamo quindi definire l'automa associato ad una grammatica.

Definizione 6.43 Sia data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$, e siano Items_G gli items della grammatica $(\mathbf{V} \cup \{S'\}, \Sigma, \mathcal{P} \cup \{S' \rightarrow S\}, S')$. Allora $\mathcal{A}_G$ è l'automa a stati finiti $(Q, \mathbf{V} \cup \Sigma \cup \{\epsilon\}, \delta, \text{CLOSURE}(\{S' \rightarrow \cdot S\}), Q)$ dove $Q = \{\text{CLOSURE}(I) \mid I \subseteq \text{Items}_G\}$ e $\delta(C, a) = \text{GOTO}(C, a)$ per ogni $a \in \mathbf{V} \cup \Sigma \cup \{\epsilon\}$ e $C \in Q$.

La prima osservazione da fare è che non tutti gli stati sono raggiungibili dallo stato iniziale, quindi possiamo anche costruirlo per raggiungibilità dallo stato iniziale.

Come usare l'automa nel parsing shift-reduce? Ogni stato corrisponde ad un simbolo della grammatica, che è quello delle transizioni che portano a lui, possiamo mettere gli stati raggiunti in una pila (ultimo stato in cima). Supponiamo che la stringa γ di simboli della grammatica porti l'automa dallo stato iniziale ad uno stato j . Facciamo SHIFT se per il prossimo simbolo in input a c'è una transizione da j etichettata a , e aggiungiamo il prossimo stato sulla pila, altrimenti REDUCE usando la produzione $A \rightarrow \alpha$ se nello stato j c'è l'item $A \rightarrow \alpha \cdot$, ed eliminiamo tanti stati dalla pila quanti sono i simboli di α , e dallo stato dove arriviamo vediamo dove una transizione etichettata A ci porta, mettendo il nuovo stato sulla pila.

Vediamo un esempio, l'automa per la grammatica classica di $a^n b^n$ con $n > 0$. Ovviamente mettiamo solo gli stati raggiungibili dell'automa. Consideriamo la grammatica $G = (\{S\}, \{a, b\}, \{S \rightarrow aSb, S \rightarrow ab\}, S)$. Allora gli items sono $\text{Items}_G = \{S' \rightarrow \cdot S, S' \rightarrow S \cdot, S \rightarrow \cdot aSb, S \rightarrow a \cdot Sb, S \rightarrow aS \cdot b, S \rightarrow aSb \cdot, S \rightarrow \cdot ab, S \rightarrow a \cdot b, S \rightarrow ab \cdot\}$, lo stato iniziale è $\text{CLOSURE}(\{S' \rightarrow \cdot S\}) = \{S' \rightarrow \cdot S, S \rightarrow \cdot aSb, S \rightarrow \cdot ab\}$ che è q_0 e l'alfabeto è $\{a, b, S\}$.

Chiamiamo $q_0 = \{S' \rightarrow \cdot S\}$, $q_1 = \{S \rightarrow S \cdot\}$, $q_2 = \{S \rightarrow a \cdot Sb, S \rightarrow a \cdot b, S \rightarrow aS \cdot b, S \rightarrow aSb \cdot\}$, $q_3 = \{S \rightarrow aSb \cdot\}$, $q_4 = \{S \rightarrow ab \cdot\}$ e $q_5 = \{S \rightarrow aSb \cdot\}$. Abbiamo che la δ è

$$\begin{aligned} \text{GOTO}(q_0, S) &= \text{CLOSURE}(\{S' \rightarrow S \cdot\}) = \{S' \rightarrow S \cdot\} \\ \text{GOTO}(q_0, a) &= \text{CLOSURE}(\{S \rightarrow a \cdot Sb, S \rightarrow a \cdot b\}) \\ &= \{S \rightarrow a \cdot Sb, S \rightarrow a \cdot b, S \rightarrow aS \cdot b, S \rightarrow aSb \cdot\} \\ \text{GOTO}(q_2, S) &= \text{CLOSURE}(\{S \rightarrow aS \cdot b\}) = \{S \rightarrow aS \cdot b\} \\ \text{GOTO}(q_2, a) &= \text{CLOSURE}(\{S \rightarrow a \cdot Sb, S \rightarrow a \cdot b\}) \\ &= \{S \rightarrow a \cdot Sb, S \rightarrow a \cdot b, S \rightarrow aS \cdot b, S \rightarrow aSb \cdot\} \\ \text{GOTO}(q_2, b) &= \text{CLOSURE}(\{S \rightarrow ab \cdot\}) = \{S \rightarrow ab \cdot\} \\ \text{GOTO}(q_3, b) &= \text{CLOSURE}(\{S \rightarrow aSb \cdot\}) = \{S \rightarrow aSb \cdot\} \end{aligned}$$

e per quegli stati e simboli che non compaiono abbiamo che $\text{GOTO}(q_0, b) = \text{GOTO}(q_1, S) = \text{GOTO}(q_1, a) = \text{GOTO}(q_3, a) = \text{GOTO}(q_4, S) = \text{GOTO}(q_4, a) = \text{GOTO}(q_4, b) = \text{GOTO}(q_5, S) = \text{GOTO}(q_5, a) = \text{GOTO}(q_5, b) = \emptyset$.

Quali sono le parole accettate dall'automa? $\epsilon, S, a^n b, a^n Sb$ con $n \geq 1$, che sono tutti prefissi di $a^n Sb^n$ per un qualche $n \geq 0$.

Ricordiamo come funziona il parser: assumiamo che la pila contenga α , che il prossimo simbolo da leggere sia t e che il DFA sia nello stato q , allora facciamo

REDUCE usando $X \rightarrow \beta$ se q contiene $X \rightarrow \beta$.

SHIFT se q contiene $X \rightarrow \beta \cdot t \gamma$ (e quindi abbiamo una transizione da q ad uno stato che contiene $X \rightarrow \beta \cdot t \gamma$).

Per farlo definiamo la tabella **action** $[q, a]$, dove la **action** $[q_i, a]$ contiene l'azione che deve effettuare il parser tra "SHIFT j ", "REDUCE $X \rightarrow \beta$ ", "accept" e "error".

Se il parser è nella condizione $\alpha|t\gamma$, l'automa si trova nello stato q_i , che è in cima alla pila degli stati, e **action** $[i, t]$ contiene SHIFT j , allora si va nello stato q_j dell'automa (che si mette nella pila degli stati) e si sposta t mettendolo sulla pila, quindi lo stato del parser sarà $\alpha t|\gamma$.

Se il parser è nella condizione $\alpha\beta|t\gamma$, l'automa si trova nello stato q_i e **action** $[i, t]$ contiene REDUCE $X \rightarrow \beta$, allora si mette X sulla pila al posto di β , si va nello stato q_j dell'automa che si determina levando dalla pila degli stati esattamente $|\beta|$ elementi e si mette lo stato che si ottiene da GOTO(q, X), dove q è lo stato in cima alla pila degli stati dopo aver levato $|\beta|$ elementi, quindi lo stato del parser sarà $\alpha X|t\gamma$.

Se il parser è condizione $S|\epsilon$, l'automa si trova nello stato q_i e **action** $[i, \epsilon]$ contiene *accept*, allora si accetta.

Se il parser è nella condizione $\alpha|t\gamma$, l'automa si trova nello stato q_i e **action** $[i, t]$ contiene *error*, allora non si accetta.

Come riempire **action** $[i, t]$? Per prima cosa osserviamo che non abbiamo bisogno di mantenere la pila degli stati dell'automa. Infatti, se $\alpha|\gamma$ è lo stato del parser, la pila degli stati la ottengo riconoscendo α usando l'automa. Poi modifichiamo leggermente la nozione di stato del parser: $\alpha|\gamma\$$ (aggiungiamo un delimitatore alla fine).

La casella **action** $[i, a]$ della tabella sarà:

action $[i, a]$, con $a \in \Sigma$, contiene "SHIFT j " se GOTO(q_i, a) = q_j . Osserviamo che se GOTO(q_i, a) = q_j allora q_i contiene un item $A \rightarrow \alpha.a\beta$ e q_j l'item $A \rightarrow \alpha a.\beta$

action $[i, a]$, con $a \in \Sigma$, contiene "REDUCE $A \rightarrow \beta$ " se q_i contiene un item $A \rightarrow \beta$.

action $[i, \$]$ contiene *accept* se q_i contiene $S' \rightarrow S$.

in tutti gli altri casi, **action** $[i, a]$ contiene *error*.

La tabella **action** per la grammatica vista prima è

	a	b	$\$$
0	SHIFT 2	<i>error</i>	<i>error</i>
1	<i>error</i>	<i>error</i>	<i>accept</i>
2	SHIFT 2	SHIFT 4	<i>error</i>
3	<i>error</i>	SHIFT 5	<i>error</i>
4	<i>error</i>	Reduce $S \rightarrow ab$	<i>error</i>
5	<i>error</i>	Reduce $S \rightarrow aSb$	<i>error</i>

Possiamo costruire un'altra tabella: **goto** $[i, A]$. Infatti quando facciamo le riduzioni dobbiamo anche trovare lo stato corretto in cui *mettere* l'automa. Mentre la tabella **action** la costruiamo per i terminali, la tabella **goto** la costruiamo per i non terminali (che corrispondono a delle riduzioni)

Nel nostro esempio abbiamo che **goto** $[4, S] = 3$. Infatti il parser si trova in questa situazione: $a^n b|b^{n-1}\$$ e **action** $[4, b]$ contiene REDUCE $S \rightarrow ab$, quindi si rimuove ab da $a^n b$, si aggiunge S e si determina lo stato eseguendo GOTO($q_0, a^{n-1}S$) = q_3 . **goto** $[5, S] = 3$: Il parser si trova in questa situazione: $a^n S b|b^{n-1}\$$ e **action** $[5, b]$ contiene REDUCE $S \rightarrow aSb$, quindi si rimuove aSb da $a^n S b$, si aggiunge S e si determina lo stato eseguendo GOTO($q_0, a^{n-1}S$) = q_3 . Tutte le altre caselle della tabella sono vuote.

Questo schema di parsing può avere dei problemi, legati a situazioni in cui non si sa decidere: o un conflitto *shift/reduce* se lo stato q_i contiene gli item $X \rightarrow \alpha.a\beta$ e $X \rightarrow \gamma$. oppure un conflitto *reduce/reduce* se lo stato q_i contiene gli item $X \rightarrow \alpha$. e $X \rightarrow \gamma$.

Una grammatica è SLR se non ci sono conflitti (S sta per simple)

Si può cercare di eliminare i conflitti? Quando abbiamo fatto le riduzioni non ci siamo preoccupati affatto di quali possano essere le prossime maniglie, e lo potremmo fare usando il FOLLOW. **action** $[i, a]$, con $a \in \Sigma$, contiene "REDUCE $X \rightarrow \beta$ " se q_i contiene un item $X \rightarrow \beta$. e $a \in \text{FOLLOW}(X)$ (ricordiamo

che $\text{FOLLOW}(X)$ contiene cosa posso derivare a destra di X) Quindi ora faccio una riduzione solo nel caso in cui dopo quella posso continuare a ridurre.

Vediamo un esempio. Consideriamo $S \rightarrow S + S \mid S * S \mid (S) \mid n$ (è la solita grammatica ambigua) e gli stati dell'automa sono

- $q_0 = \{S' \rightarrow .S, S \rightarrow .S + S, S \rightarrow .S * S, S \rightarrow .(S), S \rightarrow n\}$
- $q_1 = \{S' \rightarrow S.\}$
- $q_2 = \{S' \rightarrow n.\}$
- $q_3 = \{S \rightarrow (.S), S \rightarrow .S + S, S \rightarrow .S * S, S \rightarrow .(S), S \rightarrow n\}$
- $q_4 = \{S \rightarrow S. + S, S \rightarrow S. * S\}$
- $q_5 = \{S \rightarrow S + .S, S \rightarrow .S + S, S \rightarrow .S * S, S \rightarrow .(S), S \rightarrow n\}$
- $q_6 = \{S \rightarrow S * .S, S \rightarrow S * .S, S \rightarrow .S + S, S \rightarrow .S * S, S \rightarrow .(S), S \rightarrow n\}$
- $q_7 = \{S \rightarrow S + S., S \rightarrow S. + S, S \rightarrow S. * S\}$
- $q_8 = \{S \rightarrow S * S., S \rightarrow S. + S, S \rightarrow S. * S\}$
- $q_9 = \{S' \rightarrow (S.)\}$
- $q_{10} = \{S' \rightarrow (S). \}$

Ora lo stato q_7 (e anche q_8) contiene sia $S \rightarrow S + S$ che da un reduce, e $S \rightarrow S * S$ che da un shift. Se mettiamo le precedenze trasformiamo la grammatica in una non ambigua.

Posso fare meglio, ma prima ricordiamo che ogni grammatica LL è anche LR, ogni grammatica LR(0) è non ambigua e il linguaggio è accettato da un automa deterministico, e il parsing LR funziona anche con grammatiche ambigue.

Parser LR(1)

In questo caso l'uni tra parentesi indica il *lookahead*, di cui teniamo conto costruendo sia l'automa che la tabella action. Come calcoliamo gli item LR(1)? Gli item sono ora $[A \rightarrow \alpha.\beta, a]$ dove ora a è un terminale o \$. Gli $\text{Items}(G)^1$ si ottengono così: dato $[A \rightarrow \alpha.B\beta, a] \in \text{Items}(G)^1$ e data la produzione $B \rightarrow \gamma$, si aggiunge $[B \rightarrow .\gamma, b]$ per ogni terminale in $\text{FIRST}(\beta a)$. CLOSURE è sempre la chiusura di un insieme di items mentre $\text{GOTO}(I, X)$ sarà l'insieme di item I' della forma $[A \rightarrow \alpha X.\beta, a]$ tali che l'item $[A \rightarrow \alpha.X\beta, a]$ sta in I .

Un parser SLR(1) è un parser in cui la costruzione della tabella **action** non genera conflitti